UNSW SYDNEY

Efficient and Precise Pointer Analysis for Rust

Author:
Li, Wei

Publication Date:
2025

DOI:
https://doi.org/10.26190/unsworks/31274

License:
https://creativecommons.org/licenses/by/4.0/
Link to license to see what you are allowed to do with this resource.

Downloaded from http://hdl.handle.net/1959.4/105272 in https://unsworks.unsw.edu.au on 2026-04-20

Efficient and Precise Pointer Analysis for Rust

by

Wei Li

A THESIS IN FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

June 29, 2025

School of Computer Science and Engineering
Faculty of Engineering
The University of New South Wales

All rights reserved. This work may not be reproduced in any form without the permission of the author. © Wei Li 2025

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract	Declarations	Inclusion of Publications Statement	Corrected Thesis and Responses

ORIGINALITY STATEMENT

☑ I hereby declare that this submission is my own work and to the best of my knowledge it contains no materials previously published or written by another person, or substantial proportions of material which have been accepted for the award of any other degree or diploma at UNSW or any other educational institution, except where due acknowledgement is made in the thesis. Any contribution made to the research by others, with whom I have worked at UNSW or elsewhere, is explicitly acknowledged in the thesis. I also declare that the intellectual content of this thesis is the product of my own work, except to the extent that assistance from others in the project's design and conception or in style, presentation and linguistic expression is acknowledged.

COPYRIGHT STATEMENT

☑ I hereby grant the University of New South Wales or its agents a non-exclusive licence to archive and to make available (including to members of the public) my thesis or dissertation in whole or part in the University libraries in all forms of media, now or here after known. I acknowledge that I retain all intellectual property rights which subsist in my thesis or dissertation, such as copyright and patent rights, subject to applicable law. I also retain the right to use all or part of my thesis or dissertation in future works (such as articles or books).

For any substantial portions of copyright material used in this thesis, written permission for use has been obtained, or the copyright material is removed from the final public version of the thesis.

AUTHENTICITY STATEMENT

☑ I certify that the Library deposit digital copy is a direct equivalent of the final officially approved version of my thesis.

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract	Declarations	Inclusion of Publications Statement	Corrected Thesis and Responses

UNSW is supportive of candidates publishing their research results during their candidature as detailed in the UNSW Thesis Examination Procedure.

Publications can be used in the candidate's thesis in lieu of a Chapter provided:

- The candidate contributed **greater than 50%** of the content in the publication and are the "primary author", i.e. they were responsible primarily for the planning, execution and preparation of the work for publication.
- The candidate has obtained approval to include the publication in their thesis in lieu of a Chapter from their Supervisor and Postgraduate Coordinator.
- The publication is not subject to any obligations or contractual agreements with a third party that would constrain its inclusion in the thesis.

☑ The candidate has declared that **their thesis has publications - either published or submitted for publication - incorporated into it in lieu of a Chapter/s. Details of these publications are provided below.**.

Publication Details #1

Full Title:	A Context-Sensitive Pointer Analysis Framework for Rust and Its Application to Call Graph Construction
Authors:	Wei Li, Dongjie He, Yujiang Gui, Wenguang Chen, Jingling Xue
Journal or Book Name:	33rd ACM SIGPLAN International Conference on Compiler Construction (CC'24)
Volume/Page Numbers:	
Date Accepted/Published:	
Status:	published
The Candidate's Contribution to the Work:	I have designed and implemented the whole work.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	I have included this publication in Chapter 3.

Publication Details #2

Full Title:	Stack Filtering: Elevating Precision and Efficiency in Rust Pointer Analysis
Authors:	Wei Li, Dongjie He, Wenguang Chen, Jingling Xue
Journal or Book Name:	23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO'25)
Volume/Page Numbers:	
Date Accepted/Published:	
Status:	published
The Candidate's Contribution to the Work:	I have designed and implemented the whole work.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	I have included this publication in Chapter 3.

Candidate's Declaration

I confirm that where I have used a publication in lieu of a chapter, the listed publication(s) above meet(s) the requirements to be included in the thesis. I also declare that I have complied with the Thesis Examination Procedure.

Abstract

Rust, a rapidly growing system-level language, has gained popularity for its strong memory safety guarantees. However, its support for unsafe code, which bypasses built-in safety checks, introduces risks similar to those in C and C++, underscoring the ongoing need for program analysis to address safety concerns in Rust programs.

Although various program analysis tools have been developed, many fall short in effectively detecting safety issues in Rust due to imprecise call graph and alias/points-to information. Therefore, pointer analysis, a technique for identifying the possible memory locations pointed to by pointers and aiding in the construction of accurate call graph, is urgently needed for improving the effectiveness of safety analysis in Rust programs.

This thesis studies pointer analysis for Rust, focusing on the development of an effective analysis framework and advancements in its precision and efficiency.

- This thesis presents Rupta, the first context-sensitive pointer analysis framework functioning on Rust MIR. Rupta tackles various challenges in achieving precise points-to information and accurate call graphs, including supporting various function calls, correctly modelling type semantics associated with pointer casts and nested structs, and handling a complex memory model.

- This thesis presents *stack filtering*, a novel technique that leverages the liveness information of stack objects to eliminate spurious points-to relations

involving inactive stack objects. This technique notably enhances both the efficiency and precision of pointer analysis.

We extensively evaluate RUPTA and stack filtering on real-world Rust applications. The experimental results demonstrate that RUPTA is both efficient and capable of generating precise points-to information and accurate call graphs. Stack filtering further enhances RUPTA's effectiveness by improving analysis precision while significantly reducing time and memory usage. RUPTA, integrated with stack filtering, is expected to enhance existing security analysis tools, enabling them to identify a greater number of security vulnerabilities in Rust programs.

Publications

- **Wei Li**, Dongjie He, Wenguang Chen, and Jingling Xue. Stack Filtering: Elevating Precision and Efficiency in Rust Pointer Analysis. In *23rd IEEE/ACM International Symposium on Code Generation and Optimization (CGO'25)*.

- **Wei Li**, Dongjie He, Yujiang Gui, Wenguang Chen, and Jingling Xue. A Context-Sensitive Pointer Analysis Framework for Rust and Its Application to Call Graph Construction. In *33rd ACM SIGPLAN International Conference on Compiler Construction, (CC'24)*.

- Dongjie He, Yujiang Gui, **Wei Li**, Yonggang Tao, Changwei Zou, Yulei Sui and Jingling Xue. A Container-Usage-Pattern-Based Context Debloating Approach for Object-Sensitive Pointer Analysis. In *ACM SIGPLAN International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'23)*.

- Jingbo Lu, Dongjie He, **Wei Li**, Yaoqing Gao, Jingling Xue. Automatic Generation and Reuse of Precise Library Summaries for Object-Sensitive Pointer Analysis. In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE) (ASE'23)*.

- Jifeng Wu, **Wei Li**, Libing Wu, Mengting Yuan, Chun Jason Xue, Jingling Xue and Qingan Li. Effective Stack Wear Leveling for NVM. In *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD'23)*.

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my supervisor, Professor Jingling Xue, for his guidance and support throughout my PhD study. His mentorship has been a cornerstone of my development as a researcher, from navigating complex scientific challenges to honing my academic writing skills. His enthusiasm, diligence and commitment to excellence have profoundly impacted me, instilling principles that I will carry with me throughout my career.

I would also like to extend my heartfelt thanks to my master's supervisors, Associate Professor Qingan Li and Professor Mengting Yuan, whose mentorship laid the foundation for my research journey. Their early guidance and encouragement were crucial in sparking my interest in research. They also offered valuable advice as I transitioned into my Ph.D., which has greatly benefited my academic development.

I am also thankful to Associate Professor Yulei Sui for his pioneering work on the SVF framework and to my senior colleague Dr. Dongjie He for his development of the Qilin pointer analysis framework. Both provided excellent starting points for my own research and served as invaluable guides during the design and implementation of Rupta. I appreciate their significant contributions to the field.

I want to thank my labmates and colleagues from the PLC group—Changwei Zou, Jingbo Lu, Yujiang Gui, Yonggang Tao, Xudong Wang, Wenyao Chen, Hao

Qian, Yuechen Mu— as well as the friends I made at UNSW—Yao Liu, Chengkai Huang and many others. It has been a pleasure to work with you all, and I am thankful for the collaboration, sharing of knowledge and friendships we have built along the way, which made this experience fulfilling.

I am also grateful to Professor Ana Milanova and Professor Gang Tan for serving on my Examining Committee and offering valuable feedback.

Last but not least, I owe my deepest gratitude to my family for their unwavering support and love throughout my PhD journey. Their belief in me has been a constant source of strength. I would also like to thank my girlfriend, whom I was fortunate to meet during my Ph.D. years. Her encouragement and understanding have been a source of emotional support and immense comfort, helping me maintain balance through this challenging yet rewarding journey.

Contents

Abstract **iii**

Publications **v**

Acknowledgments **vii**

List of Figures **xiv**

List of Tables **xv**

List of Algorithms **xvii**

1 Introduction **1**

1.1 Challenges 3

1.2 Contribution 4

1.3 Organisation 5

2 Background **7**

2.1 Pointer Analysis 7

2.1.1 Andersen-style Pointer Analysis 8

2.1.2 Context-Sensitive Pointer Analysis 9

2.2 The Rust Programming Language 11

2.2.1 An Introduction to Rust 11
2.2.2 The Rust Language Model 16

3 RUPTA: A Context-Sensitive Pointer Analysis Framework for Rust 23

3.1 Overview 23
3.2 Motivation: Challenges and Insights 26
3.2.1 Analysing Calls 26
3.2.2 Modelling Pointer Casts 31
3.2.3 Representing an Object's Fields 33
3.3 RUPTA: Pointer Analysis for Rust 35
3.3.1 Andersen-Style Inclusion-Based Pointer Analysis 36
3.3.2 Callsite-Sensitive Pointer Analysis 40
3.3.3 Support for Rust's Language Features 41
3.3.4 Discussion 50
3.4 Implementation 50
3.5 Evaluation 51
3.5.1 RQ1: Is Rupta Scalable and Effective? 52
3.5.2 RQ2: Can Rupta Build Precise Call Graphs? 55
3.6 Summary 58

4 Stack Filtering: Elevating Precision and Efficiency in Rust Pointer Analysis 59

4.1 Overview 59
4.2 Motivation 62
4.2.1 Imprecision of Pointer Analysis 62
4.2.2 Cost of Achieving Context Sensitivity 64

4.2.3 Stack Filtering 66
4.3 Stack-Filtering-Enhanced Analysis 71
4.3.1 Rapid Type Analysis for Rust 72
4.3.2 Function Reachability Analysis 74
4.3.3 Pointer Analysis with Stack Filtering 78
4.3.4 Soundness 81
4.4 Evaluation 82
4.4.1 RQ1: Precision and Efficiency 83
4.4.2 RQ2: Reduction of Spurious Points-to Relations 88
4.4.3 RQ3: Effectiveness of RTA-based Pre-analysis 89
4.4.4 RQ4: Real-World Cases Study 90
4.5 Summary 92

5 Related Work **93**
5.1 Program Analysis for Rust 93
5.2 Pointer Analysis 95
5.2.1 Pointer Analysis Frameworks 95
5.2.2 Optimisations for Pointer Analysis 96

6 Conclusions and Future Work **99**
6.1 Conclusions 99
6.2 Future Work 101
6.2.1 Different Context Sensitivities for Rust Pointer Analysis . . 101
6.2.2 Flow-Sensitive Pointer Analysis for Rust 102
6.2.3 Demand-Driven Pointer Analysis for Rust 102
6.2.4 Leveraging Rust's Unique Features 103
6.2.5 Other Potential Directions 103

Bibliography **104**

List of Figures

2.1 A simplified MIR example. 17

2.2 A simplified Rust language model. 18

3.1 An example with static and dynamic dispatch. 27

3.2 Function pointers and `Fn` traits. 30

3.3 Definition of the `Fn` trait. 30

3.4 An example with a pointer cast. 32

3.5 An example with nested struct types. 34

3.6 Rules for Andersen-style inclusion-based pointer analysis. 38

3.7 Rules for k-callsite-sensitive pointer analysis (f_l is the function containing statement l being analysed). 42

3.8 An example calling for context-sensitivity. 55

4.1 A Rust program. 64

4.2 Computing the points-to information for `foo::p` and `bar::q` in the program given in Figure 4.1 by applying context-insensitive and 3-callsite-sensitive analyses. 64

4.3 A program illustrating the cost of context sensitivity. 65

4.4 Computing the points-to information for `foo::p` and `bar::q` for the program given in Figure 4.3 by applying 1-callsite-sensitive pointer analysis. 65

4.5 Allocations and deallocations of stack frames during the execution of the program given in Figure 4.1 (with calls to `id2()` and `id1()` omitted for brevity). 68
4.6 Workflow of stack-filtering-enhanced pointer analysis for Rust programs. 71
4.7 A formalisation of RTA for Rust. 73
4.8 Rules for computing reachability relations in a call graph $G = (R, E)$. 74
4.9 Rules for computing `is_alive`(o, c', f, c). 79
4.10 Rules for performing k-callsite-sensitive pointer analysis for Rust with stack filtering incorporated. In each rule, f_l is the function containing statement l being analysed. 80
4.11 A program returning a deallocated stack object. 82
4.12 Speedups of k`CS+SF` and k`CS+SF`$_s$ over k`CS`. 87
4.13 Percentages of points-to relations preserved, directly and indirectly eliminated by stack filtering. 89
4.14 Precision gains of k`CS+SF` and k`CS+SF`$_k$ over k`CS`. 90
4.15 A program involving array slicing. 91

List of Tables

2.1 Constraints for Andersen-style pointer analysis 8

2.2 Eight kinds of statements analysed in pointer analysis. 19

3.1 Rust projects (including #stars and #MIR statements) 52

3.2 Analysis times, memory usage, and average points-to sizes of RUPTA in four configurations (OOM: Out of memory). 53

3.3 Comparing call graphs built by different analyses 56

4.1 Comparison of efficiency and precision in k-callsite-sensitive pointer analyses ($k \in \{1, 2, 3\}$) with and without stack filtering. Except for "speedup" and "memory reduction" highlighted in red, smaller values indicate better performance. OOM stands for "Out of Memory". 84

4.2 Context-sensitive facts. For all the metrics, smaller is better. 86

List of Algorithms

1 $\texttt{dispatch}_{\texttt{C}}$ 43

2 static_dispatch 44

3 dynamic_dispatch 45

4 $\texttt{transmute}_{\texttt{C}}$ 47

Chapter 1

Introduction

Rust, a growing system-level programming language, is increasingly employed in low-level systems development, including operating systems [2, 18, 41, 71] and web browsers [4]. It divides a program into *safe code* and *unsafe code*. Through its innovative *ownership* model and rigorous type system, Rust achieves automatic memory management, eliminating memory safety bugs in the *safe code* at compile time. However, the Rust compiler bypasses safety checks in *unsafe code*, potentially leading to security vulnerabilities [30, 75, 105], similar to C/C++ programs. As a result, rigorous program analysis remains essential to address safety concerns in Rust's unsafe code regions.

Pointer analysis, a technique that determines the set of possible memory locations a pointer can point to and helps in constructing accurate call graphs, is crucial in identifying memory-related issues. Existing program analysis (and verification) tools [7, 10, 13, 54, 75, 97, 102] for Rust lack the ability to detect vulnerabilities due to the absence of a precise pointer analysis, hindering their bug detection capability. For instance, existing inter-procedural bug detection tools [10, 24, 54, 75] solely consider static calls at compile time, overlooking dynamic calls and thus hampering

their detection capabilities. Some vulnerability detection techniques [54, 107] rely on simple heuristics to compute alias information due to the lack of precise points-to information, with their authors acknowledging that precise pointer analysis could enhance precision. As highlighted in [107], the inability to detect eight critical bugs is attributed to two reasons: "First, our alias analysis is not precise...Second, our call-graph analysis is not good enough (e.g., unable to handle function pointers)."

Despite the essential need for accurate call graphs and precise points-to information for Rust programs, no existing pointer analysis framework is currently available. One might contend that LLVM-IR-based tools, such as SVF [88], could be employed, given that Rust source code is lowered down to LLVM IR prior to its translation into machine code. But the results can hardly be integrated with Rust analysis tools that primarily target Rust's Mid-level Intermediate Representation (MIR). Moreover, as demonstrated by existing studies [54, 97], LLVM IR loses considerable information, such as type data and the original program structure, which may lead to less precise and accessible analysis results. Therefore, it is crucial to develop a pointer analysis specifically targeting Rust MIR.

In this thesis, we explore pointer analysis on Rust MIR, focusing on its role in constructing call graphs. As a fundamental analysis supporting various other tools, the effectiveness of pointer analysis has a significant impact on the performance of these applications. The effectiveness is typically measured by two key metrics: precision and efficiency. A more precise pointer analysis reduces false bug reports, enhancing accuracy, while higher efficiency enables its use in time-sensitive applications and shortens turnaround times. We aim to develop an efficient and precise pointer analysis for Rust, but this is not trivial as discussed below.

1.1 Challenges

Currently, no pointer analysis framework exists for Rust. Developing one from scratch presents significant challenges and requires substantial engineering efforts, primarily due to certain complex language features. Simply porting existing solutions from C/C++ (e.g., SVF [88]) or Java (e.g., Qilin [39]) to Rust is not feasible for several reasons: Firstly, Rust supports various calls resolved through different mechanisms, making accurate call target computation more difficult. Secondly, the ubiquitous use of pointer casts in Rust programs, especially in the standard library, demands more complex handling to preserve type information accurately. Lastly, Rust's memory access paradigms are as intricate as those of C/C++, while its MIR operates at a higher abstraction level than LLVM IR, necessitating a proper memory model during analysis.

In addition, it is essential to consider both precision and efficiency in pointer analysis. A well-known pointer analysis algorithm is Andersen's analysis [3], which, while efficient and practical, tends to be imprecise. One primary reason for its lack of precision is its context-insensitive nature, meaning it does not differentiate between different calling contexts of functions. Instead, it merges points-to information from all possible call sites, leading to over-approximation. To improve precision, context-sensitive pointer analysis was introduced, which analyses functions separately under different calling contexts. To maintain scalability, these approaches typically limit the context length using a parameter k [39,65,66,80,82,84]. For instance, a k-callsite-sensitive pointer analysis [80, 82] use the sequence of k most recent callsites as the function's context. While context sensitivity improves precision, increasing the k-limit can significantly slow down the analysis and even render it unscalable. It has been a long-standing problem to improve the precision and efficiency of pointer analysis. Techniques typically aim to enhance efficiency

while maintaining or sacrificing precision, or conversely, to improve precision while maintaining or sacrificing efficiency [16, 36, 38, 47, 61, 63, 86, 92, 101, 104]. Finding ways to improve both at the same time remains a significant challenge.

1.2 Contribution

In this thesis, we present Rupta, the first context-sensitive pointer analysis framework designed for Rust, operating directly on Rust MIR. Additionally, we introduce *stack filtering*, a new technique that enhances both the efficiency and precision of pointer analysis. Specifically, this thesis makes the following contributions:

- The introduction of Rupta, the first context-sensitive pointer analysis framework designed specifically for Rust MIR.
  - Rupta addresses multiple challenges involved in achieving precise pointer analysis and call graph construction for Rust, including supporting various function calls, correctly modelling the type semantics associated with pointer casts and nested structs, and handling a complex memory model. This tool marks a significant milestone as the first of its kind to support pointer analysis for Rust.
  - We comprehensively evaluated Rupta using real-world Rust programs, demonstrating its high effectiveness in efficiently producing accurate call graphs and precise points-to information for Rust programs. Specifically, we implemented two additional call graph construction techniques, Rurta (Rapid Type Analysis-based) and Ruscg (static dispatch-only). Comparing Rupta with these two state-of-the-art techniques, we found that Rupta is more sound than Ruscg, and more precise than Rurta.

- The introduction of *stack filtering*, a novel technique to significantly improve both the efficiency and precision of pointer analysis for Rust.

  - Stack filtering eliminates a significant portion of spurious points-to relations in pointer analysis by leveraging stack object liveness information. Specifically, it filters out points-to relations involving variables pointing to inactive stack objects outside their lifetimes. To the best of our knowledge, this is the first time that lifetime information has been used to improve the effectiveness of pointer analysis.

  - We extensively evaluate stack filtering on real-world Rust programs. Experimental results demonstrate that stack filtering notably improves analysis efficiency, reducing both time and memory consumption. It also enhances precision, resulting in smaller points-to sets. The enhanced pointer analysis, augmented with stack filtering, is poised to become a vital tool for Rust's program analysis and verification, given Rust's reliance on automatic memory management and a preference for static stack allocation.

- We have implemented the Rupta framework, integrated with stack filtering, as an open-source tool, available at

  https://rustanlys.github.io/rupta/ .

1.3 Organisation

This thesis contains text and material from several publications:

- Our context-sensitive pointer analysis framework, RUPTA, presented in Chapter 3 is based on "A Context-Sensitive Pointer Analysis Framework for Rust and Its Application to Call Graph Construction" published in CC 2024.

- Our approach to enhancing the effectiveness of Rust pointer analysis, stack filtering, presented in Chapter 4, is based on "Stack Filtering: Elevating Precision and Efficiency in Rust Pointer Analysis" accepted in CGO 2025.

Lastly, the other chapters are organised as follows. Chapter 2 briefly reviews the background knowledge on pointer analysis and introduces a simplified Rust language model that will be used in our analysis. Chapter 5 discusses the related work of this thesis. And finally, Chapter 6 concludes the thesis and suggests directions for future research.

Chapter 2

Background

This chapter provides background information on pointer analysis for Rust. Section 2.1 reviews the inclusion-based pointer analysis, including the standard Andersen-style analysis [3] and context-sensitive pointer analysis. Section 2.2 gives a brief overview of the central features of the Rust type system, and introduces a simplified Rust language that will be used in our pointer analysis.

2.1 Pointer Analysis

Pointer analysis, essential for identifying memory locations pointed to by pointers in a program, is crucial for various forms of program analysis. Its applications span across compiler optimisation [85, 90], bug detection [24, 103, 106], security analysis [59, 89], program verification [31, 53, 54], and program understanding [53], among others. The efficiency and precision of pointer analysis significantly influence the performance of these diverse client applications.

Roughly, there are two ways to model and solve the pointer analysis problem: Andersen-style (inclusion-based) [3] and Steensgaard-style (unification-based) [87] pointer analyses. Steensgaard-style analysis is faster and more scalable, as it merges

Table 2.1: Constraints for Andersen-style pointer analysis

Category	Type	Statement	Constraints
Base	Address-of	$p = \&o$	$o \in \texttt{pts}(p)$
Simple	Assign	$p = q$	$\texttt{pts}(p) \supseteq \texttt{pts}(q)$
Complex	Load	$p = *q$	$\forall v \in \texttt{pts}(q) : \texttt{pts}(p) \supseteq \texttt{pts}(v)$
Complex	Store	$*p = q$	$\forall v \in \texttt{pts}(p) : \texttt{pts}(v) \supseteq \texttt{pts}(q)$

pointers into equivalence classes, but it sacrifices precision by treating multiple pointers similarly. In contrast, Andersen-style pointer analysis offers higher precision by maintaining distinct points-to sets for each pointer.

In this thesis, we focus on Andersen-style pointer analysis (also known as Andersen's analysis) due to its superior precision and broader adoption in both academic research and practical applications.

2.1.1 Andersen-style Pointer Analysis

Andersen-style pointer analysis abstracts programs into different types of constraints based on pointer assignments and operations. These constraints define the relationships between pointers and memory locations, and the analysis works by iteratively solving these constraints until a fixed point is reached. Let $\texttt{pts}(v)$ denote the points-to set of variable v. The core constraint rules in [3] are summarised in Table 2.1:

- **Address-of ($p = \&o$)**: If p is assigned the address of o, then p points to the memory location of o.

- **Assign ($p = q$)**: If pointer p is assigned the value of pointer q , then the points-to set of q (i.e., the memory locations q can point to) is included in the points-to set of p .

- **Load** ($p = *q$) If p is assigned the value of $*q$, then for every location v that q points to, the points-to set of v is included in the points-to set of p.

- **Store** ($*p = q$) If $*p$ is assigned the value of q , then for every location v that p points to, the points-to set of q is included in the points-to set of v .

2.1.2 Context-Sensitive Pointer Analysis

The standard Andersen-style pointer analysis is context-insensitive, while it provides efficiency and simplicity, it lacks precision due to its inability to distinguish between different calling contexts of functions. This limitation can lead to over-approximation, where multiple function calls and their effects on pointers are conflated, even when the function behaves differently in each context. Context-sensitive pointer analysis addresses this issue by considering the calling context in its analysis, allowing for more precise tracking of pointer relationships across function boundaries.

In context-sensitive pointer analysis, different invocations of the same function are treated as distinct if they occur in different calling contexts. This enables the analysis to differentiate between how pointers are used and propagated depending on the specific function call chain. For example, if a function is called from two different points in the program, context-sensitive analysis tracks the pointers separately for each call, preventing them from being mixed together. To implement context sensitivity, each program element in a function is assigned with a context. Let (o, c_o) and (v, c_v) be the object o and variable v under context c_o and c_v respectively, a context-sensitive analysis checks if (v, c_v) can point to (o, c_o).

In theory, a calling context can be generalized as a certain abstraction of the program's state regarding to the control flow at a call site. Therefore, various forms of context-sensitivity are applied, distinguished by the specific context elements

used to define the calling context. Currently, several kinds of context sensitivity are commonly used:

- **Callsite sensitivity** [45, 64, 80, 82, 100], which is probably the best-known type of context sensitivity. It distinguishes a function's contexts by sequences of call sites that lead to the function invocation, simulating the potential call stack when the target function is invoked during execution.

- **Object sensitivity** [39, 65, 66, 84], which is introduced for object-oriented programming languages like Java. It uses object instances (identified by their allocation sites) as contexts and distinguishes method contexts by the receiver objects at each call site. The intuition behind object sensitivity is that, in object-oriented languages, instance methods primarily behave differently depending on the object's state. For analysing programs written in object-oriented languages like Java, object-sensitivity is generally considered superior to callsite sensitivity [84].

- **Type sensitivity** [84], which approximates object sensitivity by replaces the receiver objects in contexts with the class types associated with the allocation sites of those objects. It provides a middle ground between context-insensitive and highly precise context-sensitive analysis, offering much of the precision of object sensitivity but with reduced computational cost.

While context-sensitive pointer analysis improves precision, it also increases computational complexity and memory usage due to the need to maintain separate points-to sets for different contexts. To address the issue of infinite calling contexts due to recursive calls and enhance scalability, the length of the contexts is typically constrained to a constant k. This, however, can result in precision loss, as different contexts may be merged when the length of the calling context exceeds k. The

k-limited context-sensitive pointer analysis strikes a balance between precision and efficiency: A higher k increases precision but may also significantly slow down the analysis or even render it unscalable.

2.2 The Rust Programming Language

2.2.1 An Introduction to Rust

Rust, a modern system programming language [1], offers memory safety akin to Java and performance on par with C/C++. Before introducing Rust's key language features, this subsection first discusses how memory is organized in Rust, particularly the use of the stack and the heap.

2.2.1.1 The Stack and the Heap in Rust

The stack and the heap are two key regions of memory used by programs, each with distinct purposes and characteristics. The stack is primarily used for local variables with fixed, known size, offering fast and automatic memory management. The heap, on the other hand, is used for dynamic memory allocation, providing more flexible memory management while incurring additional cost.

Rust's memory model is similar in layout to that of C++.

```rust
fn main() {
    let x = 42;     // stored on the stack
    let s = String::from("hello");
    // String metadata on the stack, contents on the heap
}
```

In the above example, the integer `x` is stored entirely on the stack. The `String` `s` is a stack-allocated structure made up of three parts: a pointer to the memory

that holds the contents of the string, a length, and a capacity. The actual string data `"hello"` resides on the heap.

Achieving both fine-grained control over heap memory and memory safety has been a difficult programming problem historically. C and C++ require programmers to manage heap memory manually using `malloc/free` or `new/delete`. Forgetting to deallocate memory causes memory leaks, while premature deallocation can lead to use-after-free errors and other forms of undefined behavior.

Java takes a different approach by using a garbage collector to manage heap memory. In Java, objects are all stored on the heap and the stack allocation is only used for primitive types and reference variables. The garbage collector periodically scans the memory to reclaim unreachable objects. While this eliminates manual deallocation and many memory bugs, it introduces runtime overhead and unpredictable latency, which are undesirable in performance-sensitive applications.

Rust provides C-like control over memory layout and performance, but enforces memory safety at compile time through its ownership and borrowing system, which are described in the following subsections.

2.2.1.2 Safe Rust

Rust provides strong guarantees of memory safety and concurrency at compile time with its innovative *ownership* system, by adhering to an aliasing XOR mutability principle.

Ownership. The core idea of Rust's type system is derived from concepts of *linear types* [98], which constrains that each variable can be used exactly once. While linear types provide strong resource management guarantees, they are often too strict for general use. Rust's ownership model relaxes this strictness, offering both flexibility and safety. Each value in Rust, like an integer or a string, has a

unique *owner* (the variable binding). For example, in the code below, `s` is the owner of the string.

```
fn main() {
    {
        let s = String::from("hello"); // `s` owns the String
        println!("{}", s);
    } // `s` goes out of scope, memory is reclaimed
    ...
}
```

The owner variable determines the lifespan of the value: a value is initialized when its owner variable is created, and its memory is automatically reclaimed when its owner goes out of scope. The compiler enforces this by implicitly inserting a call to the `drop` function (similar to a destructor in C++) at the end of the owner's lifetime. In this example, when `s` goes out of scope, Rust calls `drop(s)` to free the heap memory used by the string `"hello"`.

Rust's compiler manages lifetimes and ensures proper memory reclamation, much like the Resource-Acquisition-Is-Initialization (RAII) pattern used in other languages.

Borrowing. In Rust, a value can be borrowed by creating a reference to it, allowing for reading or modification without changing the ownership. In the example below, `s` is borrowed via `&s` and passed as an argument to the function `print_length`.

```
fn print_length(s: &String) {
    println!("Length: {}", s.len());
}

fn main() {
    let s = String::from("hello");
    print_length(&s);
}
```

Rust's type system ensures that references cannot outlive the value's owner, effectively preventing common memory issues like use-after-free or dangling pointers.

Aliasing XOR Mutability. Rust offers two types of references: immutable (`&`) and mutable (`&mut`), enabling two forms of borrowing. Immutable borrowing allows shared read access, while mutable borrowing grants exclusive write access. This principle is closely related to the idea of Universe type [28], a static type system to enforce ownership, encapsulation, and safe aliasing in object-oriented programs.

Rust's type system ensures that a mutable reference cannot coexist with other references, ensures that multiple references to a value for reading and writing cannot exist simultaneously, preventing data races and memory safety issues like invalid references (e.g., iterator invalidation in C++). For example, Rust will reject the code below because the immutable reference `r` is still in use when `s` is mutably borrowed by `push_str`, violating the aliasing XOR mutability principle.

```
1 fn main() {
2     let mut s = String::from("hello");
3     let r = &s;              // immutable borrow
4     s.push_str(", world!"); // `push_str` borrows `s` mutably
5     print_length(r);        // immutable borrow later used here
6 }
```

2.2.1.3 Unsafe Rust

While Rust's safety rules provide strong guarantees of memory safety, they are often too restrictive to accommodate the needs of certain low-level behaviours, especially in systems programming. To address these limitations, Rust offers an unsafe mode, allowing developers to bypass some of the language's safety checks. Unsafe code is encapsulated within a block marked by the `unsafe` keyword, signaling to the compiler that the code may perform operations outside of Rust's usual safety guarantees.

Raw Pointers. A key feature of unsafe Rust is its support for raw pointers. Unlike Rust's safe references (`&` and `&mut`), raw pointers (`*const` and `*mut`) are not subject to the borrowing rules that guarantee memory safety. Raw pointers can point to any memory location, including uninitialized or invalid memory. This allows for unchecked memory access, manual memory management, and operations such as interfacing with C code. For example, the previous code compiles successfully if we rewrite it with a raw pointer, as shown below.

```
1 fn main() {
2     let mut s = String::from("hello");
3     let r = &s as *const String;  // raw pointer
4     s.push_str(", world!");     // `push_str` borrows `s` mutably
5     unsafe { print_length(&*r); } // raw pointer later used here
6 }
```

While raw pointers enable more fine-grained control over performance and memory, they require careful handling to avoid the pitfalls of undefined behaviour that Rust's safety rules are designed to prevent.

The unsafe Rust highlights the ongoing need for advanced pointer analysis techniques tailored to Rust's unique features and challenges. However, current Rust pointer/alias analyses [23], mainly intra-procedural, focus more on references than raw pointers. The Rust compiler's checks are limited to the sharing XOR mutability discipline for references per function. Recent research [23] does not account for unsafe code due to the absence of alias analysis for raw pointers, hindering its effectiveness in identifying information flow vulnerabilities. Techniques for detecting Rust's memory-safety bugs [10,74,75] rely on heuristic aliasing approximations. Efficient and precise pointer analysis for Rust, especially in the presence of unsafe code, remains largely unexplored. In the presence of raw pointers, properties like aliasing restrictions and memory safety are no longer statically guaranteed by the

compiler, making the program behavior more similar to that of C or C++. Thus, this thesis focuses on developing a pointer analysis that conservatively models these cases without assuming the usual Rust safety invariants.

2.2.2 The Rust Language Model

The Rust compiler processes Rust programs through various Intermediate Representations (IRs): High-level IR (HIR), Mid-level IR (MIR), and LLVM IR. HIR is a desugared form of the AST (Abstract Syntax Tree), preserving the high-level structure of the source code but eliminating syntactic sugar. MIR, on the other hand, is a simplified, control-flow-oriented representation that carries explicit type information. MIR is favoured for various Rust program analyses [54, 97] due to its structured format and rich type information compared to HIR and LLVM IR. Figure 2.1 shows the MIR of a simple Rust program from the previous subsection, which creates a String, and passes its reference to the `print_length` function.

Figure 2.2 presents a simple Rust language model based on Rust MIR. A Rust program encompasses functions and global variables, which can be treated in a unified manner (since the initialization procedure for each global variable can be viewed as a function). Each function contains a set of local variables: the first local v_0 denotes the return value, followed by locals representing function arguments $(v_1, v_2, ...)$, and then user-declared variables and temporaries introduced by the compiler. Each function starts with declarations that specify the local variables used within the function, including their types. A type can be a Rust primitive type, such as `u8`, `u16`, or `u32`, a structure type like `String`, or other types like references. Following the declarations is a sequence of basic blocks, each identified by a **BlockNumber**. A basic block contains a sequence of statements and ends with a terminator—a special statement representing a conditional or unconditional

```
fn main() -> () {
    let mut _0: ();
    let mut _1: std::string::String;
    let _2: ();
    let _3: &std::string::String;
    bb0: {
        _1 = <String as From<&str>>::from(const "hello") -> bb1;
    }
    bb1: {
        _3 = &_1;
        _2 = print_length(copy _3) -> bb2;
    }
    bb2: {
        drop(_1) -> bb3;
    }
    bb3: {
        return;
    }
}
```

Figure 2.1: A simplified MIR example.

jump, directing control flow. Together, these basic blocks form a control flow graph (CFG).

In Rust MIR, each memory object is represented by a *Place Expression*, which consists of a base variable and a list of projection elements that *project out* from the base variable, such as dereferencing a pointer ($*\rho$), accessing a field or indexing an array ($\rho.j$). For example, to access the length field of a string via a reference `r`, we use the place expression `(*r).length`, where the projection elements include a dereference followed by a field access. As is standard, we analyse an array object by collapsing its elements into a single pseudo-field [19, 21, 39, 88].

Function	fn	$::=$	$f(v_1 : \tau_1, v_2 : \tau_2, ...)$ `->` τ { d; bb }
Declaration	d	$::=$	$d_1; d_2$ | `let` $v : \tau$
BasicBlock	bb	$::=$	$bb_1\ bb_2$ | bn : { s; t; }
Projection	j	$::=$	ϵ | $n.j$
PlaceExpr	ρ	$::=$	v | $*\rho$ | $\rho.j$
Rvalue	r	$::=$	ρ | $\&\rho$ | ρ `as` τ | ...
Statement	s	$::=$	$s_1; s_2$ | $\rho = r$
FunctionOp	φ	$::=$	f | v
Terminator	t	$::=$	$\rho = \varphi(\rho_1, \rho_2, ...)$ | `switch`$(\rho, [bn_1, bn_2, ...])$ | `goto`(bn) | `drop`(ρ) | `return` | ...

$f \in \mathbb{F}$ is a set of function names

$v \in \mathbb{V}$ is a set of variables

$n \in \mathbb{N}$ is a set of projection elements

$\tau \in \mathbb{T}$ is a set of types

$bn \in \mathbb{Z}$ is a set of BasicBlock numbers

Figure 2.2: A simplified Rust language model.

Each statement in a basic block is typically an assignment of the form $\rho = r$, which updates the value of ρ with an *rvalue* r. This *rvalue* may be a place expression (ρ), a place expression with its address taken ($\&\rho$), a type cast (ρ `as` τ), or other expressions like binary operations (omitted in Figure 2.2 for simplicity). Rust's type system ensures that ρ and r must have the same type.

A terminator can be 1) a function call ($\rho = \varphi(\rho_1, \rho_2, ...)$), which takes a list of arguments $\rho_1, \rho_2, ...$ and assigns the result to ρ. The function operand φ can be either a function identifier or a variable (function pointer); 2) A branch statement, such as `switch` and `goto`; 3) a `drop` statement, which explicitly free the resources owned by a place; 4) a return statement, where v_0 is returned as the default value.

While this language model provides a good representation of Rust MIR, it is somewhat complex for use in pointer analysis. In this thesis, we focus on flow-insensitive pointer analysis, considering only assignments and function calls while ignoring control flow and other irrelevant MIR statements. In addition, we intro-

Table 2.2: Eight kinds of statements analysed in pointer analysis.

Statement	Kind	Statement	Kind
$v = \texttt{alloc}(\tau)$	ALLOC	$p_1 = \&p_2$	ADDROF
$p_1 = p_2$	ASSIGN	$p_1 = p_2$ `as` τ	CAST
$p = (*v).j$	LOAD	$(*v).j = p$	STORE
$p = \&(*v).j$	GEP	$p_0 = \varphi(p_1, p_2, ...)$	CALL

duce *Places* as follows:

$$\textbf{Place} \quad p \quad ::= \quad v.j$$

Places are a subset of place expressions that exclude dereferences, always corresponding to a specific part of a local variable on the stack. On the other hand, a place expression may represent a memory access on the heap. For example, `s.length` is a place because it directly accesses a field of the local variable `s`, while `(*r).length` is a place expression that involves dereferencing a pointer `r`, which may point to heap-allocated memory. If a projection does not exist (when $j = \epsilon$), a place p degenerates simply into a local variable.

To simplify the Rust language model, we extract only the core statements relevant to Rust pointer analysis, as shown in Table 2.2. For complex assignments that do not fit these categories (e.g., $(*v_1).j_1 = (*v_2).j_2$), we break them down into simpler statements using temporary variables.

We use more specific representations, like p and $(*v).j$, to denote objects (or memory locations), rather than a general place expression ρ. Our analysis assumes that all pointers being dereferenced are local variables. As discussed in Section 2.2.1, there are two types of pointers: references (e.g., `&u32` and `&mut u32`) and raw pointers (e.g., `*const u32` and `*mut u32`). We treat both types uniformly in our analysis without distinguishing between them.

Let us explore the eight statement types in Table 2.2. We use an ALLOC statement v = `alloc`(τ) to represent the creation of an object o of type τ either on the stack or the heap. For stack allocation, the ALLOC statement corresponds to a `let` $v : \tau$ declaration in MIR, and v directly represents o. For heap allocation, the ALLOC statement corresponds to a call to an allocation function, like `alloc`, `realloc`, `alloc_zeroed`, and `exchange_malloc`. In this case, v points to the heap object o, and the heap object is identified by its allocation site. Heap objects default to type `u8`, as the return type of `alloc` is `*u8`. For example, the function call `String::from("hello")` internally invokes `alloc` to allocate memory on the heap, and the pointer inside the resulting string points to this memory region.

An ADDROF statement assigns the address of p_2 to another place, creating a pointer p_1. This typically corresponds to a borrow in Rust, such as `let r = &s;`. To read from or write into a place dereferenced from a pointer v, we use LOAD or STORE statements. For example, accessing the interior pointer field of a string via a reference `r` involves a LOAD statement like `let ptr = (*r).ptr;`. A GEP statement is similar to an LLVM IR `GetElementPtr` instruction, returning the address of a field or element in the base object pointed to by v. For example, `let p = &(*r).length` creates a reference p that points to the `length` field of the string referenced by `r`. ASSIGN or CAST statements copy the value from p_2 to p_1, with CAST reinterpreting it as a new type τ. Rust has two main types of type casts: primitive cast (for primitive data types, e.g. `let x = y as u32;`) and pointer cast (for pointer types, e.g. `let p = q as *const String`). In pointer analysis, we only need to handle pointer casts. Some Rust MIR cast statements are categorised as ASSIGN because they don't reinterpret the underlying objects, such as casting from a reference to a raw pointer or from a concrete type to a dynamic trait type.

When analysing a CALL statement, we must account for inter-procedural assignments, which may include argument-passing and value-return. In the case of a function pointer call, the target function is represented as a variable.

In pointer analysis, we typically analyse statements that copy memory addresses from one pointer to another. In Rust MIR, whole structs or arrays can be assigned from one variable to another. For instance, a statement like `s1 = s2` can copy all fields or elements from the `s1` variable to the `s2` variable. Therefore, it is important to be cautious when handling assignments in Rust MIR, as implicit pointer assignments can occur when copying pointer-type fields between structs. For example, if both `s1` and `s2` are of `String` type, then the assignment `s1 = s2` also implies an interior assignment `s1.ptr = s2.ptr`, where `ptr` is the internal pointer field of the `String` type. Failing to account for such implicit copies can lead to missing key aliasing relationships.

In Chapter 3 and Chapter 4, we will develop our pointer analysis based on the simplified Rust language model discussed in this section.

Chapter 3

RUPTA: A Context-Sensitive Pointer Analysis Framework for Rust

3.1 Overview

As introduced in Chapter 2, the unsafe Rust underscores the ongoing need for advanced pointer analysis techniques tailored to Rust MIR, especially since existing Rust analysis tools for detecting memory safety issues often lack precision due to imprecise points-to or alias information and inaccuracies in the call graph.

Developing a dedicated pointer analysis for Rust MIR presents significant challenges and requires substantial engineering efforts, primarily due to the complexity of certain language features as highlighted in Chapter 1:

- Firstly, Rust supports various calls resolved through different mechanisms like static and dynamic dispatch via its unique *trait* system. Additional features, such as the `Fn*` traits, further complicate its call resolution logic.

Accurately computing call targets for these diverse calls at Rust MIR level requires correct modelling of the associated semantics for each kind of call, a challenging task due to limited documentation in Rust.

- Secondly, achieving precise pointer analysis and constructing the call graph demands maintaining accurate type information for objects in a Rust program, which is less necessary in C/C++ pointer analysis that operates on a lower-level IR, as explained in Section 3.2. This task is complicated by pointer casts, which can interpret the same object as having different types at different program points. Given the prevalence of pointer casts in Rust programs, particularly in the standard library, more complex handling is needed to accurately preserve type information during analysis.

- Finally, Rust's memory access paradigms are as intricate as those of C/C++, but its MIR operates at a higher abstraction level than LLVM IR (used by SVF [88]), necessitating a proper memory model and more complex handling during pointer analysis. Especially careful handling of nested structs is essential to preserve type information accurately.

In this chapter, we introduce Rupta, the first context-sensitive pointer analysis framework designed specifically for Rust MIR. Rupta incorporates callsite-based context-sensitivity and on-the-fly call graph construction, effectively addressing earlier challenges. It offers comprehensive support for resolving various types of calls and maintaining precise type information during analysis. To handle pointer casts, Rupta adopts a novel approach, symbolising memory objects with multiple abstractions of different types. Additionally, Rupta uses a projection-based approach, unlike the field-index-based method commonly used in C/C++ pointer analysis [72, 88], to differentiate fields in nested structs while preserving their type

information. RUPTA is conceived as an open-source project, built entirely from scratch using Rust, and currently comprises approximately 10,000 lines of code. It aims to function as a tool for advanced Rust analyses, enhancing the safety and security of Rust programs.

To illustrate RUPTA's effectiveness in call graph construction, we implemented two additional call graph construction tools commonly used in Rust program analysis: RURTA (a sound but less precise technique based on Rapid Type Analysis (RTA) [9] utilised in [97]) and RUSCG (a commonly used unsound technique considering only static dispatch [54, 75]).

In our extensive evaluation of 13 real-world Rust projects, RUPTA demonstrated its ability to provide highly precise points-to information while maintaining high efficiency. When comparing RUPTA's call graph construction capabilities with those of RURTA and RUSCG, we found that RUPTA outperforms RUSCG by discovering 24% more call graph edges and is more precise than RURTA by eliminating about 74% of spurious dynamic call edges.

Our evaluation reveals a notable insight for Rust: in our framework, 1-callsite-sensitive pointer analysis offers better scalability compared to Andersen-style analysis, easily achievable through our RUPTA framework. We found that many Rust Standard Library functions, especially those safe functions encapsulating unsafe code, show markedly less precision in context-insensitive analysis, leading to longer analysis times. This encourages exploration into targeted context-sensitive pointer analysis methods, potentially boosting Rust's analysis efficiency.

In summary, this chapter makes the following contributions:

- Introduction of RUPTA, the first context-sensitive pointer analysis framework for Rust MIR, public available at `https://rustanlys.github.io/rupta/`

- Addressing challenges in precise Rust pointer analysis and call graph construction, including diverse function calls, accurate type semantics for pointer casts and complex memory models.

- Extensive evaluation of Rupta to demonstrate its effectiveness in efficiently producing accurate call graphs and precise points-to information for Rust.

The rest of this chapter is organised as follows. Section 3.2 highlights the design challenges and motivates our approach. Section 3.3 introduces the Rupta framework. Section 3.4 covers implementation details. Lastly, Section 3.5 presents performance evaluation.

3.2 Motivation: Challenges and Insights

As Rust gains traction in systems development, the demand for research in pointer analysis for the language is growing. However, addressing Rust's unique characteristics presents new challenges, especially when designing tools like Rupta to operate on Rust MIR. Achieving precision and efficiency while accommodating Rust-specific features becomes paramount. We delve into the encountered challenges during development, which involve preserving type information for each pointed-to object. These challenges include analysing calls, modelling pointer casts, and representing object fields, such as nested structs.

3.2.1 Analysing Calls

To efficiently compute precise points-to information and call graphs in Rupta, we need to effectively resolve call targets. In Rust, *traits* are essential for defining shared behaviours between types and abstracting over implementations. An ex-

ample of a trait `Shape`, defining a common method `area()`, is given in Figure 3.1. Two structs, `Circle` and `Rectangle`, implement this trait.

```
1  trait Shape {
2      fn area(&self) -> f64;
3  }
4  struct Circle { r: f64 }
5  impl Shape for Circle {
6      fn area(&self) -> f64 { PI * self.r * self.r }
7  }
8  struct Rectangle { w: f64, h: f64 }
9  impl Shape for Rectangle {
10     fn area(&self) -> f64 { self.w * self.h }
11 }
12 fn sprint_area<T: Shape>(obj: T) {
13     // static dispatch
14     print!("Area = {}", obj.area());
15 }
16 fn dprint_area(obj: &dyn Shape) {
17     // dynamic dispatch
18     print!("Area = {}", obj.area());
19 }
20
21 let c = Circle { r: 1.0 };
22 let r = Rectangle { w: 2.0, h: 1.0 };
23 sprint_area(c);
24 sprint_area(r);
25 dprint_area(&c as &dyn Shape);
```

Figure 3.1: An example with static and dynamic dispatch.

Traits enable polymorphism, enhancing flexibility and code reuse. Unlike Java interfaces, Rust traits are more expressive and play a deeper role in the type system. Traits support two dispatch mechanisms: static dispatch through generics

and monomorphization as well as dynamic dispatch via dynamic trait objects. In contrast, Java interfaces are inherently dynamically dispatched and lack such fine-grained control over implementation resolution and specialization.

3.2.1.1 Static Dispatch

Static dispatch in Rust allows for the creation of generic functions that can be applied to arguments of different types using generics or parametric polymorphism (similar to C++'s class templates). For instance, consider the function `sprint_area()` in Figure 3.1. It can be called with arguments of any type that has implemented the `Shape` trait, such as `Circle` and `Rectangle`.

Generics are compiled away by monomorphization when lowering Rust MIR to LLVM IR. When `sprint_area()` is called with `Circle` and `Rectangle` objects (lines 23-24), two separate copies are called, each for a concrete argument type. The effect is roughly equivalent to calling the following two functions:

```
1 fn sprint_area_circle(obj: Circle) { ... }
2 fn sprint_area_rectangle(obj: Rectangle) { ... }
```

Within each function, the call to `obj.area()` will execute the specific implementation associated with the argument type.

Rust's type system provides complete reasoning capabilities regarding generic types at compile time, which empowers a pointer analysis tool on Rust MIR, such as Rupta (introduced in this chapter), to effectively analyse functions with generics through monomorphized duplication for every potential implementation.

3.2.1.2 Dynamic Dispatch

Dynamic dispatch achieves polymorphism using *dynamic trait objects*, which refer to values of any type that implement a given trait. The specific type of the object is

only known at runtime. In Figure 3.1, `dprint_area()` is similar to `sprint_area()`, but its parameter `obj` is a dynamic trait object, indicated by the keyword "dyn". Functions invoked on such objects are dynamically dispatched at runtime, enabling runtime polymorphism. A dynamic trait object is generated at a cast site, which is a point where a concrete type is converted to a `dyn` type either explicitly or implicitly, e.g., `&c as &dyn Shape` in line 25.

Nightly Rust[1] also includes support for an unstable feature—*trait upcasting coercion* [17], allowing the creation of a dynamic trait object by casting from another trait object, provided the source trait type is a subtrait of the target trait type. For instance, if `Display` is a supertrait of `Shape`, we can create a `dyn Display` trait object by casting from a `dyn Shape` trait object, and they will share the same concrete types.

The pointer for a dynamic trait object is essentially a fat pointer that includes a data pointer to the object itself and a pointer to the *virtual method table* (vtable) (*vtable*) [29]. The vtable enables runtime resolution of trait-defined method calls. Dynamic dispatch offers flexibility for working with traits and is crucial for runtime polymorphism. However, constructing a precise static call graph becomes challenging with dynamic dispatch, requiring an accurate model of the concrete types that a dynamic trait object can refer to.

3.2.1.3 Function Pointers

Rust, like C/C++, supports function pointers for dynamic runtime function invocation. These pointers refer to function items or closures with unknown identities at compile time. In Figure 3.2, `fp` in line 6 is a function pointer pointing to

[1]The nightly release channel of the Rust compiler, which includes the latest features and experimental capabilities not yet stabilized for the stable release.

the function item `times2`, and the dynamic function call in line 7 uses `fp` as the operand.

```
1  // function pointer
2  fn times2(x: u32) -> u32 {
3      x * 2
4  }
5  // Function pointer call
6  let fp: fn(u32) -> u32 = times2;
7  let r = fp(2);
8  // Fn trait calls
9  fn foo<F: Fn(u32) -> u32>(x: u32, f: F) -> u32 {
10     f(x)
11 }
12 foo(2, |x| x * 2); // Closure
13 foo(2, times2);    // Function Item
14 foo(2, fp);        // Function Pointer
```

Figure 3.2: Function pointers and `Fn` traits.

```
1 pub trait Fn<Args: Tuple>: FnMut<Args>
2 {
3     // Required method
4     extern "rust-call" fn call(&self, args: Args)
5         -> Self::Output;
6 }
```

Figure 3.3: Definition of the `Fn` trait.

3.2.1.4 Fn Traits

The Rust standard library defines three special traits: `Fn`, `FnMut`, and `FnOnce`, representing different function call styles. We'll use `Fn*` for brevity to refer to any of these traits. An `Fn*` object acts as a regular function and can be used as a

function call operand. In Figure 3.2, the function `foo()` takes an `Fn*` argument and calls it as `f(x)` on line 10.

The `Fn` trait is defined as in Figure 3.3. Any type that implements an `Fn*` trait must implement the `call()` method defined within the trait. The Rust compiler will route a call via an `Fn*` trait object (e.g., `f(x)`) to the corresponding implementation. Similar to regular traits, `Fn*` traits can be dispatched statically or dynamically. In Rust, `Fn*` traits are automatically implemented by closures, function items and function pointers (lines 12-14 in Figure 3.2). Special handling is required to accurately model function calls through the `Fn*` trait objects.

In conclusion, accurately computing points-to information and building precise call graphs for Rust programs requires addressing Rust's complex static and dynamic dispatch semantics. Handling dynamic trait objects and function pointers is also crucial. Existing pointer analysis frameworks like SVF [88] and Pinpoint [81] for C/C++, as well as Qilin [39] and Doop [19] for Java, do not inherently support Rust's `Fn*` traits, function pointers, or Rust's specific features.

3.2.2 Modelling Pointer Casts

Handling pointer casts, a common practice in low-level programming languages, poses a challenge in correctly inferring underlying object types. In Rust, a pointer cast statement, similar to the `bitcast` instruction in LLVM IR, allows reinterpretation of the underlying object into one with a different type, leading to varying object types at different points.

Existing pointer analysis frameworks for Java, such as Qilin [39] and Doop [19], do not address pointer casts between two unrelated types, as illustrated in Figure 3.4, since such casts are not permitted in Java. On the other hand, frameworks

```
1 let r = 2.0;
2 let p = &r as *const f64 as *const Circle;
3 let q = p as *const dyn Shape;
4 let s: &dyn Shape = &(*q);    // s is equivalent to q
5 s.area();
```

Figure 3.4: An example with a pointer cast.

for C/C++, such as SVF [88], treat pointer cast statements as straightforward assignments without accurately modelling the type information of the cast object.

In Figure 3.4, we reuse the `Shape` trait and the `Circle` struct from Figure 3.1. Here, `p` is obtained by casting an `f64` pointer referring to variable `r` into a `Circle` object pointer (line 2), and then `p` itself is cast to a dynamic `Shape` object (lines 3–4). Finally, the trait method `area()` is called on `s` (line 5).

In Rust's pointer analysis, dynamic calls are resolved by identifying the types of objects to which the pointers of dynamic trait objects point. This approach is similar to that used in Java pointer analysis, primarily due to the lack of vtable information in Rust MIR. Treating pointer casts as simple assignments, as in existing C/C++ pointer analysis [81,88], would lead to the failure to resolve the call `s.area()`, as `s` would be assumed to point to `r` of type `f64`.

In Rust, the vtable pointer of a dynamic trait object is initialized at the cast site, depending on the concrete type from which the trait object is being cast. Thus, the vtable pointer of `s` actually points to the vtable corresponding to the implementation of `Circle` for `Shape`. As a result, calling `Circle::area()` will yield $\pi \times 2^2 = 12.56$.

To accurately model pointer casts for a given object, we propose using multiple abstractions for the underlying object, each associated with a unique type. For example, in the casting scenario from Figure 3.4, we create a new abstraction `r'`

of type `Circle` and associate it as the pointed-to object for pointers `q` and `s`. This approach enables RUPTA to correctly resolve `s.area()` calls, enhancing analysis soundness and precision through type filtering [16].

3.2.3 Representing an Object's Fields

The representation of an object's fields, particularly in nested structs in Rust, greatly affects pointer analysis precision, necessitating the preservation of type information in analysis.

Rust's memory model shares similarities with C/C++ but differs from Java. Java uses reference semantics, storing all objects on the heap. In contrast, C/C++ employs both value and reference semantics, enabling objects to reside on both the stack and the heap. Rust, by default, supports value semantics, allowing objects to contain subobjects as field components. Additionally, any object, regardless of its allocation location (stack or heap), can have its address taken and stored in a variable. These features have substantial implications for handling field information in Rust program pointer analysis.

Consider an example illustrated in Figure 3.5. In Figure 3.5a, the `Outer` struct contains a subobject `inn` of type `Inner`, stored directly within the `Outer` instance, accessible without dereferencing. In Figure 3.5b, `ptr` holds the address of `obj.inn` subobject, and it is then used to store the address of `a` into the `obj.inn.x` field. Precise pointer analysis requires field sensitivity to distinguish fields within the same struct.

Existing pointer analysis frameworks for C/C++ [81,88] use a field-index-based approach on LLVM IR, representing the i-th field of `obj` as `(obj,` i`)`. This approach typically allows structs to share the same index with their first field, facilitating the handling of pointer casts that convert a struct pointer into its first field

```
1  struct Inner {
2      x: &u32,
3      y: &u32  }
4  struct Outer {
5      inn: Inner }
```

(a) Struct declaration

```
6  let a: u32 = 2;
7  let mut obj = Outer {
8      inn: Inner {...} };
9  let mut ptr = &mut obj.inn;
10 (*ptr).x = &a;
```

(b) Complex field access

```
11 impl ToString for Inner { fn to_string(&self) -> String {...} }
12 impl ToString for Outer { fn to_string(&self) -> String {...} }
13 impl ToString for &u32 { fn to_string(&self) -> String {...} }
14
15 let tostr = ptr as &dyn ToString;
16 tostr.to_string();
```

(c) Dynamic call

Figure 3.5: An example with nested struct types.

(e.g., casting `*Outer` to `*Inner`). However, when applied to Rust MIR, this representation introduces challenges. Rust MIR includes a wider range of type categories than LLVM IR, such as `Enum` and `Union`, making it difficult to properly represent each kind of field. Additionally, this representation creates ambiguity regarding object types. For example, the abstraction `(obj, 0)` can represent three different types: `Outer`, `Inner`, and `&u32`. This ambiguity hampers the full utilisation of Rust's precise type system and may result in precision issues.

In the scenario where the `ToString` trait is implemented for three types, `Outer`, `Inner`, and `&u32`, as depicted in Figure 3.5c, a complexity emerges when creating a dynamic trait object. This complexity arises when casting `ptr` into `tostr` (line 15) and then invoking the trait-defined method `to_string()` on it (line 16). The field-index-based approach is unable to determine which of these three implementations should be called. This ambiguity stems from

the fact that the object `(obj, 0)` that `tostr` points to could potentially be any one of the three types: `Outer`, `Inner`, or `&u32`.

In our RUPTA framework, we use a projection-based representation to distinguish object fields, identifying each subobject by its base object and full field projection. This preserves the source code's access pattern (e.g., `obj.inn` and `obj.inn.x`) and assigns a unique type to each object. In the example from Figure 3.5c, our approach correctly identifies that `tostr` points to the `obj.inn` object of type `Inner`, ensuring accurate resolution of the dynamic call in line 15.

In general, pointer analysis operates on a low-level IR with a simple memory model (e.g. LLVM IR), where complex language abstractions have been lowered down into simple primitives such as reading/writing registers. However, despite having been simplified, the memory access paradigms in Rust MIR closely mirror those in source code. Especially, an entire structure or array can be copied to another. For example, using the `Outer` struct defined in Figure 3.5a, there can be a statement like `out2 = out1` in Rust MIR, wherein both `out1` and `out2` are of the `Outer` type. In such a case, all the fields or elements are copied from the source to the destination variable. Therefore we have `out2.inn.x = out1.inn.x` and `out2.inn.y = out1.inn.y`.

3.3 RUPTA: Pointer Analysis for Rust

In this section, we introduce the core pointer analysis technique used in RUPTA. Section 3.3.1 outlines the rules for performing Andersen-style, i.e., context-insensitive inclusion-based pointer analysis [4] for Rust. Section 3.3.2 describes how to adapt these rules to support callsite-based context sensitivity. Finally, Section 3.3.3 discusses RUPTA's effective handling of various Rust language features.

3.3.1 Andersen-Style Inclusion-Based Pointer Analysis

We formalise Andersen-style pointer analysis based on the language model in Section 2.2.2. In addition to the notations $\mathbb{F}$, $\mathbb{V}$, $\mathbb{N}$ and $\mathbb{T}$ from Figure 2.2 that represent the sets of functions, variables, data types and projection elements, we use $l \in \mathbb{L}$ to represent the set of statement labels. Let $\mathbb{J} = \mathbb{N}^*$ denote the universe of projections. Thus, $\mathbb{V} \times \mathbb{R}$ represents the set of places, whereas $\mathbb{P}$ and $\mathbb{O}$ denote the sets of pointers and pointed-to memory locations, respectively.

As introduced in Section 3.2.2, we adopt a novel approach compared to existing pointer analysis frameworks for C/C++ [81, 88] to address pointer casts. In our approach, a memory location involved in a cast statement is represented by multiple abstract objects, each with a distinct type. We use $o_{\succ\tau} \in \mathbb{O}$ to denote an abstract object referring to the type variant of o, which is specialised to have the type τ. Our pointer analysis ensures that (1) each object is associated with a single type, and (2) each pointer, excluding those for dynamic trait objects, is restricted to point only to objects of a specific type. It is worth noting that maintaining multiple abstract objects for the same memory location introduces additional challenges. In particular, ignoring the correlations between these abstract objects may result in serious soundness issues. We will elaborate on this problem and present our solution in Section 3.3.3.2.

Our pointer analysis mainly computes the points-to relation:

- `pts` : $\mathbb{P} \mapsto \wp(\mathbb{O})$

which records the points-to information for each pointer.

The following auxiliary functions are also used:

- `type_of` : $(\mathbb{P} \cup \mathbb{O}) \mapsto \mathbb{T}$
- `proj_ty` : $\mathbb{T} \mapsto \wp(\mathbb{J} \times \mathbb{T})$

- `ptr_proj` : $\mathbb{T} \mapsto \wp(\mathbb{J})$
- `deref_ty` : $\mathbb{T} \mapsto \mathbb{T}$
- `transmute` : $\mathbb{O} \times \mathbb{T} \mapsto \mathbb{O}$
- `dispatch` : $(\mathbb{F} \cup \mathbb{P}) \times \mathbb{P} \mapsto \wp(\mathbb{F})$

where `type_of` records the type of a pointer or an abstract object, `proj_ty` returns the projection for all fields of a given type, along with the field type (including an empty projection ϵ paired with the given type), `ptr_proj` fetches the projections of the pointer type fields of a given type (resulting in ϵ for pointer types or $\emptyset$ if no pointer type field is found), `deref_ty` indicates the type of an object obtained when dereferencing a pointer of a given pointer type, `transmute` returns the type variant of the given object, which is specialised for a different type, and `dispatch` determines the actual implementation of a function to be invoked for a call. More details on the last two can be found in Section 3.3.3.

Figure 3.6 formalises Andersen-style inclusion-based pointer analysis with inference rules. We will briefly explain each rule using the examples from Figure 3.5. In Section 3.3.3, we explore the analysis of various advanced language features, addressing the corresponding challenges from Section 3.2.

- [AllocStack] is responsible for managing stack allocation for a local variable. It creates an abstract object for this variable and its constituent fields (if any), while recording their types. For instance, with the type `Outer`, we can derive field-type pairs such as `(`ϵ`, Outer)`, `(inn, Inner)`, `(inn.x, &u32)`, and `(inn.y, &u32)` using the function `proj_ty`. Therefore, objects `obj`, `obj.inn`, `obj.inn.x` and `obj.inn.y` are created and their respective types are recorded for the local variable `obj`.

$$\frac{l : v = \texttt{alloc}(\tau) \qquad (j, \tau') \in \texttt{proj_ty}(\tau)}{\texttt{type_of}(v.j) = \tau'} \text{ [AllocStack]}$$

$$\frac{l : p = \texttt{alloc}(\tau) \qquad (j, \tau') \in \texttt{proj_ty}(\tau)}{o_l \in \texttt{pts}(p) \quad \texttt{type_of}(o_l.j) = \tau'} \text{ [AllocHeap]}$$

$$\frac{l : p_1 = \&p_2}{p_2 \in \texttt{pts}(p_1)} \text{ [AddrOf]}$$

$$\frac{l : p_1 = p_2 \qquad \tau = \texttt{type_of}(p_1) \qquad j \in \texttt{ptr_proj}(\tau)}{\texttt{pts}(p_2.j) \subseteq \texttt{pts}(p_1.j)} \text{ [Assign]}$$

$$\frac{l : p = (*v).j \qquad o \in \texttt{pts}(v) \qquad \tau = \texttt{type_of}(p) \qquad j' \in \texttt{ptr_proj}(\tau)}{\texttt{pts}(o.j.j') \subseteq \texttt{pts}(p.j')} \text{ [Load]}$$

$$\frac{l : (*v).j = p \qquad o \in \texttt{pts}(v) \qquad \tau = \texttt{type_of}(p) \qquad j' \in \texttt{ptr_proj}(\tau)}{\texttt{pts}(p.j') \subseteq \texttt{pts}(o.j.j')} \text{ [Store]}$$

$$\frac{l : p = \&(*v).j \qquad o \in \texttt{pts}(v)}{o.j \in \texttt{pts}(p)} \text{ [Gep]}$$

$$\frac{l : p_1 = p_2 \texttt{ as } \tau \qquad o \in \texttt{pts}(p_2) \qquad \tau' = \texttt{deref_ty}(\tau)}{o_{\succ\tau'} = \texttt{transmute}(o, \tau') \qquad o_{\succ\tau'} \in \texttt{pts}(p_1)} \text{ [Cast]}$$

$$\frac{\begin{array}{c} l : p_0 = \varphi(p_1, ..., p_r) \qquad f \in \texttt{dispatch}(\varphi, p_1) \\ \tau_0 = \texttt{type_of}(p_0) \qquad \tau_1 = \texttt{type_of}(p_1) \qquad ... \qquad \tau_r = \texttt{type_of}(p_r) \end{array}}{\begin{array}{c} \forall j_0 \in \texttt{ptr_proj}(\tau_0) : \texttt{pts}(v_0^f.j_0) \subseteq \texttt{pts}(p_0.j_0) \\ \forall i \in [1, r], \forall j_i \in \texttt{ptr_proj}(\tau_i) : \texttt{pts}(p_i.j_i) \subseteq \texttt{pts}(v_i^f.j_i) \end{array}} \text{ [Call]}$$

Figure 3.6: Rules for Andersen-style inclusion-based pointer analysis.

- [AllocHeap] is responsible for allocating a heap object. It operates in a manner akin to [AllocStack] except that the abstract object o_l, identified by the allocation site l, is added to the points-to target of the pointer p.

- [AddrOf] simply causes p_1 to point to the place p_2.

- [Assign] handles implicit pointer copy in an Assign statement element-wise, based on the projections of the pointer type fields of the type of p_1 (and p_2). For instance, if p_1 and p_2 are of the `Outer` type, $\texttt{pts}(p_2.\texttt{inn.x}) \subseteq \texttt{pts}(p_1.\texttt{inn.x})$ and $\texttt{pts}(p_2.\texttt{inn.y}) \subseteq \texttt{pts}(p_1.\texttt{inn.y})$ will be established. If p_1 and p_2 are of a pointer type, then $\texttt{ptr_proj}(\tau) = \epsilon$, reducing [Assign] to $\texttt{pts}(p_2) \subseteq \texttt{pts}(p_1)$.

- [Load] and [Store] obtain every memory location o that may be pointed to by v from the `pts` function and handle Load or Store statements as an Assign statement between $o.j$ and p, for all pointer type fields of $o.j$ and p. For example, given the pointer `ptr`, which points to the object `obj.inn`, we can load it into a variable `in` of type `Inner` using `in = *ptr` (or store to it using `(*ptr) = in`). It will be processed as `in.x = obj.inn.x` and `in.y = obj.inn.y` (or `obj.inn.x = in.x` and `obj.inn.y = in.y`).

- [Gep] finds any object o that may be pointed to by v and inserts its field $o.j$ into the points-to set of p.

- In [Cast], only a pointer cast is considered where τ is a pointer type. The function `transmute` is applied to each object o in the points-to set of p_2 to obtain its type variant $o_{\succ\tau'}$, where τ' is the dereference type of τ. $o_{\succ\tau'}$ is added to the points-to set of p_1 instead of o.

- [CALL] applies `dispatch` to determine the actual implementation of a function invoked. In Rust MIR, a method call $p_0 = p_1.f(p_2, ..., p_r)$ is translated into $p_0 = f(p_1, p_2, ..., p_r)$, with p_1 as the first argument in the call. We identify the variables in a function f as $v_0^f, v_1^f, ...$ Thus, a call is modelled as assignments from its actual arguments p_i to their corresponding formal parameters v_i^f, and from its return variable v_0^f to the target variable p_0 of the call statement.

Employing the inference rules from Figure 3.6, we use the incremental worklist algorithm [49] for pointer analysis. This method maintains and updates a worklist of pointers, propagating changes through the constraint graph of pointer-related assignments (Table 2.2), until reaching a fixed point.

3.3.2 Callsite-Sensitive Pointer Analysis

The Andersen-style pointer analysis in Figure 3.6 is context-insensitive, producing a single analysis result for each function in a program. To improve the precision of analysis, RUPTA supports context sensitivity through *cloning*, which creates multiple instances of a function under distinct calling contexts to prevent information from one context from flowing to another. It employs a *k-callsite-sensitive* pointer analysis, distinguishing the calling context of a function based on its k-limited call path (the last k call sites, each labelled as $l \in \mathbb{L}$).

The callsite-based context-sensitive pointer analysis is formalised with inference rules in Figure 3.7. Let $\mathbb{C} = \mathbb{L}^*$ be the universe of contexts. For a context $c = [l_1, ..., l_n] \in \mathbb{C}$ and a call site $l \in \mathbb{L}$, $l{::}c$ represents $[l, l_1, ..., l_n]$, and $\lceil c \rceil_k$ returns the k-limited context $[l_1, ..., l_k]$ for c. We use (o, c) and (p, c) to denote specialised instances of object o and variable p under context c, respectively. We introduce a new function

- `ctxt_of` : $\mathbb{F} \mapsto \wp(\mathbb{C})$

which maintains the contexts for each function.

In addition, several functions used in the Andersen-style pointer analysis (Figure 3.6) are adapted into their context-sensitive versions:

- $\texttt{pts}_{\texttt{C}} : \mathbb{P} \times \mathbb{C} \mapsto \wp(\mathbb{O} \times \mathbb{C})$
- $\texttt{transmute}_{\texttt{C}} : \mathbb{O} \times \mathbb{T} \times \mathbb{C} \mapsto \mathbb{O}$
- $\texttt{dispatch}_{\texttt{C}} : (\mathbb{F} \cup \mathbb{P}) \times \mathbb{P} \times \mathbb{C} \mapsto \wp(\mathbb{F})$

The adapted functions retain the same logic as the originals while incorporating context information. For example, $(o, c') \in \texttt{pts}_{\texttt{C}}(v, c)$ means that object o under context c' is included in the points-to set of variable v under context c. $\texttt{transmute}_{\texttt{C}}(o, \tau', c')$ obtains the type variant $o_{\succ \tau'}$ of o under context c'. Finally, $f \in \texttt{dispatch}_{\texttt{C}}(\varphi, p_1, c)$ is a possible target function for the call to φ under c.

We do not explain the rules in Figure 3.7 extensively as they are context-sensitive versions of those in Figure 3.6.

3.3.3 Support for Rust's Language Features

We delve into our approaches for addressing various pointer analysis challenges, including analysing calls, modelling pointer casts, and representing an object's fields (among others). In Section 3.3.3.1 and Section 3.3.3.2, we will explain the $\texttt{dispatch}_{\texttt{C}}$ and $\texttt{transmute}_{\texttt{C}}$ functions, respectively. We focus on the context-sensitive versions, as the context-insensitive ones can be easily derived by simplifying the context-sensitive versions.

3.3.3.1 Calls

Different kinds of function calls are resolved through $\texttt{dispatch}_{\texttt{C}}$ in our analysis according to Algorithm 1.

$$\frac{l : v = \texttt{alloc}(\tau) \qquad (j, \tau') \in \texttt{proj_ty}(\tau)}{\texttt{type_of}(v.j) = \tau'} \text{ [CS-AllocStack]}$$

$$\frac{l : p = \texttt{alloc}(\tau) \qquad c \in \texttt{ctxt_of}(f_l) \qquad (j, \tau') \in \texttt{proj_ty}(\tau)}{(o_l, c) \in \texttt{pts}_\texttt{C}(p, c) \;\; \texttt{type_of}(o_l.j) = \tau'} \text{ [CS-AllocHeap]}$$

$$\frac{l : p_1 = \&p_2 \qquad c \in \texttt{ctxt_of}(f_l)}{(p_2, c) \in \texttt{pts}_\texttt{C}(p_1, c)} \text{ [CS-AddrOf]}$$

$$\frac{\begin{array}{c} l : p_1 = p_2 \\ c \in \texttt{ctxt_of}(f_l) \qquad \tau = \texttt{type_of}(p_1) \qquad j \in \texttt{ptr_proj}(\tau) \end{array}}{\texttt{pts}_\texttt{C}(p_2.j, c) \subseteq \texttt{pts}_\texttt{C}(p_1.j, c)} \text{ [CS-Assign]}$$

$$\frac{\begin{array}{c} l : p = (*v).j \qquad c \in \texttt{ctxt_of}(f_l) \\ (o, c') \in \texttt{pts}_\texttt{C}(v, c) \qquad \tau = \texttt{type_of}(p) \qquad j' \in \texttt{ptr_proj}(\tau) \end{array}}{\texttt{pts}_\texttt{C}(o.j.j', c') \subseteq \texttt{pts}_\texttt{C}(p.j', c)} \text{ [CS-Load]}$$

$$\frac{\begin{array}{c} l : (*v).j = p \qquad c \in \texttt{ctxt_of}(f_l) \\ (o, c') \in \texttt{pts}_\texttt{C}(v, c) \qquad \tau = \texttt{type_of}(p) \qquad j' \in \texttt{ptr_proj}(\tau) \end{array}}{\texttt{pts}_\texttt{C}(p.j', c) \subseteq \texttt{pts}_\texttt{C}(o.j.j', c')} \text{ [CS-Store]}$$

$$\frac{l : p = \&(*v).j \qquad c \in \texttt{ctxt_of}(f_l) \qquad (o, c') \in \texttt{pts}_\texttt{C}(v, c)}{(o.j, c') \in \texttt{pts}_\texttt{C}(p, c)} \text{ [CS-Gep]}$$

$$\frac{\begin{array}{c} l : p_1 = p_2 \texttt{ as } \tau \\ c \in \texttt{ctxt_of}(f_l) \qquad (o, c') \in \texttt{pts}_\texttt{C}(p_2, c) \qquad \tau' = \texttt{deref_ty}(\tau) \end{array}}{o_{\succ\tau'} = \texttt{transmute}_\texttt{C}(o, \tau', c') \qquad (o_{\succ\tau'}, c') \in \texttt{pts}_\texttt{C}(p_1, c)} \text{ [CS-Cast]}$$

$$\frac{\begin{array}{c} l : p_0 = \varphi(p_1, ..., p_r) \\ c \in \texttt{ctxt_of}(f_l) \qquad c' = \lceil l{::}c \rceil_k \qquad f \in \texttt{dispatch}_\texttt{C}(\varphi, p_1, c) \\ \tau_0 = \texttt{type_of}(p_0) \qquad \tau_1 = \texttt{type_of}(p_1) \qquad ... \qquad \tau_r = \texttt{type_of}(p_r) \end{array}}{\begin{array}{c} c' \in \texttt{ctxt_of}(f) \\ \forall j_0 \in \texttt{ptr_proj}(\tau_0) : \texttt{pts}_\texttt{C}(v_0^f.j_0, c') \subseteq \texttt{pts}_\texttt{C}(p_0.j_0, c) \\ \forall i \in [1, r], \forall j_i \in \texttt{ptr_proj}(\tau_i) : \texttt{pts}_\texttt{C}(p_i.j_i, c) \subseteq \texttt{pts}_\texttt{C}(v_i^f.j_i, c') \end{array}} \text{ [CS-Call]}$$

Figure 3.7: Rules for k-callsite-sensitive pointer analysis (f_l is the function containing statement l being analysed).

Algorithm 1: $\texttt{dispatch}_{\texttt{C}}$

Input: The call's operand φ, the call's first argument p_1, and a context c

Output: A set of target functions $\mathcal{F}$ resolved

```
1  if φ is a function identifier then
2      f ← φ
3      if f is statically dispatched then
4          f′ ← static_dispatch(f)
5          F ← {f′}
6      else // f is dynamically dispatched
7          F ← dynamic_dispatch(f, p1, c)
8  else // φ is a function pointer
9      F ← ∅
10     foreach f′ ∈ pts_C(φ, c) do
11         F ∋ static_dispatch(f′)
```

Static Dispatch. A statically dispatched call in Rust aims for a specific compile-time-determined implementation. However, Rust's use of generics adds complexity. In Rust MIR, generic types remain, and monomorphization transforms MIR into lower-level codegen IR. To handle static dispatch in Rupta, we perform on-the-fly monomorphization. When calling a generic function, Rupta determines the real types of the generics in a context-sensitive manner and substitutes them into the target function. In Rust MIR, a function is identified by a (*def_id*, *generic_args*) pair, where *def_id* uniquely identifies the function, and *generic_args* lists the (argument) types for generic substitution. For example, the monomorphized `sprint_area()` functions in Figure 3.1 are represented by $(id_{\texttt{sprint_area}}, [\texttt{Circle}])$ and $(id_{\texttt{sprint_area}}, [\texttt{Rectangle}])$.

To determine the target function for a call that is to be statically dispatched, Rupta uses `static_dispatch` given in Algorithm 2. Specifically, when resolving a call to a trait-defined method f (lines 5–18), Rupta treats the trait method as

Algorithm 2: `static_dispatch`

Input: A call's operand f
Output: A target function f' resolved

```
1  (def_id, generic_args) ← f
2  if def_id is not a trait method then
3      f' ← f
4  else
5      if def_id is not an Fn* trait method then
6          f' ← Instance :: resolve(def_id, generic_args)
7      else
8          if generic_args[0] is Closure(def_id', generic_args') then
9              f' ← (def_id', generic_args')
10         else if generic_args[0] is FnDef(def_id', generic_args') then
11             f'' ← (def_id', generic_args')
12             f' ← static_dispatch(f'')
13         else if generic_args[0] is FnPtr then
14             f' ← dummpy_function()
15         else
16             f' ← Instance :: resolve(def_id, generic_args)
```

a generic function, since a default implementation can be provided within the trait and monomorphized for various types that implement this trait. In this case, the *def_id* of the callee corresponds to the method's definition within the trait scope, and the first element of *generic_args* represents the concrete type on which the method is called. In Figure 3.1, when the method `area()` defined in trait `Shape` is called on a `Circle` object, the *def_id* and *generic_args* are $id_{\texttt{Shape::area}}$ and [`Circle`], respectively. To identify the exact implementation for the invocation, Rupta uses the Rust compiler's internal API, `Instance::resolve()`. If the trait method is implemented for the concrete type, the corresponding implementation is

Algorithm 3: `dynamic_dispatch`

Input: The call's operand f, the call's first argument p_1, and a context c

Output: A set of target functions $\mathcal{F}$ resolved

1 $\mathcal{F} \leftarrow \emptyset$

2 **foreach** $(o, c') \in \texttt{pts}_{\texttt{C}}(p_1, c)$ **do**

3 $\quad \tau \leftarrow \texttt{type_of}(o)$

4 $\quad$ replace $generic_args[0]$ of f with τ

5 $\quad \mathcal{F} \ni \texttt{static_dispatch}(f)$

selected. Otherwise, the default implementation within the trait is monomorphized for the given concrete type.

Handling calls through `Fn*` traits (lines 8–16) involves specific treatments for closures, function items, and function pointers, corresponding to `Closure`, `FnDef`, and `FnPtr` in Algorithm 2. For a closure, the target function is identified as the associated closure itself. In the case of a function item, `static_dispatch` is utilised to further process it. When dealing with a function pointer under a `Fn*` trait, a new dummy function is generated to act as the target function:

```
1 fn dummy_func(fp: fn(T)->U, args: T) -> U { fp(args) }
```

which only involves a function pointer call. The values of `fp` and `args` will come from the arguments of the `Fn*` trait call. All the other calls via `Fn*` traits are handled in line 16.

Dynamic Dispatch. Dynamic dispatch for virtual calls on dynamic trait objects necessitates knowledge of the concrete types these objects refer to. Call graph edges are then dynamically identified using the gathered points-to information. Algorithm 3 details the `dynamic_dispatch` process for resolving targets of such calls. In these calls, the function f's def_id relates to a trait method's definition, with the first element of $generic_args$ being a dynamic type, like `dyn Shape` in Figure 3.1.

We simulate dynamic dispatch as static by substituting $generic_args[0]$ with the type of each object the trait object's pointer references, using `static_dispatch`.

Function Pointers. The process of resolving a function pointer call is described in lines 8–11 of Algorithm 1. A function pointer may point to a function item or a closure. Therefore, we identify all potential targets during the pointer analysis. Then, for each potential target, we simulate dynamic dispatch statically by using `static_dispatch`.

3.3.3.2 Pointer Casts

Like C++'s `reinterpret_cast`, these statements often pose challenges, leading to unsound (under-approximate) and imprecise (spurious) points-to information. To enhance modelling of pointer casts and achieve superior soundness and precision compared to state-of-the-art pointer analyses for C/C++ [81, 88] (Section 3.2.2), we propose using multiple abstract objects with distinct types to represent memory locations. Each abstract object o is initially associated with a type τ. When a cast statement transforms o into a new type τ', a new abstract object, denoted as $o_{\succ\tau'}$, is created to represent the modified memory location.

Maintaining the correlations between an object o and its type variants $o_{\succ\tau'}$ is crucial. For example, in the `Outer` struct from Figure 3.5a, a developer might cast a pointer to an `Outer` struct into a pointer to a `(&u32, &u32)` tuple:

```
1 let p = &obj as *const Outer;
2 let q = p as *const (&u32, &u32);
```

As `q` points to a new abstract object $\texttt{obj}_{\succ\texttt{(\&u32,\&u32)}}$, it is essential to maintain the points-to information in both `obj` and $\texttt{obj}_{\succ\texttt{(\&u32,\&u32)}}$ in sync. This ensures that $\texttt{obj}_{\succ\texttt{(\&u32,\&u32)}}\texttt{.0}$ is associated with `obj.inn.x` and $\texttt{obj}_{\succ\texttt{(\&u32,\&u32)}}\texttt{.1}$ is associated with `obj.inn.y`. Neglecting this synchronization can lead to unsound results.

Algorithm 4, as applied in CAST (Figure 3.7), manages the $\texttt{transmute}_{\texttt{C}}()$ function. When a memory location o undergoes a cast to type τ', a new abstract object $o_{\succ\tau'}$ is created, provided it does not already exist. Type coercion constraints are then set between o and $o_{\succ\tau'}$. For each pair of pointer type fields $(o.j,\ o_{\succ\tau'}.j')$ sharing the same offset, mutually reciprocal assignments are inserted, classified as ASSIGN or CAST based on whether their pointer types are identical. Each assignment is added under a specific context c (provided as an input to $\texttt{transmute}_{\texttt{C}}$) in the context-sensitive pointer analysis. The corresponding rule [CS-ASSIGN] or [CS-CAST] is applied accordingly. For example, if a new ASSIGN statement $o.j = o_{\succ\tau'}.j'$ is added under c, we establish a new constraint $\texttt{pts}_{\texttt{C}}(o.j, c) \subseteq \texttt{pts}_{\texttt{C}}(o_{\succ\tau'}.j, c)$. This process ensures synchronization of points-to information between the original object and its transformed variants.

Algorithm 4: $\texttt{transmute}_{\texttt{C}}$

Input: An abstract object o, a cast type τ', and a context c

Output: A new abstract object $o_{\succ\tau'}$ of type τ'

```
1  o_{≻τ'} ← get_or_create(o, τ')
2  proj ← ptr_proj(type_of(o))
3  proj' ← ptr_proj(τ')
4  for j ∈ proj, j' ∈ proj' do
5      if o.j, o_{≻τ'}.j' have the same offset then
6          τ_j ← type_of(o.j)
7          τ'_{j'} ← type_of(o_{≻τ'}.j')
8          if τ_j == τ'_{j'} then
9              Emit a new ASSIGN stmt under context c: o.j = o_{≻τ'}.j'
10             Emit a new ASSIGN stmt under context c: o_{≻τ'}.j' = o.j
11         else
12             Emit a new CAST stmt under context c: o.j = o_{≻τ'}.j' as τ_j
13             Emit a new CAST stmt under context c: o_{≻τ'}.j' = o.j as τ'_{j'}
```

Our approach effectively addresses various issues caused by pointer casts, but it still encounters challenges when a pointer is cast to an integer, a problem that remains unresolved in C/C++ pointer analysis [81, 88]. Additionally, we observed that many library functions perform pointer casts to `*u8` pointers and then revert them back to their original types after some operations. These intermediate steps often result in imprecise `*u8` pointers that point to multiple extraneous objects, leading to unnecessary type variants, imprecise points-to relations, and spurious subobjects. To mitigate this, we impose restrictions on such casts. Specifically, we only allow a non-heap object o to be transmuted to $o_{\succ u8}$ and further cast to $o_{\succ \tau'}$ if $o_{\succ \tau'}$ has already been created before. While this approach may be unsound in rare cases, it aligns with existing techniques [11, 72] that prioritise precision and selectively support well-established code patterns rather than being overly conservative, as mentioned in [8].

Another issue arises with recursive casting:

```
1 struct Foo { x: u32, y: u32 }
2 p = &(*p).y as *const u32 as *const Foo;
```

where `p` initially points to an object `foo` of type `Foo`. Analysing line 2 leads it to point to the abstract object $(\texttt{foo.y})_{\succ \texttt{Foo}}$. However, re-analysing this line for the new pointed-to object induces another object, $((\texttt{foo.y})_{\succ \texttt{Foo}}.\texttt{y})_{\succ \texttt{Foo}}$, and this process repeats indefinitely, causing our analysis to fail to terminate. This issue is known as the *Positive Weight Cycle (PWC)* problem in pointer analysis for C/C++ [72]. To address it, we currently ignore casts that may lead to recursive casting, as such casts are typically incompatible and unsafe, stemming from the inherent imprecision of pointer analysis.

3.3.3.3 Representing an Object's Fields

Let us explain how our projection-based approach for representing object fields enables accurate analysis for the example in Figure 3.5, using the rules from Figure 3.7. When initialising `obj`, we create abstract objects `obj`, `obj.inn`, `obj.inn.x`, and `obj.inn.y` through [AllocStack], with their types being `Outer`, `Inner`, `&u8`, and `&u8`, respectively. [AddrOf] handles the statement that assigns the address of `obj.inn` to `ptr`, causing `obj.inn` to be included in the points-to set of `ptr`. The casting of `ptr` to `tostr` is processed as a direct assignment by [Assign], resulting in `tostr` encompassing all objects pointed to by `ptr`. Finally, [Call] resolves the call `tostr.to_string()` by invoking the `dispatch` function (Algorithm 1), which identifies the types of the pointed-to objects of `tostr` according to Algorithm 3. As `tostr` is found to point to a single object `obj.inn` with a precise type, `Inner`, this call is correctly resolved to `Inner::to_string()`, as desired.

3.3.3.4 Other Language Features

Rupta effectively handles several other Rust features, such as `Enum` and `Union` types. Enum variants are processed as if they are fields of the enum, thus eliminating the need for specialised handling. In the case of `Union` types, where different fields occupy the same storage space, type coercion constraints are set among these fields, akin to the approach in Algorithm 4.

In Rust, `Builder::spawn()` initiates threads by accepting a generic `Fn*` trait object and executing its associated function in a new thread. This function, indirectly invoked via the external C function `thread_start()`, is not directly resolvable in Rust MIR. We have precisely modelled `spawn()`'s semantics, enabling analysis of the associated function.

3.3.4 Discussion

RUPTA performs analysis on Rust MIR—the canonical intermediate representation used by the Rust compiler which faithfully captures the full semantics and expressive power of the Rust language. Our simplified Rust language model, outlined in Figure 2.2, effectively models Rust MIR for the purpose of pointer analysis. All statements that are relevant to flow-insensitive pointer analysis are extracted and categorized into eight kinds, as shown in Table 2.2. RUPTA is sound with respect to this language model, except for the case of "ρ `as` τ". This is because our inference rules in Figure 3.6 and Figure 3.7 consistently over-approximate all statements analysed. It is important to note that in practical scenarios, no pointer analysis frameworks are entirely sound due to dynamic language elements like reflection, native code, and pointer casts, as discussed in [58]. As described in Section 3.3.3.2, RUPTA is more sound and precise in handling "ρ `as` τ" than the state of the art [88].

3.4 Implementation

Rupta. RUPTA, developed in Rust, consists of about 10K lines of code and is designed as a custom callback function for the official Rust compiler, `rustc 1.77.0-nightly`. It seamlessly integrates with Cargo, the official Rust package manager, and can be executed with a single command, `cargo pta`. By default, RUPTA includes `build-std`, which compiles the standard library as part of the crate graph compilation. Moreover, RUPTA includes dedicated handlers to enhance the precision of analysing two standard library functions, `Result::map_err` and `Into::into<Unique, NonNull>`. Achieving similar precision with these two functions typically necessitates more in-depth context analysis.

Baselines. As the first pointer analysis framework for Rust, RUPTA's call graph construction will be assessed against two state-of-the-art solutions: RUSCG (Static Call Graph for Rust) [10,54,75] and RURTA (Rapid Type Analysis [9] for Rust) [97]. RUSCG constructs a static call graph, omitting dynamic calls, a common approach in existing inter-procedure analysis tools [54,75]. RURTA determines possible types for dynamic trait objects using information about dynamic trait object creation. It maintains a cache of concrete types at cast sites, allowing dynamic traits to refer to any type in the cache implementing the specified trait. RURTA also applies to function pointers, treating them as pointing to any function item or closure with the same signature encountered in assignments. The technical details of RURTA will be discussed in Section 4.3.1.

3.5 Evaluation

We demonstrate RUPTA's efficiency and precision in performing k-callsite-based pointer analysis for extensive Rust programs, outperforming RUSCG and RURTA in constructing accurate call graphs. We also highlight the inefficiency of Andersen, mainly due to Rust's distinctive handling of raw pointers. The four RUPTA configurations we evaluate are Andersen (context-insensitive), `1CS` (*1-callsite-sensitive*), `2CS` (*2-callsite-sensitive*), and `3CS` (*3-callsite-sensitive*). Our two key research questions are:

- **RQ1.** Is RUPTA scalable and effective?
- **RQ2.** Can RUPTA build precise call graphs?

Benchmarks. To evaluate the scalability, efficiency, and precision of RUPTA, we conducted experiments on real-world Rust projects. We have selected 13 large open-source projects with high popularity, based on their number of downloads or

Table 3.1: Rust projects (including #stars and #MIR statements)

Project	#Stars	#Stmts	Version	Description
atuin	20.6k	889K	31c6ec0b	Shell history management for an SQLite database
bandwhich	10k	348K	6ef2d569	A terminal bandwidth utilisation tool
dust	8.8k	320K	b86e5c8c	A more intuitive version of 'du'
exa	23.6k	112K	fb05c421	A modern replacement for 'ls'
fselect	4k	477K	f9a9e54d	Finding files with SQL-like queries
gitui	18.4k	656K	50538504	A fast terminal-ui for git
grin	5k	921K	43b43d97	An implementation of the Mimblewimble protocol
lsd	13.4k	388K	69b28bf4	An advanced version of 'ls' command tool
mdbook	18k	789K	bd323fb9	A utility to create books from markdown files
navi	15k	492K	d713d501	An interactive cheatsheet tool for 'cmd'
resvg	2.8k	392K	6c69d298	An SVG rendering library
rustscan	14.5k	421K	4c933b39	A modern take on the port scanner
zoxide	22.4k	230K	9f67fb5b	A smarter 'cd' command tool

stars on GitHub. Each project in our dataset includes a binary target, enabling us to start the analysis from the main entry point. The dependencies (including the Rust Standard Library) are compiled together with the project, allowing us to analyse functions within them. The repositories we selected, along with their GitHub stars, number of MIR statements (in reachable functions discovered by Rurta), repository version, and a brief description, are presented in Table 3.1.

Experimental Setting. We have conducted all our experiments on Ubuntu 20.04.5, with an Intel(R) Xeon(R) W-2275 @ 3.30GHz CPU and 512GB of RAM.

3.5.1 RQ1: Is Rupta Scalable and Effective?

Table 3.2 presents the analysis times, memory usage and average points-to sizes of Rupta for its four configurations. Rupta, the first context-sensitive pointer analysis tool for Rust, demonstrates scalable analysis times for Andersen, `1CS`, and `2CS` across all 13 projects. `1CS` can complete the analysis within 30 seconds for the majority of the projects (9 out of 13), and within 60 seconds for all except

Table 3.2: Analysis times, memory usage, and average points-to sizes of Rupta in four configurations (OOM: Out of memory).

Program	Metrics	Andersen	1CS	2CS	3CS
atuin	analysis time (s)	178.7	100.9	474.2	3013.5
	mem (GB)	67.0	32.6	103.2	375.9
	#avg-pts	288.17	56.49	34.62	21.27
bandwhich	analysis time (s)	22.4	11.9	33.5	112.0
	mem (GB)	5.6	3.2	8.7	26.5
	#avg-pts	133.89	19.57	11.66	7.02
dust	analysis time (s)	12.5	7.1	18.4	66.3
	mem (GB)	4.1	2.1	7.3	32.4
	#avg-pts	75.95	8.57	6.71	4.32
exa	analysis time (s)	1.4	2.0	5.4	15.5
	mem (GB)	0.3	0.4	1.3	4.3
	#avg-pts	28.30	5.84	5.03	3.85
fselect	analysis time (s)	29.9	23.4	133.2	552.5
	mem (GB)	9.0	8.3	26.1	116.8
	#avg-pts	136.91	24.72	20.07	13.21
gitui	analysis time (s)	134.0	43.7	191.4	1422.1
	mem (GB)	49.5	18.9	52.0	178.0
	#avg-pts	282.11	21.75	15.81	10.79
grin	analysis time (s)	184.9	100.8	635.0	3059.0
	mem (GB)	59.0	42.1	103.5	418.1
	#avg-pts	372.03	52.25	46.68	26.82
lsd	analysis time (s)	18.5	10.5	29.1	144.5
	mem (GB)	6.5	3.6	11.7	65.1
	#avg-pts	90.64	10.44	6.79	4.56
mdbook	analysis time (s)	189.1	47.2	724.8	
	mem (GB)	46.8	12.1	131.6	OOM
	#avg-pts	315.48	22.14	17.42	
navi	analysis time (s)	37.8	16.9	97.5	585.2
	mem (GB)	15.4	5.3	22.5	110.3
	#avg-pts	148.74	16.60	12.74	8.43
resvg	analysis time (s)	36.5	18.7	54.7	314.5
	mem (GB)	13.8	7.0	24.7	94.1
	#avg-pts	184.94	37.23	29.13	23.55
rustscan	analysis time (s)	26.8	13.9	74.9	589.7
	mem (GB)	7.7	4.0	18.4	93.6
	#avg-pts	125.25	14.05	11.16	8.78
zoxide	analysis time (s)	10.7	6.3	25.8	141.3
	mem (GB)	3.6	1.6	5.7	22.7
	#avg-pts	95.27	13.64	11.21	8.79

`atuin` and `grin`. Meanwhile, `2CS` manages to complete for 11 projects within 8 minutes. The analysis time and memory usage increase significantly for `3CS`, and it runs out of memory when analysing `mdbook` (consisting of 789K statements). This unscalability aligns with the observation that published papers on callsite-based pointer analysis for Java and C/C++ [39, 50, 61] have not even utilised $k \geqslant 3$.

Surprisingly, `1CS` outperforms Andersen, achieving faster analysis times across all 13 benchmarks except for `exa`, contrary to typical outcomes in pointer analyses for Java [39, 51]. This is primarily due to encapsulating unsafe code, especially raw pointers within safe Rust code. Without context-sensitivity, analysing many functions in the Rust Standard Library produces highly imprecise points-to results, resulting in significantly larger points-to sets for many pointers and an overall increase in analysis time. As shown in Table 3.2, increasing k from 1 to 3 in Rupta improves precision, evidenced by a slight decrease in the average size of points-to sets for each project. Nevertheless, the disparity between Andersen and `1CS` remains considerable, with Andersen's average points-to set sizes being an order of magnitude larger.

Let us explore why Rupta's performance benefits from context-sensitivity in practice. In Figure 3.8, we provide simplified definitions of two commonly used structs: `NonNull` and `Iter`, along with the constructor for `Iter`. `NonNull` plays a critical role in efficient heap memory manipulation, serving as a wrapper for raw pointers in widely used types like `Box` and `Vec`. On the other hand, `Iter` is used for iterating a slice. When creating a new `Iter` instance from a slice, the constructor first converts the `&[T]` slice to a pointer `ptr` of type `*T` using the `as_ptr` function. Then, `ptr` is passed as an argument to `new_unchecked()` to create a new `NonNull` object. These two functions, `as_ptr()` and `new_unchecked()`, are frequently called in a program. In context-insensitive analysis, the pointer field of the created `Iter`

```
1  pub struct NonNull<T> { p: *const T }
2  pub struct Iter<T> { ptr: NonNull<T>, end: *const T }
3  impl<T> Iter<T> {
4      pub fn new(slice: &[T]) -> Self {
5          let ptr = slice.as_ptr();
6          Self {
7              ptr: NonNull::new_unchecked(ptr),
8              end: ptr.add(slice.len())
9          }
10     }
11 }
```

Figure 3.8: An example calling for context-sensitivity.

object would point to every memory location referenced by the arguments of these two functions. This could result in numerous pointed-to objects, which might further propagate, potentially causing issues with many other library functions.

The problem of inflated points-to targets worsens as the project's scale increases, as shown in Table 3.2 (with average points-to sizes reaching 315.48 for `mdbook` and 372.03 for `grin`). However, this problem is substantially mitigated with context sensitivity, where even `1CS` proves sufficient to avoid conflating analysis results across contexts for functions like `as_ptr` and `new_unchecked`. Consequently, the average points-to size decreases significantly (from 315.48 to 22.14 for `mdbook` and from 372.03 to 52.25 for `grin`).

3.5.2 RQ2: Can Rupta Build Precise Call Graphs?

Table 3.3 summarises four key statistics for call graphs built using Rupta, Rurta, and Ruscg: (1) *#call-edges*: the number of call graph edges discovered, (2) *#reach-funcs*: the number of reachable functions found, (3) *#dyn-callsites*: the number of

Table 3.3: Comparing call graphs built by different analyses

Program	Metrics	Ruscg	Rurta	Ander	1CS	2CS	3CS
atuin	#call-edges	160733	199916	180046	179032	178998	178919
	#reach-funcs	69941	83590	77602	77246	77237	77229
	#dyn-callsites	-	287	179	175	173	171
	#dyn-call-edges	-	7608	1295	1150	1127	1119
bandwhich	#call-edges	62008	78815	74715	70903	70903	70874
	#reach-funcs	29514	34600	33352	32653	32653	32648
	#dyn-callsites	-	122	102	98	98	96
	#dyn-call-edges	-	2267	674	531	531	513
dust	#call-edges	62560	73701	66143	65250	65250	65239
	#reach-funcs	28469	32064	29680	29426	29426	29419
	#dyn-callsites	-	87	61	58	58	57
	#dyn-call-edges	-	1401	407	315	315	313
exa	#call-edges	22238	25279	24269	23760	23760	23758
	#reach-funcs	10289	11207	10925	10819	10819	10818
	#dyn-callsites	-	56	51	44	44	43
	#dyn-call-edges	-	660	252	185	185	184
fselect	#call-edges	84633	103696	94635	93539	93536	93226
	#reach-funcs	35525	41401	38928	38663	38660	38572
	#dyn-callsites	-	209	189	188	188	186
	#dyn-call-edges	-	3024	891	749	748	679
gitui	#call-edges	95960	143652	139151	125652	125652	125647
	#reach-funcs	42464	58738	57334	52424	52424	52421
	#dyn-callsites	-	131	108	99	99	96
	#dyn-call-edges	-	2741	1044	778	778	775
grin	#call-edges	137681	198785	171632	169553	169543	169537
	#reach-funcs	55915	77430	69075	68481	68474	68471
	#dyn-callsites	-	854	454	450	449	447
	#dyn-call-edges	-	7971	2261	2006	2004	2000
lsd	#call-edges	70724	83430	76288	75373	75371	75369
	#reach-funcs	30188	34436	32200	31914	31913	31912
	#dyn-callsites	-	109	81	73	73	72
	#dyn-call-edges	-	1476	457	318	317	316
mdbook	#call-edges	94807	173376	147219	146396	146394	-
	#reach-funcs	42272	71630	62997	62758	62757	-
	#dyn-callsites	-	242	140	139	138	-
	#dyn-call-edges	-	4372	1081	921	919	-
navi	#call-edges	80599	112978	99408	98561	98561	98559
	#reach-funcs	34952	46664	42178	41918	41918	41917
	#dyn-callsites	-	157	98	95	95	94
	#dyn-call-edges	-	3363	650	529	529	528
resvg	#call-edges	47579	81818	81003	80225	80225	80225
	#reach-funcs	19760	31437	31236	30948	30948	30948
	#dyn-callsites	-	192	189	187	187	187
	#dyn-call-edges	-	968	534	474	474	474
rustscan	#call-edges	78810	96525	87611	87136	87131	87127
	#reach-funcs	36086	40903	38422	38300	38299	38297
	#dyn-callsites	-	278	227	227	227	225
	#dyn-call-edges	-	3128	939	888	884	882
zoxide	#call-edges	32315	50801	49876	48816	48812	48810
	#reach-funcs	14711	21988	21788	21440	21440	21439
	#dyn-callsites	-	85	65	54	54	53
	#dyn-call-edges	-	1247	672	426	422	421

dynamic callsites detected, and (4) *#dyn-call-edges*: the number of dynamic call edges resolved.

Let us first compare Rupta and Ruscg. On average, Rupta discovers approximately 24% more call edges and 21% more reachable functions, resulting in more sound call graphs, and consequently, enabling uncovering potentially more security vulnerabilities than Ruscg (once used by security analysis tools [7, 10, 13, 54, 75, 102]). Comparing Rupta and Rurta in Table 3.3, we Rupta is more precise than Rurta by discovering less call edges and reachable function. Especially, 1CS outperforms Rurta by reducing spurious dynamic edges by an average of 74%.

Our results show significant precision improvements in call graph construction with context-sensitive pointer analysis compared to Andersen, reducing dynamic edges by approximately 19% on average. Andersen exhibits notable imprecision, resulting in a substantial divergence between its call graphs and those produced by context-sensitive analyses (1CS to 3CS). In most cases, 1CS to 3CS produce nearly identical results across all metrics, with most benchmarks showing slightly better precision for 3CS or 2CS over 1CS. Rust differs fundamentally from other popular languages like C/C++ and Java, with relatively fewer dynamic call sites even in real-world Rust projects (Table 3.1). However, for Rust programs using certain language features, such as structured types and pointer casts, Rupta demonstrates good precision and efficiency. Therefore, 1CS is generally sufficient for constructing precise call graphs for such Rust programs, and increasing the context length may have limited benefits. It is worth noting that in Java pointer analysis, increasing the context length—for example, from 2-object-sensitive to 3-object-sensitive as in Qilin [39] and Doop [84]—may also yield only small precision improvements in constructing call graphs for large Java applications. Nevertheless, Rupta rep-

resents a significant advancement in pointer analysis over the state of the art for Rust. It is substantially more sound than RUSCG (discovering 24% more call graph edges) and more precise than RURTA (eliminating about 74% of spurious dynamic call edges).

3.6 Summary

We introduce RUPTA as the first context-sensitive pointer analysis framework for Rust, functioning on Rust MIR. This framework effectively addresses crucial aspects like call analysis, pointer cast modelling, and the management of nested struct fields. Through evaluations conducted on real-world projects, RUPTA has proven its capability in generating precise points-to information and creating accurate call graphs. RUPTA is expected to significantly enhance existing program analysis and verification tools, focusing on bug and security vulnerability detection. This advancement is aimed at improving the overall safety and reliability of Rust programs.

Chapter 4

Stack Filtering: Elevating Precision and Efficiency in Rust Pointer Analysis

4.1 Overview

Existing pointer analyses, like Andersen's traditional method [3], often produce many spurious points-to relations due to a lack of context sensitivity, compromising precision. Context-sensitive alternatives, notably k-callsite-sensitive pointer analysis [39, 65, 80, 82, 84], enhance precision by analysing functions separately based on their distinct calling contexts, identified by the sequences of their k most recent callsites. To manage the exponential growth of contexts, these methods typically limit k to small values, often no more than 3. However, this k-limiting approach can result in numerous spurious context-sensitive points-to relations by not adequately distinguishing the calling contexts for certain functions. The computation

and maintenance of such spurious relations not only reduce the precision of pointer analysis but also consume significant time and memory resources unnecessarily.

Pointer analysis often incorporates additional program information to improve precision and efficiency. A prime example is type filtering in pointer analysis for strongly-typed languages like Java, where type information is used to filter points-to sets, allowing pointers to only include objects of compatible types [39, 49, 83]. This strategy effectively eliminates many spurious points-to relations with minimal computational expense. Despite these advancements, certain valuable program information is still frequently overlooked, indicating significant opportunities for further enhancements in this field.

The lifetime of objects is a crucial piece of information overlooked in existing pointer analysis frameworks. Ignoring object lifetimes leads to spurious points-to relations, where pointers are inaccurately predicted to point to objects outside their lifetimes. This chapter delves into utilising lifetime information to eliminate such spurious points-to relations.

In Rust pointer analysis, there are three types of objects: static, heap, and stack objects. Static objects endure throughout the program's execution, while heap objects, allocated dynamically in heap memory, possess unpredictable lifetimes. On the other hand, stack-allocated objects have statically determinable lifetimes, tied directly to their declaration scope within their declaring functions. These stack objects are automatically deallocated at the end of their scope, making them accessible via pointers only when their declaring functions are active and have not yet returned.

Building on this crucial insight, we introduce *stack filtering*, a novel approach to enhance precision and efficiency in context-sensitive pointer analysis for Rust. Stack filtering first determines the liveness of stack objects at specific program points

using function reachability relations from a sound pre-built call graph, created in a lightweight manner. It then removes spurious context-sensitive points-to relations during the main pointer analysis phase. When propagating points-to information for a stack object o within a function f, we evaluate o's liveness by considering the reachability between f and the declaring function f' of o. By accounting for stack object lifetimes, stack filtering effectively reduces spurious points-to relations involving pointers to inactive stack objects. In practice, this approach not only improves precision but also enhances efficiency of pointer analysis by trimming points-to sets.

To the best of our knowledge, stack filtering has not been previously considered in pointer analysis for any programming language. Java-focused research has primarily aimed at enhancing context-sensitive pointer analysis through object sensitivity [65], improved heap object modelling [93], selective context-sensitivity [36,52], and the application of machine learning [44]. In contrast, C/C++ research has targeted advancements in flow-sensitive pointer analysis, utilising methods like sparse data-flow [34, 81, 88] and strong updates [48, 88].

We have implemented stack filtering in Rupta. To build a sound call graph approximating function reachability relations, we used Rapid Type Analysis (RTA) [9]. We tailored this approach for Rust, particularly to refine its precision in handling dynamic dispatch. In our evaluation with 13 real-world Rust programs in Table 3.1, stack filtering markedly enhanced analysis efficiency, leading to reductions in analysis time and memory usage. It also improved precision, as shown by the decreased average points-to set size.

In summary, this chapter makes the following main contributions:

- Introduction of *stack filtering*, a novel technique designed to enhance both the precision and efficiency of context-sensitive pointer analysis for Rust. This

is achieved by filtering out spurious points-to relations on the fly during the pointer analysis process, utilising stack object liveness information derived from pre-computed reachability relations between functions.

- Extensive evaluation of the effectiveness of *stack filtering* on real-world Rust programs. Experimental results demonstrate that *stack filtering* significantly accelerates pointer analyses while notably improving precision. These findings highlight stack filtering's suitability for new programming languages like Rust, which prioritise automatic memory management and often prefer static allocation on the stack.

The rest of this chapter is structured as follows: Section 4.2 outlines the motivation behind our approach. Section 4.3 details the formalisation of context-sensitive pointer analysis with stack filtering for Rust. Lastly, Section 4.4 presents the evaluation of stack filtering.

4.2 Motivation

We begin with a brief discussion of the imprecision of both context-insensitive and context-sensitive pointer analysis in Section 4.2.1, followed by the increased costs of achieving context sensitivity in Section 4.2.2. In Section 4.2.3, we introduce the core concept behind stack filtering.

4.2.1 Imprecision of Pointer Analysis

Andersen-style inclusion-based pointer analysis [3], known for its efficiency-precision balance, can suffer significant precision loss in certain code regions of a program due to its context-insensitivity. This issue is exemplified in a small Rust program shown in Figure 4.1, representing a typical scenario (Section 4.4).

In `main()`, functions `foo()` and `bar()` are called, creating stack objects `a` and `b`. These objects' addresses are passed to `id3()`, leading to the call sequence `id3()` $\rightarrow$ `id2()` $\rightarrow$ `id1()`. Each function in this sequence returns the same reference it receives, tracing the path of the original reference through `id3()`.

In context-insensitive pointer analysis, functions are analysed once, merging all points-to information regardless of its calling contexts. This leads to conflation as seen in Figure 4.2(a), where `&foo::a` and `&bar::b` merge at `id3::x` (line 13), causing spurious flows into `foo::p` and `bar::q`. Consequently, we have $\texttt{pts}(\texttt{foo::p}) = \texttt{pts}(\texttt{bar::q}) = \{\texttt{foo::a}, \texttt{bar::b}\}$.

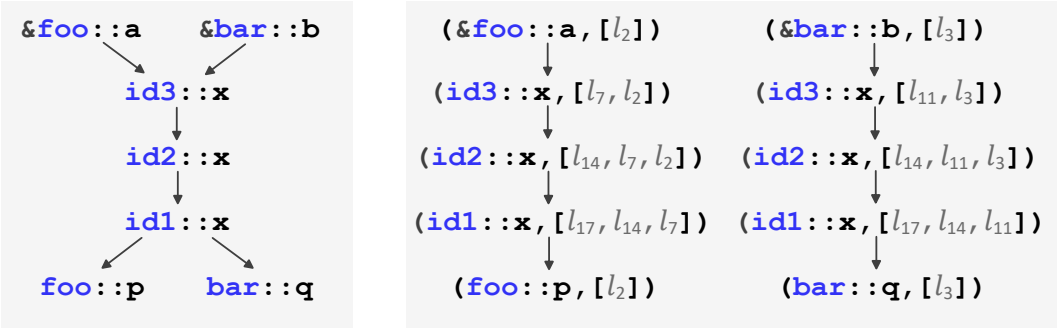
Context-sensitive pointer analysis enhances precision by treating functions distinctly based on their unique calling contexts. It assigns a context to each program element in a function, including variables and heap objects, and checks if $(o, c_o) \in \texttt{pts}_{\texttt{C}}(v, c_v)$, where c_o and c_v represent the contexts of object o and variable v, respectively. Callsite-based context sensitivity, often limited by k-limiting and widely used for context abstraction, defines a function's (calling) context by the sequence of its k most recent callsites. For scalability, the context depth is usually capped at a maximum of k, commonly no more than 3.

Returning to Figure 4.1, precise points-to information for `foo::p` and `bar::q` necessitates a 3-callsite-sensitive analysis. As shown in Figure 4.2(b), `id3()`, `id2()`, and `id1()` undergo analysis twice, each under two distinct call chains: $l_2 \rightarrow l_7 \rightarrow l_{14} \rightarrow l_{17}$ and $l_3 \rightarrow l_{11} \rightarrow l_{14} \rightarrow l_{17}$, identified by their callsites. This allows `&foo::a` and `&bar::b` to follow separate paths to `foo::p` and `bar::q`, leading to precise points-to outcomes. Thus, $\texttt{pts}_{\texttt{C}}(\texttt{foo::p}, [l_2]) = \{(\texttt{foo::a}, [l_2])\}$ and $\texttt{pts}_{\texttt{C}}(\texttt{bar::q}, [l_3]) = \{(\texttt{bar::b}, [l_3])\}$ are precisely obtained.

Under both 1-callsite and 2-callsite sensitivity, merging `&foo::a` with `&bar::b` is unavoidable, occurring at $(\texttt{id2::x}, [l_{14}])$ and $(\texttt{id1::x}, [l_{17}, l_{14}])$, respectively.

```
1  fn main() {
2      foo();
3      bar();
4  }
5  fn foo() {
6      let a: u32 = 2;
7      let p = id3(&a);
8  }
9  fn bar() {
10     let b: u32 = 3;
11     let q = id3(&b);
12 }
13 fn id3(x: &u32) -> &u32 {
14     id2(x)
15 }
16 fn id2(x: &u32) -> &u32 {
17     id1(x)
18 }
19 fn id1(x: &u32) -> &u32 {
20     x
21 }
```

Figure 4.1: A Rust program.



(a) Context-insensitive **(b) 3-callsite-sensitive**

Figure 4.2: Computing the points-to information for `foo::p` and `bar::q` in the program given in Figure 4.1 by applying context-insensitive and 3-callsite-sensitive analyses.

Consequently, spurious points-to information affects `foo::p` and `bar::q`, resulting in both having the same points-to sets, `pts(foo::p)` = `pts(bar::q)` = {`foo::a`, `bar::b`}, mirroring the results of context-insensitive analysis discussed earlier. We will soon demonstrate that stack filtering can effectively bypass these inaccuracies, obtaining precise points-to information in this case.

4.2.2 Cost of Achieving Context Sensitivity

Improving pointer analysis precision with context sensitivity significantly increases analysis time. This process requires differentiating program elements, such as vari-

```
1  fn main() {
2      m();
3      n();
4  }
5  fn m() {
6      foo();
7      bar();
8  }
9  fn n() {
10     foo();
11     bar();
12 }
13 fn foo() {
14     let a: u32 = 2;
15     let p = id(&a);
16 }
17 fn bar() {
18     let b: u32 = 3;
19     let q = id(&b);
20 }
21 fn id(x: &u32) -> &u32 {
22     x
23 }
```

Figure 4.3: A program illustrating the cost of context sensitivity.

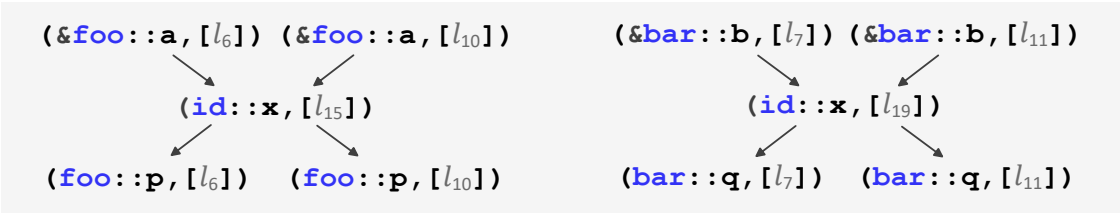


Figure 4.4: Computing the points-to information for `foo::p` and `bar::q` for the program given in Figure 4.3 by applying 1-callsite-sensitive pointer analysis.

ables and abstract locations, in a function based on its calling contexts, leading to the need to compute and maintain numerous context-sensitive points-to relations. As a result, this method leads to higher consumption of computational resources and lengthier analysis times.

In context-sensitive pointer analysis, numerous context-sensitive points-to relations $(o, c_o) \in \mathtt{pts_C}(v, c_v)$ arise, many of which are spurious due to k-limiting. While these do not always affect the precision of the final analysis, they do significantly slow down the process. To illustrate this, consider a Rust program in Figure 4.3. The functions `foo()` and `bar()` are similar to those in Figure 4.1, but with `id()` called instead of `id3()`. `id()` takes a `&u32` reference and returns it. Both `foo()` and `bar()` are called identically in `m()` and `n()`.

In 1-callsite-sensitive pointer analysis, precise points-to information for `foo::p` and `bar::q` is eventually determined, yet some context-sensitive relations during the analysis are spurious, as seen in Figure 4.4. Ultimately, it is clear that $\texttt{pts}(\texttt{foo::p}) = \{\texttt{foo::a}\}$ and $\texttt{pts}(\texttt{bar::q}) = \{\texttt{bar::b}\}$. However, `foo()` and `bar()` undergo analysis twice due to calls from `m()` and `n()`, creating two separate contexts for each function, at callsites $[l_6]$ and $[l_{10}]$ for `foo()` and callsites $[l_7]$ and $[l_{11}]$ for `bar()`. Focusing on `foo()` (for reasons of symmetry), `id()` merges $(\texttt{\&foo::a}, [l_6])$ and $(\texttt{\&foo::a}, [l_{10}])$ at $(\texttt{id::x}, [l_{15}])$. This leads to spurious points-to relations $(\texttt{foo::a}, [l_6]) \in \texttt{pts}_\texttt{C}(\texttt{foo::p}, [l_{10}])$ and $(\texttt{foo::a}, [l_{10}]) \in \texttt{pts}_\texttt{C}(\texttt{foo::p}, [l_6])$, resulting in $\texttt{pts}_\texttt{C}(\texttt{foo::p}, [l_6]) = \texttt{pts}_\texttt{C}(\texttt{foo::p}, [l_{10}]) = \{(\texttt{foo::a}, [l_6]), (\texttt{foo::a}, [l_{10}])\}$.

We will also leverage stack filtering to eliminate such spurious context-sensitive points-to relations. Though they do not affect precision, removing them can substantially increase overall efficiency, especially since such spurious points-to information might otherwise be propagated throughout many parts of the program during the analysis.

4.2.3 Stack Filtering

In this chapter, we propose using object lifetime information to remove spurious points-to relations in pointer analysis, improving both precision and efficiency. The basic idea is simple: during program execution, pointers only reference valid, accessible memory locations on the runtime stack. Therefore, an object's liveness at a program point indicates the validity of a points-to relation.

In this thesis, we focus on inclusion-based pointer analysis, which is often framed as an iterative algorithm that utilises a constraint graph to manage the flow of pointed-to objects across pointers in the program. This graph consists of edges

symbolising program statements and nodes representing variables, instance fields, or objects. Points-to information circulates along the edges of the graph, continuously updating the points-to sets at each program point. By determining if the pointed-to objects are still active at specific program points, one can detect and eliminate spurious updates in points-to relations.

In Rust, static objects persist throughout the program, and heap objects are dynamically allocated with unpredictable lifetimes. Conversely, stack objects have fixed, statically determinable lifetimes, simplifying the assessment of their validity at specific program points. This chapter focuses on using stack objects' lifetimes to reduce spuriously pointed-to stack objects in pointer analysis. For clarity and easy understanding, we emphasise a simple fact about the Rust compiler's automatic management of stack objects.

Fact 4.1 *In Rust, the lifetime of stack-allocated objects is tied directly to their declaration scope within their declaring functions. These stack objects are automatically deallocated at the end of their scope.*

Simply put, any value stored in the program's runtime stack is accessible only while its declaring function is still executing and has not completed.

4.2.3.1 Improving the Precision of Pointer Analysis

Revisiting the example in Figure 4.1, we demonstrate how stack filtering, by utilising the lifetime information of stack objects, eliminates spurious points-to relations

Figure 4.5 displays the program's stack frame allocations and deallocations. Notably, when `foo()` is called, `bar()` has not yet been called, and by the time `bar()` is invoked, `foo()` has completed its execution. Consequently, `foo::p` and `bar::q` cannot point to memory locations in the stack frames of `bar()` and `foo()`,

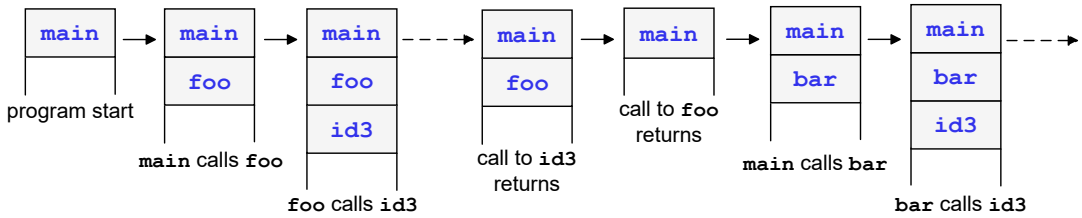


Figure 4.5: Allocations and deallocations of stack frames during the execution of the program given in Figure 4.1 (with calls to `id2()` and `id1()` omitted for brevity).

respectively. The points-to information for `foo::p` comes from `id3()`'s return value in `foo()` (line 7). At this stage, only the stack frames of `main()` and `foo()` are present on the runtime stack, making only stack objects within these frames relevant to `foo::p`'s points-to set. This allows filtering out irrelevant points-to relations from other frames, precisely determining that $\texttt{pts}(\texttt{foo::p}) = \{\texttt{foo::a}\}$, even when the pointer analysis is performed context-insensitively.

4.2.3.2 Improving the Efficiency of Pointer Analysis

As highlighted in section 4.2.2, a significant number of context-sensitive points-to relations are spurious, exemplified by $(\texttt{foo::a}, [l_6])) \in \texttt{pts}_{\texttt{C}}(\texttt{foo::p}, [l_{10}])$. This is illustrated in Figure 4.4 for the program in Figure 4.3, analysed with 1-callsite sensitivity. Although these spurious relations might not affect the precision of the final analysis, employing stack filtering can effectively eliminate them, substantially improving the overall efficiency.

In distinct calling contexts, a single function runs with different stack frames. For example, in the program depicted in Figure 3, `foo()` has two stack frames: one from its call at line 6 in `m()` and another from line 10 in `n()`, identified as $(\texttt{foo()}, [l_6])$ and $(\texttt{foo()}, [l_{10}])$, respectively. When the first frame is active, the second has not been allocated yet, rendering $(\texttt{foo::a}, [l_{10}]))$ inaccessible. Therefore,

$(\texttt{foo::a}, [l_{10}]) \in \texttt{pts}_{\texttt{C}}(\texttt{foo::p}, [l_6])$ can be filtered out. Conversely, when the second frame is active, the first has been deallocated, making $(\texttt{foo::a}, [l_6]))$ inaccessible. This allows filtering out $(\texttt{foo::a}, [l_6])) \in \texttt{pts}_{\texttt{C}}(\texttt{foo::p}, [l_{10}])$. However, in this case, removing these spurious points-to relations does not increase the final analysis's precision, as `foo::p` will eventually be determined to point to `foo::a`, as previously noted in section 4.2.2.

4.2.3.3 Determining Stack Object Liveness

To enable stack filtering, it is necessary to ascertain whether a stack object is live or not at a particular program point. In this chapter, we propose solving this by utilising the reachability relations between functions in the program. The following lemma is straightforward.

Lemma 4.1 *For a stack object o allocated in its declaring function f of a program, o is live at a program point in function f' only if f' is reachable from f in the program's call graph.*

Proof. When f' is reachable from f, it indicates a potential execution path from f to f' during program execution. As per Fact 4.1, a stack object o's lifetime is linked to its containing function f. o remains active through the execution of any subsequent functions until f completes. Thus, o can be live during f''s execution. Conversely, if f' is not reachable from f, no path exists in the call graph, and when f' runs, f is either not invoked or completed. Therefore, o is not live during f''s execution, as their stack frames cannot coexist. □

For the program in Figure 4.1, analysed in Figure 4.2, `foo()` and `bar()` are not mutually reachable in its call graph, when analysed both context-insensitively in Figure 4.2(a) and context-sensitively in Figure 4.2(b). Consequently, no stack

objects in `foo()` can be pointed to by pointers in `bar()`, and vice versa. In the program from Figure 4.3, analysed with 1-callsite sensitivity in Figure 4.4, the two instances of `foo()` at locations [l_6] and [l_{10}] in its context-sensitive call graph are not reachable from each other. Therefore, the stack object (`foo::a`, [l_{10}]) is inactive when (`foo()`, [l_6]) is executing, and the stack object (`foo::a`, [l_6]) is inactive when (`foo()`, [l_{10}]) is in execution. Thus, all the spurious points-to relations previously discussed for these two programs can be eliminated.

A sound call graph is vital for correctly determining stack object liveness. Missing a call edge can inaccurately assess function reachability, leading to potential misfiltering of valid points-to relations. Although a sound and precise call graph may be developed dynamically during pointer analysis, continuous updates to incorporate new call edges can be costly in terms of performance. To circumvent this, we use a pre-built call graph from Rapid Type Analysis (RTA) [9], reducing runtime overhead in the main pointer analysis. RTA's slightly less precise graph is still adequate for stack filtering, as shown in Section 4.4.3, making it useful as a pre-analysis. Crucially, any imprecision in this call graph does not undermine the soundness of stack filtering; overestimating function reachability merely reduces the extent of spurious points-to relation filtering, rather than erroneously eliminating valid relations.

The context-insensitive call graph produced by RTA presents a challenge for direct application in reasoning about function reachability in context-sensitive pointer analysis. To address this, we have devised a fast algorithm to derive the reachability relations of functions analysed context-sensitively from this call graph. This strategy eliminates the need for ongoing maintenance and dynamic updating of context-sensitive reachability relations throughout the pointer analysis, thereby substantially reducing both time and space complexities.

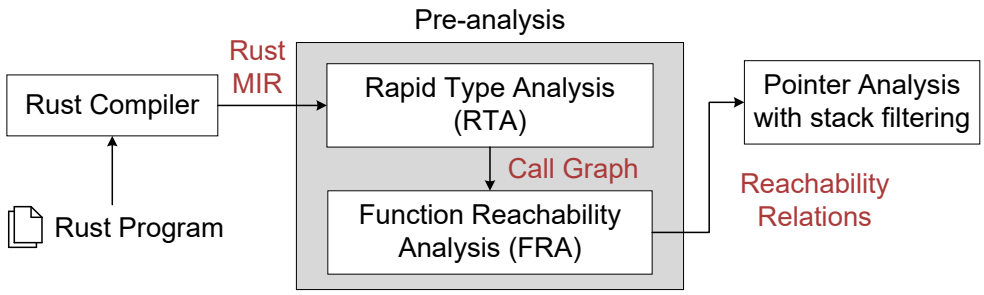


Figure 4.6: Workflow of stack-filtering-enhanced pointer analysis for Rust programs.

4.3 Stack-Filtering-Enhanced Analysis

We present stack filtering, a method to enhance pointer analysis precision and efficiency by using stack object liveness information. As outlined in Section 4.2.3, stack filtering assesses the liveness of stack objects at specific program points by analysing function reachability, utilising a pre-constructed sound call graph. In the main phase of pointer analysis, stack filtering eliminates spurious points-to relations involving stack objects that have been deallocated.

In Figure 4.6, we outline the workflow of k-callsite-based context-sensitive pointer analysis for Rust, enhanced with stack filtering. Our analysis is seamlessly integrated as a callback function within the Rust compiler, activating after the source code is translated to MIR. It starts with a lightweight pre-analysis of the MIR, comprising two components: Rapid Type Analysis (RTA) for creating an over-approximated call graph and Function Reachability Analysis (FRA) for establishing function reachability relations necessary for identifying and eliminating spurious points-to relations during the main pointer analysis phase. This process leads to callsite-sensitive pointer analysis, using these reachability relations. Throughout this analysis, function reachability information is used to eliminate spurious points-to relations, aligning with the motivation described in Section 4.2.

Below we elaborate on each of these steps in more detail.

4.3.1 Rapid Type Analysis for Rust

RTA, an interprocedural method for constructing call graphs, is widely used in object-oriented languages like C++ [9], Java [91,94], and C# [78]. Its core principle is that a receiver can only be of a type that has been instantiated. RTA thus keeps track of types instantiated in methods reachable from the main method, using this information to estimate the potential runtime types of receivers. However, due to Rust's distinct call features, outlined in Section 3.2.1, an adaptation of the RTA algorithm [9] is required for its application in Rust.

Figure 4.7 formalises the RTA algorithm adapted for Rust, computing a set R of reachable functions and a set E of call graph edges for a Rust program. We use $\texttt{type_of}(v)$ to indicate the type of a variable v, $\texttt{static_dispatch}(f)$ for the statically dispatched function from f, and $\texttt{dynamic_dispatch}_{\texttt{T}}(f, \tau)$ for the target function when f is invoked on an object of runtime type τ. Specifically, `static_dispatch` follows the same process as in Algorithm 2, while $\texttt{dynamic_dispatch}_{\texttt{T}}$ is adapted from Algorithm 3 by directly taking a possible concrete type of the receiver object as a parameter rather than retrieving it from the pointed-to objects of the call's first argument.

RTA initiates with the main function as the only initially reachable function ([RTA-Main]). For each static callsite in a reachable function f, the target f' is directly identified, added to R, and the call edge $f \to f'$ is included in E ([RTA-StaticDispatch]). The last five rules are specifically designed for Rust, based on Rust's distinct call features.

In our Rust-specific RTA, we maintain a map, `concrete_types`, linking each dynamic trait type τ to all concrete types associated with τ, rather than just

$$\frac{}{main \in R} \text{ [RTA-Main]}$$

$$\frac{\begin{array}{c}\textit{static call "}g(...)\textit{" occurs in } f \\ f \in R \qquad f' = \texttt{static_dispatch}(g)\end{array}}{f' \in R \qquad f \rightarrow f' \in E} \text{ [RTA-StaticDispatch]}$$

$$\frac{\begin{array}{c}\textit{cast stmt "}v \text{ as } \tau\textit{" occurs in } f \\ \tau \textit{ is a dynamic trait type} \qquad f \in R \qquad \tau' = \texttt{type_of}(v)\end{array}}{\tau' \in \texttt{concrete_types}(\tau)} \text{ [RTA-DynInit]}$$

$$\frac{\begin{array}{c}\textit{trait upcast stmt "}v \text{ as } \tau\textit{" occurs in } f \\ f \in R \qquad \tau' = \texttt{type_of}(v)\end{array}}{\texttt{concrete_types}(\tau') \subseteq \texttt{concrete_types}(\tau)} \text{ [RTA-TraitUpcasting]}$$

$$\frac{\begin{array}{c}\textit{dynamic call "}a.m(...)\textit{" occurs in } f \qquad f \in R \\ \tau = \texttt{type_of}(a) \qquad \tau' \in \texttt{concrete_types}(\tau) \\ f' = \texttt{dynamic_dispatch}_{\texttt{T}}(m, \tau')\end{array}}{f' \in R \qquad f \rightarrow f' \in E} \text{ [RTA-DynamicDispatch]}$$

$$\frac{\begin{array}{c}\textit{cast stmt "}f' \text{ as } \tau\textit{" occurs in } f \\ \tau \textit{ is a function pointer type} \qquad f \in R\end{array}}{f' \in \texttt{fnptr_funcs}(\tau)} \text{ [RTA-FnptrInit]}$$

$$\frac{\begin{array}{c}\textit{function pointer call "}fp(...)\textit{" occurs in } f \\ f \in R \qquad \tau = \texttt{type_of}(fp) \qquad f' \in \texttt{fnptr_funcs}(\tau)\end{array}}{f' \in R \qquad f \rightarrow f' \in E} \text{ [RTA-FnptrDispatch]}$$

Figure 4.7: A formalisation of RTA for Rust.

keeping a set of instantiated types as done for other languages. As discussed in Section 3.2.1, dynamic trait objects in Rust are generated by casting from either (1) specific instances, where a dynamic trait type τ is associated with a concrete type τ' that has been cast from τ' to τ ([RTA-DynInit]), or (2) dynamic trait objects of subtrait types, where a dynamic trait type τ is associated with all concrete types of its subtrait τ' that τ has been upcast from ([RTA-TraitUpcasting]). $\texttt{concrete_types}(\tau)$ provides an accurate estimate of τ's runtime types for dynamic

dispatch ([RTA-DynamicDispatch]). For example, in Figure 3.1, both `Circle` and `Square` objects are instantiated in reachable functions, but only the `Circle` object is cast to a dynamic `Shape` object. In this scenario, `concrete_types(&dyn Shape)` is {`&Circle`} (instead of {`&Circle, &Square`}), leading to the resolution of the dynamic call at line 18 in Figure 3.1 exclusively to `<Circle>::draw()` (instead of `<Circle>::draw()` and `<Square>::draw()`).

Predicting the potential targets of a function pointer is managed similarly. Initialising a function pointer to a specific function item occurs through a cast statement. For a function pointer type τ, we approximate its points-to set by the set of function items cast to τ, represented as `fnptr_funcs`(τ) ([RTA-FnptrInit]). Calls via function pointers of type τ are resolved using this set ([RTA-FnptrDispatch]).

4.3.2 Function Reachability Analysis

Utilising the call graph created by RTA, FRA computes the reachability relations among functions in the program. These relations follow the reflexive and transitive properties typical in graph theory, as detailed in Figure 4.8. We use $f_i \rightsquigarrow f_j$ to indicate that f_j is reachable from f_i. This analysis employs the depth-first search (DFS) algorithm, a conventional method for graph reachability issues. The resulting reachability relations are stored in a matrix, where each cell at row i

$$\frac{f_i \in R}{f_i \rightsquigarrow f_i}\ [\text{ReachReflexive}] \qquad \frac{f_i \rightarrow f_j \in E \qquad f_j \rightsquigarrow f_k}{f_i \rightsquigarrow f_k}\ [\text{ReachTransitive}]$$

Figure 4.8: Rules for computing reachability relations in a call graph $G = (R, E)$.

and column j denotes if function f_j is reachable from function f_i. Consequently, reachability queries for any pair of functions can be resolved in $O(1)$ time.

Such context-insensitive reachability relations cannot directly be applied to filter spurious points-to relations from context-sensitive pointer analysis, where functions are analysed according to their specific calling contexts. Instead, context-sensitive reachability relations, which track reachability between functions under distinct calling contexts, such as (f, c) and (f', c'), are necessary to determine whether a stack object allocated in (f, c) can be alive when (f', c') is in execution.

A simplistic method would just naively extend context-insensitive reachability from function f to f' to their context-sensitive counterparts (f, c) and (f', c'), neglecting the contexts c and c'. While this approach might work for certain programs, like the one shown in Figure 4.1, it falls short in effectively reducing spurious context-sensitive points-to relations. For instance, in the case presented in Figure 4.3, such a strategy fails to filter out spurious relations as mentioned in Section 4.2.3.2. It incorrectly assumes reachability between `(foo(), [`l_6`])` and `(foo(), [`l_{10}`])` based on `foo()`'s reflexive reachability, without accounting for the apparent context discrepancy in this case.

In k-callsite-sensitive pointer analysis, a function's context is defined by its k most recent callsites, which represent the last k functions in the call stack before the current function call. This information is crucial for precisely determining context-sensitive reachability relations from the context-insensitive reachability data acquired.

Definition 4.1 *For a call graph $G = (R, E)$, a **walk** of length n is a sequence of functions $[f_0, f_1, ..., f_{n-1}]$, where $\forall\ 0 \leq i < n$, $f_i \in R$, such that $\forall\ 0 \leq i < n - 1$, $f_i \rightarrow f_{i+1} \in E$.*

We refer to a *context-sensitive function* as a function analysed under some context.

Definition 4.2 *Every context-sensitive function* $(f, [l_1, \cdots, l_k])$ *corresponds to a walk* $[f_k, \cdots, f_1, f]$ *of length* $k+1$ *in* $G = (R, E)$*, where* f_i *is the function containing callsite* l_i*.*

Context-sensitive functions can be represented as walks, allowing for an over-approximation of context-sensitive reachability from context-insensitive reachability. This is feasible because the callsite-based context of each function can be approximated by a caller-based context in its walk. We formalise this idea in more detail below.

Lemma 4.2 *Given the call graph* $G = (R, E)$*, if a context-sensitive function* (f', c') *is reachable from* (f, c)*, denoted as* $(f, c) \rightsquigarrow (f', c')$*, then there exists a walk* W *in* G *that includes both the walk of* (f', c') *(*$W_{f'}$*) and the walk of* (f, c) *(*W_f*). In this scenario,* W_f *is either identical to* $W_{f'}$ *or the first element of* W_f *precedes the first element of* $W_{f'}$ *in* W*.*

Proof. Follows from Definitions 4.1 and 4.2 on walks and the construction of G (Figure 4.8). □

Consider two talks: $W_f = [f_k, \cdots, f_1, f]$ for (f, c) and $W_{f'} = [f'_k, \cdots, f'_1, f']$ for (f', c'). When (f', c') is reachable from (f, c), both W_f and $W_{f'}$ are part of a longer walk W. Thus, we can categorise this scenario into two cases:

- **Case 1:** The distance between W_f and $W_{f'}$ in W, i.e., the number of edges from f to f' in $G = (R, E)$ is fewer than or equal to k, indicating that (f', c') is identical to (f, c) or is called within the k calls following (f, c).

- **Case 2:** The distance between W_f and $W_{f'}$ in W is greater than k. This implies that (f', c') is called after a sequence of at least k calls subsequent to (f, c).

It is not difficult to see that if (f', c') is reachable from (f, c), either Case 1 or Case 2 applies, or both may simultaneously hold, especially in cycles present in $G = (R, E)$.

To establish a necessary condition for (f', c') to be reachable from (f, c), we state the following two conditions:

- **Cond 1:** There is an overlap of i elements between a suffix of W_f and a prefix of $W_{f'}$, where $0 < i \leq k + 1$, i.e., $\forall\ 0 \leq j < i, W_f[k + 1 - i + j] = W_{f'}[j]$.

- **Cond 2:** The starting function f'_k in $W_{f'}$ is reachable from f, the last function in W_f, i.e., $f \rightsquigarrow f'_k$

Cond 1 is a restatement of Case 1, with equivalent terms. However, Cond 2 is weaker than Case 2, since the distance between W_f and $W_{f'}$ in W exceeding k implies $f \rightsquigarrow f'_k$.

Therefore, the following theorem follows immediately.

Theorem 4.3 *If (f', c') is reachable from (f, c), then either Cond 1 or Cond 2 must hold.*

According to this theorem, Cond 1 $\vee$ Cond 2 represents a necessary condition for (f', c') to be reachable from (f, c). Therefore, we can conservatively deduce that (f', c') is reachable from (f, c) when either Cond 1 or Cond 2 is satisfied.

Revisiting the program in Figure 4.3, let us apply our reachability analysis to assess the absence of reachability between `(foo(), [`l_6`])` and `(foo(), [`l_{10}`])` in 1-callsite-sensitive analysis, as discussed in Section 4.2.3.2. The context-sensitive

functions (`foo()`, [l_6]) and (`foo()`, [l_{10}]) correspond to the walks [`m()`, `foo()`] and [`n()`, `foo()`], respectively. Since no suffix of [`m()`, `foo()`] ([`n()`, `foo()`]) overlaps with any prefix of [`n()`, `foo()`] ([`m()`, `foo()`]), Cond 1 is not met. Furthermore, since `m()` and `n()` are not reachable from `foo()`, Cond 2 does not hold either. Thus, (`foo()`, [l_6]) and (`foo()`, [l_{10}]) are not reachable from each other.

Both Cond 1 and Cond 2 can be verified efficiently, with their worst-case time complexities being $O(k^2)$ and $O(1)$, respectively, under k-limiting. Given that k is usually a small value, typically not exceeding 3, the overall time complexity for these conditions can be effectively regarded as constant.

4.3.3 Pointer Analysis with Stack Filtering

This section explains how to enhance the k-callsite-sensitive pointer analysis introduced in Section 3.3.2, with stack filtering.

To support stack filtering, we introduce $\texttt{is_alive} : \mathbb{O} \times \mathbb{C} \times \mathbb{F} \times \mathbb{C} \mapsto \{\texttt{true}, \texttt{false}\}$. This function assesses the liveness of an object o, created in context c', when accessed at a program point in function f under context c. `is_alive` is defined by two rules in Figure 4.9. Stack filtering primarily focuses on stack objects. Therefore, we conservatively consider each static or heap object always live at any program point. For a stack object, its liveness is determined based on the reachability relation between its declaring function (f', c') and the current function (f, c), as defined by our reachability analysis algorithm in Section 4.3.2 (Theorem 4.3).

We also introduce a new subset relation, denoted as $\subseteq_{f,c}$. If $\texttt{pts}_\texttt{C}(p_1, c_1) \subseteq_{f,c} \texttt{pts}_\texttt{C}(p_2, c_2)$, then the points-to set of (p_2, c_2) includes only those objects (o, c') that are pointed to by (p_1, c_1) and are live within the function (f, c). The function `is_alive` is used to determine this subset relation.

Figure 4.10 adapts the formulation of k-callsite-sensitive pointer analysis in Figure 3.7 to incorporate stack filtering, with relevant modifications highlighted in blue. The strength of this approach lies in its simplicity, achieving significant enhancements in both precision and efficiency through straightforward modifications to standard pointer analysis (Section 4.4).

The first three rules are standard and identical to those in Figure 3.7.

In an assignment $p_1 = p_2$, [SF-Assign] retrieves all pointer type field pairs $(p_1.j, p_2.j)$ and forms constraints between their points-to sets. The specialised subset relation $\subseteq_{f,c}$ is used to filter out inactive objects within function (f, c).

[SF-Load] retrieves each memory location (o, c') potentially pointed to by (v, c). It then effectively transfers the points-to information from each pointer field $(o.j.j', c')$ to $(p.j', c')$. During this process, stack filtering is applied in two phases: (1) when iterating over the points-to set of (v, c), any stack object (o, c') that is deemed inactive in (f, c), as determined by $\boldsymbol{is_alive}(o, c', f, c)$, is excluded, and (2) while propagating points-to information to $(p.j', c)$, any objects pointed to by $(o.j.j', c')$ that are inactive in (f, c) are not added to the points-to set of $(p.j', c)$ due to the use of $\subseteq_{f,c}$.

$$\frac{o \textit{ is a static or heap object}}{\texttt{is_alive}(o, _, _, _) = \texttt{true}}$$

$$\frac{\begin{array}{c} o \textit{ is a stack object} \\ f' \textit{ is the function declaring or allocating } o \qquad (f', c') \rightsquigarrow (f, c) \end{array}}{\texttt{is_alive}(o, c', f, c) = \texttt{true}}$$

Figure 4.9: Rules for computing $\texttt{is_alive}(o, c', f, c)$**.**

$$\frac{l : v = \texttt{alloc}(\tau) \qquad (j, \tau') \in \texttt{proj_ty}(\tau)}{\texttt{type_of}(v.j) = \tau'} \text{ [SF-AllocStack]}$$

$$\frac{l : p = \texttt{alloc}(\tau) \qquad c \in \texttt{ctxt_of}(f_l) \qquad (j, \tau') \in \texttt{proj_ty}(\tau)}{(o_l, c) \in \texttt{pts}_\texttt{C}(p, c) \;\; \texttt{type_of}(o_l.j) = \tau'} \text{ [SF-AllocHeap]}$$

$$\frac{l : p_1 = \&p_2 \qquad c \in \texttt{ctxt_of}(f_l)}{(p_2, c) \in \texttt{pts}_\texttt{C}(p_1, c)} \text{ [SF-AddrOf]}$$

$$\frac{\begin{array}{c} l : p_1 = p_2 \qquad c \in \texttt{ctxt_of}(f_l) \\ \tau = \texttt{type_of}(p_1) \qquad j \in \texttt{ptr_proj}(\tau) \end{array}}{\texttt{pts}_\texttt{C}(p_2.j, c) \subseteq_{f,c} \texttt{pts}_\texttt{C}(p_1.j, c)} \text{ [SF-Assign]}$$

$$\frac{\begin{array}{c} l : p = (*v).j \qquad c \in \texttt{ctxt_of}(f_l) \qquad (o, c') \in \texttt{pts}_\texttt{C}(v, c) \\ \tau = \texttt{type_of}(p) \qquad j' \in \texttt{ptr_proj}(\tau) \qquad \textit{is_alive}(o, c', f, c) \end{array}}{\texttt{pts}_\texttt{C}(o.j.j', c') \subseteq_{f,c} \texttt{pts}_\texttt{C}(p.j', c)} \text{ [SF-Load]}$$

$$\frac{\begin{array}{c} l : (*v).j = p \qquad c \in \texttt{ctxt_of}(f_l) \qquad (o, c') \in \texttt{pts}_\texttt{C}(v, c) \\ \tau = \texttt{type_of}(p) \qquad j' \in \texttt{ptr_proj}(\tau) \qquad \textit{is_alive}(o, c', f, c) \end{array}}{\texttt{pts}_\texttt{C}(p.j', c) \subseteq_{f,c} \texttt{pts}_\texttt{C}(o.j.j', c')} \text{ [SF-Store]}$$

$$\frac{\begin{array}{c} l : p = \&(*v).j \\ c \in \texttt{ctxt_of}(f_l) \qquad (o, c') \in \texttt{pts}_\texttt{C}(v, c) \qquad \textit{is_alive}(o, c', f, c) \end{array}}{(o.j, c') \in \texttt{pts}_\texttt{C}(p, c)} \text{ [SF-Gep]}$$

$$\frac{\begin{array}{c} l : p_1 = p_2 \texttt{ as } \tau \qquad c \in \texttt{ctxt_of}(f_l) \\ (o, c') \in \texttt{pts}_\texttt{C}(p_2, c) \qquad \tau' = \texttt{deref_ty}(\tau) \qquad \textit{is_alive}(o, c', f, c) \end{array}}{o_{\succ\tau'} = \texttt{transmute}_\texttt{C}(o, \tau', c') \qquad (o_{\succ\tau'}, c') \in \texttt{pts}_\texttt{C}(p_1, c)} \text{ [SF-Cast]}$$

$$\frac{\begin{array}{c} l : p_0 = \varphi(p_1, ..., p_r) \\ c \in \texttt{ctxt_of}(f_l) \qquad c' = \lceil l{::}c \rceil_k \qquad f \in \texttt{dispatch}_\texttt{C}(\varphi, p_1, c) \\ \tau_0 = \texttt{type_of}(p_0) \qquad \tau_1 = \texttt{type_of}(p_1) \qquad ... \qquad \tau_r = \texttt{type_of}(p_r) \end{array}}{\begin{array}{c} c' \in \texttt{ctxt_of}(f) \\ \forall j_0 \in \texttt{ptr_proj}(\tau_0) : \texttt{pts}_\texttt{C}(v_0^f.j_0, c') \subseteq_{f,c} \texttt{pts}_\texttt{C}(p_0.j_0, c) \\ \forall i \in [1, r], \forall j_i \in \texttt{ptr_proj}(\tau_i) : \texttt{pts}_\texttt{C}(p_i.j_i, c) \subseteq_{f,c} \texttt{pts}_\texttt{C}(v_i^f.j_i, c') \end{array}} \text{ [SF-Call]}$$

Figure 4.10: Rules for performing k-callsite-sensitive pointer analysis for Rust with stack filtering incorporated. In each rule, f_l is the function containing statement l being analysed.

[SF-Store] mirrors [SF-Load], setting an inverse subset relation between points-to sets of $(o.j.j', c')$ and $(p.j', c)$, with stack filtering functioning similarly to exclude inactive objects.

[SF-Gep] identifies objects (o, c') pointed to by (v, c) and directs (p, c) to the subobject $(o.j, c')$. Since a subobject's lifetime is tied to its containing object, stack filtering removes the points-to relation if (o, c') is not live in (f, c).

[SF-Cast] iterates over p_2's points-to set, and for each object o, its type variant $o_{\succ\tau'}$ for type τ' (the dereferenced type of τ) is obtained and added to p_1's points-to set. Stack filtering is applied by skipping objects that are not live in (f, c), determined by ***is_alive***.

[SF-Call] models a call statement as two types of assignments: (1) assigning each actual argument p_i to its corresponding formal parameter v_i^f, and (2) assigning the return variable v_0^f to the call statement's target variable p_0. Stack filtering applies $\subseteq_{f,c}$ to filter out spurious points-to relations during these assignments.

4.3.4 Soundness

The pointer analysis in Figure 4.10 remains sound with stack filtering, except for one specific scenario discussed below. Stack filtering, as incorporated in Figure 4.10, uses ***is_alive*** and $\subseteq_{f,c}$ to remove spurious points-to relations. These functions are based on function reachability analysis, which over-approximates context-sensitive reachability. Consequently, objects filtered out at any program point are considered inaccessible at that point according to Rust's semantics, ensuring the soundness of the pointer analysis with stack filtering.

However, a caveat exists for Rust programs with dangling pointer errors, as shown in Figure 4.11. Returning a pointer of type `*const i32` to the local stack object `foo::a` results in undefined behaviour. Without stack filtering, the analysis

```
1 fn foo() -> *const i32 {
2     let a = 1;
3     &a as *const i32
4 }
5 fn main() {
6     let p = foo();
7     unsafe { print!("{}", *p); }
8 }
```

Figure 4.11: A program returning a deallocated stack object.

in Figure 3.7 assumes `p` points to `foo::a`. With stack filtering, `foo::a` is excluded from `p`'s points-to set since `main()` (where `foo::a` is accessed) is not reachable from `foo()` (where `a` is created). According to Rust semantics, a sound analysis requires that the dangling pointer point to all possible memory locations. This makes it unsound by letting p point to the inactive object `foo::a` or nothing at all. C/C++ faces similar issues. As a fully sound analysis may be overly imprecise and inefficient with dangling pointers, no leading C/C++ pointer analysis frameworks, including SVF [88], address this issue in a sound manner.

However, if necessary, stack filtering can be used to identify dangling points-to relations, as these are among the spurious relations eliminated during the analysis.

4.4 Evaluation

We demonstrate that stack filtering can significantly enhance both the precision and efficiency of standard k-callsite-based context-sensitive pointer analysis for Rust. Our evaluation focuses on four research questions:

- **RQ1.** Does stack filtering improve the precision and efficiency of context-sensitive pointer analysis?
- **RQ2.** How effective is stack filtering at reducing spurious points-to relations?
- **RQ3.** Does the RTA-based pre-analysis effectively support stack filtering?

- **RQ4.** What impact does stack filtering have on precision in real-world Rust projects?

We have implemented stack filtering in RUPTA (Chapter 3) and evaluated all 13 open-source Rust projects listed in Table 3.1. Our experiments were carried out under the same setting as in Section 3.5: a system running Ubuntu 20.04.5, equipped with an Intel(R) Xeon(R) W-2275 @ 3.30GHz CPU and 512GB of RAM.

To address our research questions, we conducted a comparative analysis of RUPTA with stack filtering (*k*`CS+SF`) and without it (*k*`CS`). Our focus was on evaluating the differences in precision and efficiency, specifically for $k \in \{1, 2, 3\}$.

4.4.1 RQ1: Precision and Efficiency

Table 4.1 presents our primary findings. To measure the precision of pointer analysis, we focused on context-insensitive points-to information using two standard metrics: (1) *#dyn-call-edges*, representing the number of dynamic call edges resolved for dynamic call sites, including calls on dynamic trait objects and function pointers, and (2) *#avg-pts*, representing the average size of points-to sets for pointers. For efficiency, we evaluated the time taken and the memory used during the program analysis.

4.4.1.1 Precision

Stack filtering targets the elimination of spurious points-to relations without introducing new ones, thereby expected to improve pointer analysis precision.

Compared to *k*`CS`, *k*`CS+SF` significantly reduces the average size of points-to sets (*#avg-pts*) for each benchmark, as detailed in Table 4.1. Specifically, for $k \in \{1, 2, 3\}$, *k*`CS+SF` reduces *#avg-pts* by 57.0%, 56.8%, and 48.5%, respectively, compared to *k*`CS`. Notably, 1`CS+SF` shows higher precision than 3`CS` in *#avg-pts* for

Table 4.1: Comparison of efficiency and precision in k-callsite-sensitive pointer analyses ($k \in \{1, 2, 3\}$) with and without stack filtering. Except for "speedup" and "memory reduction" highlighted in red, smaller values indicate better performance. OOM stands for "Out of Memory".

e Program	Pre-analysis	Metrics	1CS	1CS+SF	2CS	2CS+SF	3CS	3CS+SF
atuin	RTA: 2.71s FRA: 0.27s Total: 2.98s	analysis time (s)	100.9	41.6 (2.4×)	474.2	150.6 (3.1×)	3013.5	1149.7 (2.6×)
		mem (GB)	32.6	26.1 (1.2×)	103.2	72.1 (1.4×)	375.9	299.8 (1.3×)
		#dyn-call-edges	1150	1150	1127	1127	1119	1119
		#avg-pts	56.49	14.20	34.62	9.33	21.27	6.81
bandwhich	RTA: 0.83s FRA: 0.13s Total: 0.96s	analysis time (s)	11.9	7.0 (1.7×)	33.5	14.8 (2.3×)	112.0	41.3 (2.7×)
		mem (GB)	3.2	2.2 (1.5×)	8.7	6.8 (1.3×)	26.5	18.6 (1.4×)
		#dyn-call-edges	531	531	531	531	513	496
		#avg-pts	19.57	10.38	11.66	4.69	7.02	3.44
dust	RTA: 0.75s FRA: 0.10s Total: 0.85s	analysis time (s)	7.1	4.6 (1.5×)	18.4	10.0 (1.8×)	66.3	35.7 (1.9×)
		mem (GB)	2.1	1.0 (2.1×)	7.3	3.4 (2.1×)	32.4	25.9 (1.3×)
		#dyn-call-edges	315	315	315	315	313	296
		#avg-pts	8.57	4.42	6.71	3.56	4.32	2.95
exa	RTA: 0.29s FRA: 0.01s Total: 0.31s	analysis time (s)	2.0	1.2 (1.6×)	5.4	2.3 (2.4×)	15.5	6.6 (2.4×)
		mem (GB)	0.4	0.3 (1.7×)	1.3	0.5 (3.0×)	4.3	1.3 (3.2×)
		#dyn-call-edges	185	185	185	185	184	184
		#avg-pts	5.84	3.45	5.03	2.88	3.85	2.64
fselect	RTA: 1.22s FRA: 0.14s Total: 1.37s	analysis time (s)	23.4	11.0 (2.1×)	133.2	34.0 (3.9×)	552.5	148.7 (3.7×)
		mem (GB)	8.3	3.2 (2.6×)	26.1	10.6 (2.5×)	116.8	87.7 (1.3×)
		#dyn-call-edges	749	749	748	748	679	679
		#avg-pts	24.72	7.03	20.07	5.02	13.21	4.04
gitui	RTA: 1.54s FRA: 0.13s Total: 1.67s	analysis time (s)	43.7	20.1 (2.2×)	191.4	76.6 (2.5×)	1422.1	557.2 (2.6×)
		mem (GB)	18.9	9.9 (1.9×)	52.0	36.2 (1.4×)	178.0	131.1 (1.4×)
		#dyn-call-edges	778	778	778	778	775	757
		#avg-pts	21.75	8.21	15.81	6.39	10.79	5.07
grin	RTA: 2.26s FRA: 0.27s Total: 2.53s	analysis time (s)	100.8	48.1 (2.1×)	635.0	194.6 (3.3×)	3059.0	1138.8 (2.7×)
		mem (GB)	42.1	23.7 (1.8×)	103.5	73.4 (1.4×)	418.1	337.4 (1.2×)
		#dyn-call-edges	2006	2006	2004	2004	2000	1984
		#avg-pts	52.25	16.33	46.68	14.46	26.82	9.90
lsd	RTA: 0.88s FRA: 0.13s Total: 1.00s	analysis time (s)	10.5	6.5 (1.6×)	29.1	17.1 (1.7×)	144.5	73.8 (2.0×)
		mem (GB)	3.6	1.5 (2.4×)	11.7	8.5 (1.4×)	65.1	53.4 (1.2×)
		#dyn-call-edges	318	318	317	317	316	316
		#avg-pts	10.44	4.90	6.79	3.69	4.56	3.13
mdbook	RTA: 2.99s FRA: 0.22s Total: 3.21s	analysis time (s)	47.2	26.4 (1.8×)	724.8	255.9 (2.8×)	OOM	OOM
		mem (GB)	12.1	10.1 (1.2×)	131.6	93.2 (1.4×)		
		#dyn-call-edges	921	921	919	919		
		#avg-pts	22.14	8.16	17.42	6.51		
navi	RTA: 1.21s FRA: 0.13s Total: 1.34s	analysis time (s)	16.9	12.1 (1.4×)	97.5	51.3 (1.9×)	585.2	395.7 (1.5×)
		mem (GB)	5.3	4.7 (1.1×)	22.5	18.1 (1.2×)	110.3	95.4 (1.2×)
		#dyn-call-edges	529	529	529	529	528	528
		#avg-pts	16.60	8.62	12.74	6.70	8.43	5.37
resvg	RTA: 0.65s FRA: 0.12s Total: 0.76s	analysis time (s)	18.7	11.7 (1.6×)	54.7	25.4 (2.2×)	314.5	86.2 (3.6×)
		mem (GB)	7.0	6.5 (1.1×)	24.7	20.1 (1.2×)	94.1	80.2 (1.2×)
		#dyn-call-edges	474	474	474	474	474	470
		#avg-pts	37.23	13.36	29.13	11.07	23.55	8.76
rustscan	RTA: 0.98s FRA: 0.12s Total: 1.10s	analysis time (s)	13.9	9.6 (1.4×)	74.9	44.1 (1.7×)	589.7	351.9 (1.7×)
		mem (GB)	4.0	3.2 (1.3×)	18.4	9.8 (1.9×)	93.6	80.0 (1.2×)
		#dyn-call-edges	888	888	884	884	882	882
		#avg-pts	14.05	6.45	11.16	4.86	8.78	4.23
zoxide	RTA: 0.52s FRA: 0.04s Total: 0.56s	analysis time (s)	6.3	4.4 (1.4×)	25.8	18.0 (1.4×)	141.3	109.7 (1.3×)
		mem (GB)	1.6	1.3 (1.3×)	5.7	4.5 (1.3×)	22.7	19.6 (1.2×)
		#dyn-call-edges	426	426	422	422	421	421
		#avg-pts	13.64	7.56	11.21	6.97	8.79	5.99

the majority of evaluated projects (8 out of 13), highlighting the significant role of stack filtering in enhancing points-to information precision.

With the improved points-to information from stack filtering, 1`CS+SF` produces fewer dynamic call edges (*#dyn-call-edges*) in about half of the evaluated programs compared to 3`CS`, resulting in more precise call graphs. In Rust programs, dynamic calls are less common than static calls, as indicated in Table 3.3. Consequently, the precision gains from increasing k or applying stack filtering might be subtle. Nonetheless, the notable reduction in #avg-pts underscores the effectiveness of stack filtering, especially in modern programming languages like Rust that emphasise automatic memory management and static allocation on the stack.

4.4.1.2 Efficiency

Table 4.1 highlights the speedups and memory usage improvements of k`CS+SF` compared to k`CS`, marked in red within parentheses. By minimizing spurious points-to relations, k`CS+SF` consistently outperforms k`CS` in terms of efficiency across all 13 programs. The average speedups are 1.8×, 2.4×, and 2.4× for $k = 1, 2$, and 3, respectively. The most notable improvement is seen in `fselect`, where 2`CS` takes over 2 minutes, but 2`CS+SF` completes in 34 seconds, achieving a speedup of 3.9×. Additionally, k`CS+SF` is more memory-efficient than k`CS`, thanks to its smaller points-to sets. On average, it attains memory reductions of 1.6×, 1.7×, and 1.4× for $k = 1, 2$, and 3, respectively.

The pre-analysis phase, including rapid type analysis (RTA) and function reachability analysis (FRA) shown in Figure 4.6, is highly efficient. As indicated in the second column of Table 4.1, this phase completes within 1 second for about half of the benchmarks and takes no longer than 3.21 seconds for `mdbook`. The time taken for the pre-analysis is consistently shorter than that for the subsequent pointer

analysis. Moreover, the results from this phase can be reused across different k values in kCS+SF, further enhancing its efficiency.

Table 4.2: Context-sensitive facts. For all the metrics, smaller is better.

Program	Metrics	1CS	1CS+SF	2CS	2CS+SF	3CS	3CS+SF
atuin	#points-to-relations	106.4M	14.8M	367.9M	44.3M	1676.2M	280.6M
	#points-to-stack-relations	94.2M	3.1M	349.1M	25.6M	1619.9M	224.4M
	#avg-cs-pts	140.87	19.69	251.98	30.43	584.95	98.42
bandwhich	#points-to-relations	13.2M	3.8M	40.3M	6.5M	129.7M	13.5M
	#points-to-stack-relations	10.2M	807.5K	36.1M	2.4M	123.3M	7.1M
	#avg-cs-pts	47.60	13.63	76.26	12.32	133.45	14.02
dust	#points-to-relations	7.1M	1.5M	21.5M	3.5M	59.3M	13.3M
	#points-to-stack-relations	6.1M	588.4K	19.5M	1.5M	50.7M	4.8M
	#avg-cs-pts	26.72	5.74	41.66	6.87	61.51	13.92
exa	#points-to-relations	1.8M	422.9K	6.0M	883.2K	20.3M	2.7M
	#points-to-stack-relations	1.6M	191.0K	5.6M	455.7K	18.9M	1.3M
	#avg-cs-pts	20.28	4.71	33.44	4.95	58.84	7.92
fselect	#points-to-relations	26.7M	3.6M	151.6M	10.2M	494.9M	39.9M
	#points-to-stack-relations	24.2M	1.1M	147.1M	5.7M	472.6M	17.6M
	#avg-cs-pts	69.79	9.34	196.80	13.24	321.34	25.99
gitui	#points-to-relations	39.4M	6.7M	143.3M	24.4M	721.0M	154.1M
	#points-to-stack-relations	35.8M	3.2M	137.0M	18.0M	704.0M	137.1M
	#avg-cs-pts	73.40	12.56	139.27	23.71	355.04	76.36
grin	#points-to-relations	103.3M	22.6M	374.3M	74.9M	1317.1M	298.8M
	#points-to-stack-relations	93.6M	13.0M	354.2M	54.8M	1269.8M	251.7M
	#avg-cs-pts	129.44	28.35	252.43	50.62	470.33	107.91
lsd	#points-to-relations	10.8M	2.3M	29.3M	6.7M	97.7M	23.4M
	#points-to-stack-relations	9.3M	827.1K	25.3M	2.6M	85.4M	11.1M
	#avg-cs-pts	31.07	6.65	42.62	9.70	74.61	17.87
mdbook	#points-to-relations	49.5M	8.1M	412.1M	88.9M	OOM	OOM
	#points-to-stack-relations	45.2M	3.8M	401.0M	77.8M		
	#avg-cs-pts	72.29	11.78	133.34	28.79		
navi	#points-to-relations	20.5M	5.6M	90.1M	27.4M	479.7M	160.1M
	#points-to-stack-relations	17.5M	2.8M	83.0M	20.4M	461.5M	141.9M
	#avg-cs-pts	49.31	13.61	108.85	33.15	293.51	98.05
resvg	#points-to-relations	27.6M	7.2M	65.6M	13.0M	211.9M	43.6M
	#points-to-stack-relations	21.1M	651.3K	54.2M	1.5M	172.1M	3.8M
	#avg-cs-pts	75.55	19.61	94.41	18.63	157.48	32.44
rustscan	#points-to-relations	14.9M	3.5M	78.6M	19.2M	528.5M	128.1M
	#points-to-stack-relations	13.1M	1.7M	75.1M	15.8M	514.0M	113.6M
	#avg-cs-pts	41.78	9.91	108.13	26.46	355.19	86.30
zoxide	#points-to-relations	7.6M	2.8M	30.5M	13.0M	193.8M	89.1M
	#points-to-stack-relations	6.7M	1.8M	28.4M	10.9M	188.5M	83.9M
	#avg-cs-pts	38.39	13.96	79.74	33.95	253.94	116.99

Table 4.2 outlines three key metrics for evaluating context-sensitive pointer analysis on each program: (1) *#points-to-relations*: the number of context-sensitive points-to relations identified, (2) *#points-to-stack-relations*: the number of context-sensitive points-to relations involving stack objects, and (3) *#avg-cs-pts*: the av-

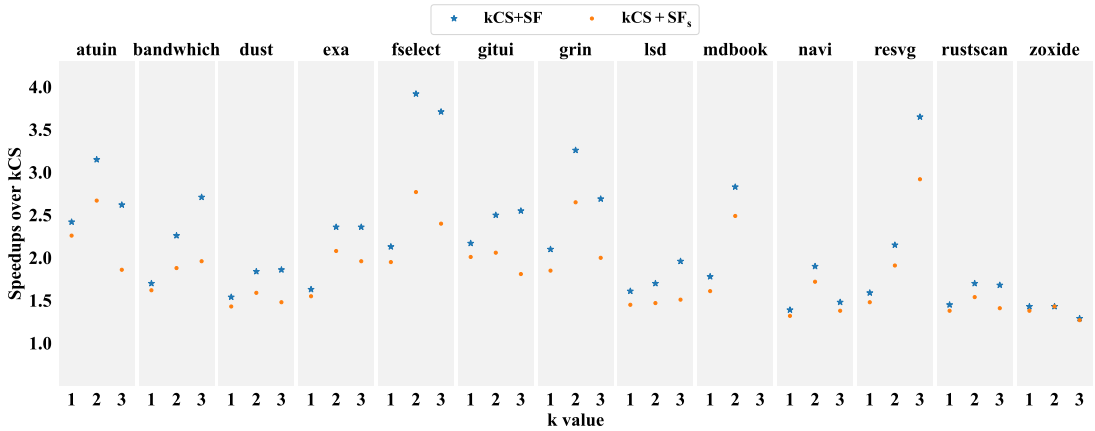


Figure 4.12: Speedups of kCS+SF and kCS+SF$_s$ over kCS.

erage size of context-sensitive points-to sets. Compared to kCS, kCS+SF achieves both reduced analysis time and lower memory usage, primarily due to a notable decrease in context-sensitive facts computed. The reduction is particularly marked in `fselect`, with a significant decrease across all three metrics. For 2CS and 3CS, *#points-to-stack-relations* account for over 90% and 95% of *#points-to-relations* in most benchmarks, highlighting the importance of minimizing spurious points-to relations involving stack objects. This enhances the precision and efficiency of pointer analysis, demonstrating the critical role of stack filtering in optimizing analysis for modern languages like Rust.

We compared kCS+SF and kCS+SF$_s$, with the latter using a simplistic stack filtering approach as described in Section 4.3.2. In this naive approach, $(f, c) \rightsquigarrow (f', c')$ is assumed if and only if $f \rightsquigarrow f'$. While this conservative method can filter out spurious points-to relations for Figure 4.1, it is less effective for Figure 4.3, as also discussed in Section 4.3.2.

kCS+SF and kCS+SF$_s$ exhibit similar *#avg-pts* across all 13 programs, with kCS+SF being slightly more precise as most eliminated spurious context-sensitive relations do not affect the final context-insensitive points-to information (as in Fig-

ure 4.3). Thus, we focus on their efficiency differences. Figure 4.12 illustrates the speedups of kCS+SF and kCS+SF$_s$ over kCS, with kCS+SF's more accurate reachability assessments leading to greater efficiency improvements. As k increases, kCS generates more spurious context-sensitive points-to relations. Not filtering these can hinder efficiency. Therefore, kCS+SF generally surpasses kCS+SF$_s$ in efficiency, particularly at higher k values.

4.4.2 RQ2: Reduction of Spurious Points-to Relations

Table 4.2 shows that kCS+SF, with stack filtering, significantly reduces the number of points-to relations processed and maintained by the baseline kCS. The average reduction rates are 77.6%, 79.6%, and 78.1% for $k = 1, 2$, and 3, respectively.

To understand stack filtering's effectiveness, we analysed the proportion of context-sensitive points-to relations that are (1) preserved, (2) directly eliminated (using the rules in Figure 4.10), and (3) indirectly eliminated due to direct eliminations' cascading effect. This analysis, detailed in Figure 4.13, was performed for $k \in \{1, 2, 3\}$. The findings reveal that while only a small fraction of spurious relations are directly filtered out, a substantial majority, varying from around 50% to over 90%, are indirectly eliminated. With the increase in k, there is a rise in the occurrence of spurious points-to relations. This increase in spurious relations results in a reduction in the percentage of direct eliminations, which drops to below 5% when k reaches 3.

During analysis, stack filtering continuously assesses stack object liveness to identify spurious points-to relations. While this adds computational costs, the substantial net performance gain outweighs them. Eliminating a single spurious relation can trigger a chain reaction that removes many dependent relations, significantly offsetting the added costs.

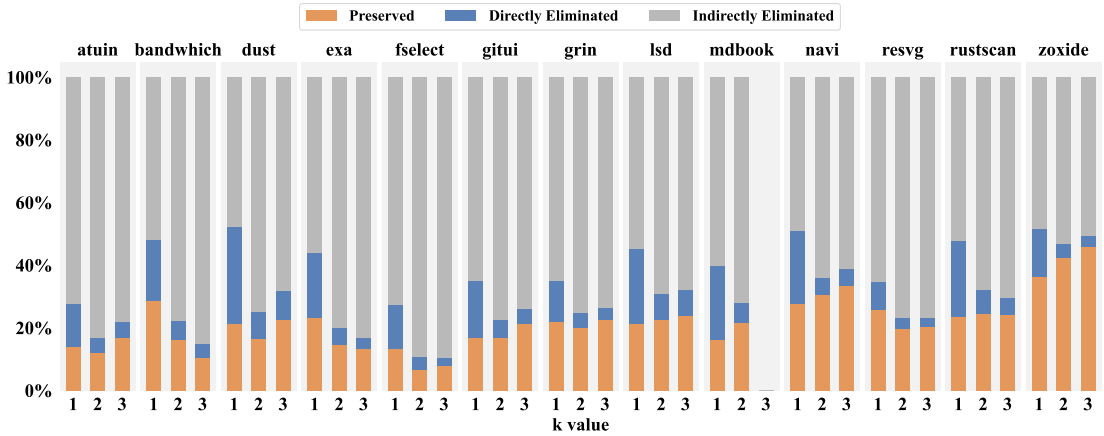


Figure 4.13: Percentages of points-to relations preserved, directly and indirectly eliminated by stack filtering.

4.4.3 RQ3: Effectiveness of RTA-based Pre-analysis

Stack filtering utilises a pre-built, sound call graph to filter out spurious points-to relations. The precision of this call graph significantly affects stack filtering's effectiveness, with higher precision leading to identifying more spurious relations. We use RTA for call graph construction, balancing efficiency with reasonable precision. Additionally, in Rust programming, static dispatch is commonly preferred over dynamic dispatch. As shown in Table 3.3, Rust programs typically feature few dynamic calls. Therefore, even if RTA's resolution of dynamic calls is somewhat imprecise, it is unlikely to majorly impact the call graph's overall precision.

To demonstrate the effectiveness of the RTA-based pre-analysis, we compared k`CS+SF` with k`CS+SF`$_k$, where the latter augments k`CS` with an 'oracle' stack filtering. Specifically, k`CS+SF`$_k$ runs k`CS` twice. The first run constructs a call graph on-the-fly, which is more precise than the RTA-based call graph used in k`CS+SF`. Based on this call graph, it directly computes context-sensitive reachability relations between functions. The second k`CS` run then applies stack filtering using these reachability relations. With more precise reachability information, k`CS+SF`$_k$ is expected to outperform k`CS+SF` in terms of precision.

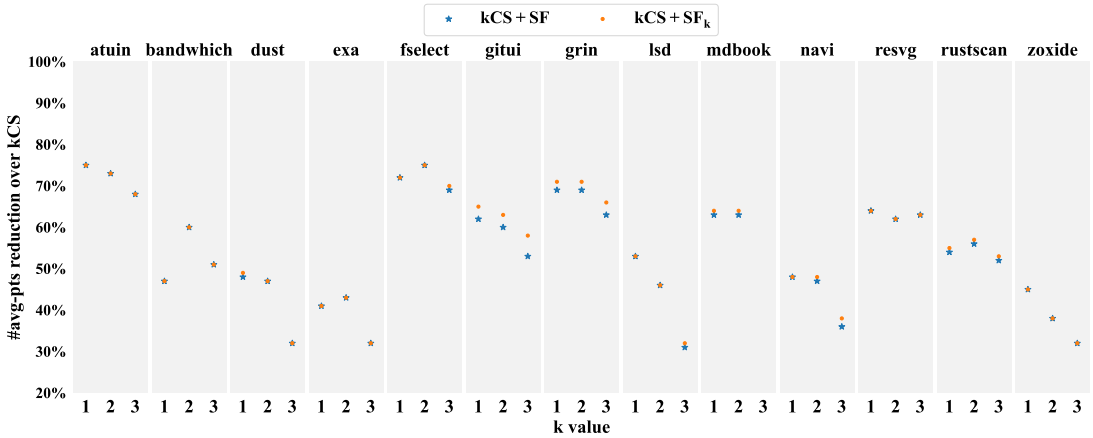


Figure 4.14: Precision gains of kCS+SF and kCS+SF$_k$ over kCS.

Figure 4.14 compares the precision enhancements of kCS+SF and kCS+SF$_k$ over the baseline kCS, focusing on the percentage reduction in *#avg-pts*. Across most of the 13 evaluated programs, kCS+SF performs comparably to kCS+SF$_k$ at various k values, with only minor differences in some programs like `gitui` and `grin`. These disparities stem from two main factors: the call graph constructed by RTA is slightly less precise than that of kCS, and RTA's reachability analysis is less precise than kCS's. These results imply that the precision gain from stack filtering based on a dynamically constructed call graph in pointer analysis is minimal.

Despite these precision variances, our RTA-based pre-analysis is lightweight and highly effective. It avoids directly managing reachability relations between context-sensitive functions. For instance, in `atuin` under 3CS, managing over 700K context-sensitive functions requires more than 57 GB to store reachability relations in a bit matrix, while context-insensitive ones need less than 1 GB.

4.4.4 RQ4: Real-World Cases Study

We examine an example of array slicing given in Figure 4.15, abstracted from real-world Rust programs, to demonstrate how stack filtering enhances the pre-

```
1  fn foo() {
2    let x: [i32; 5] = [1, 2, 3, 4, 5];
3    let p: &[i32] = &x[0..4];
4  }
5  fn bar() {
6    let y: [i32; 5] = [6, 7, 8, 9, 10];
7    let q: &[i32] = &y[2..3];
8  }
9  impl Index for [i32; 5] {
10   fn index(&self, r: Range) -> &[i32] {
11     <[i32]>::index(self as &[i32], r)
12   }
13 }
14 impl Index for [i32] {
15   fn index(&self, r: Range) -> &[i32] {
16     <Range>::index(r, self)
17   }
18 }
19 impl SliceIndex<[i32]> for Range {
20   fn index(self, s: &[i32]) -> &[i32]
21   {
22     let ret = self.get_unchecked(
23       s as *const [i32]
24     )
25     &(*ret)
26   }
27   fn get_unchecked(self,
28     s: *const [i32]) -> *const [i32]
29   {
30     let new_len = ...;
31     ptr::slice_from_raw_parts(
32       s.as_ptr().add(self.start),
33       new_len
34     )
35   }
36 }
```

Figure 4.15: A program involving array slicing.

cision of pointer analysis. Arrays are fixed-size sequences (like `[i32; 5]`), while slices are dynamically sized, fat pointers referencing a sequence (e.g., `&[i32]`). Array slicing, as shown at lines 3 and 7, creates slices to reference parts of arrays. Typically, pointer analysis treats these operations as array-element-insensitive, so slices `p` and `q` are inferred to point to arrays `x` and `y`, respectively. However, slicing is complex: it translates to calls to the `<[i32; 5]>::index` function (line 10), leading to a series of calls: `<[i32]>::index()` (line 11), `<Range>::index()` (line 16), `<Range>::get_unchecked()` (line 22) and `ptr::slice_from_raw_parts()` (line 31). These functions resemble the `id` functions in Figure 4.1. Under 3-callsite-sensitive analysis, `foo::x`'s and `bar::y`'s flows merge, leading to the imprecise set: `pts(foo::p)` = `pts(bar::q)` = {`foo::x`, `bar::y`}. With stack filtering, `foo()` and `bar()` are deemed mutually unreachable, eliminating the spurious `bar::y` from the points-to set `pts(foo::p)` and `foo::x` from the points-to set `pts(bar::q)` respectively.

In Rust programs, it is common to slice strings or vectors, resulting in slices pointing to heap-allocated memory. However, this slicing often utilises functions similar to those used for array slicing, leading to incorrect associations where slices appear to point to stack objects. Stack filtering effectively addresses this issue by removing these spurious points-to relations, ensuring more accurate memory location tracking for slices during the pointer analysis.

4.5 Summary

In this chapter, we present stack filtering, a novel method that significantly improves both the precision and efficiency of context-sensitive pointer analysis for Rust programs. This approach marks the first foray into using stack object liveness information to refine context-sensitive pointer analysis. Stack filtering works by identifying and removing spurious points-to relations during analysis at specific program points, targeting relations where the pointed-to objects are no longer live. Designed for simplicity and efficiency, stack filtering assesses the liveness of stack objects utilizing function reachability relations derived from a sound call graph, which is pre-constructed using Rapid Type Analysis (RTA). This filtering process is integrated seamlessly into the propagation phase of pointer analysis, effectively eliminating spurious points-to relations. Through evaluations on real-world Rust projects, we demonstrate the substantial benefits of stack filtering, highlighting its capability to enhance pointer analysis in practical applications.

Chapter 5

Related Work

In this chapter, we delve into related work closely tied to Rust pointer analysis (yet not reviewed earlier in the thesis). As Rust's popularity grows, particularly in low-level system development [4, 18, 41, 71], there has been a surge in the development of Rust program analysis and verification tools in recent years [7, 10, 54, 75, 102]. However, research focused on Rust's pointer analysis remains limited.

5.1 Program Analysis for Rust

Several studies have adapted existing program analysis tools for Rust. Linder *et al.* [56] adopted KLEE, a symbolic execution engine [20], to verify whether a Rust program or library is panic-free by analysing the LLVM code compiled from the Rust source code. Rust2Viper [32] and Prusti [5, 7] verify functional correctness properties of Rust programs based on the Viper verification infrastructure [70]. Baranowski et al. [13] support the verification of Rust by extending SMACK [76], a software verification toolchain that translates LLVM IR code into Boogie intermediate verification language [14], which is in turn verified using back-end Boogie

verifiers. CRUST [95] and Kani [97] translated Rust code to C and verified it using the CBMC model checker [22].

There have been many static analysis tools that target Rust MIR. Rudra [10] identifies three bug patterns in unsafe Rust and discovers them at the MIR level. Facebook's experimental MIRAI [68] is a formal verification tool that performs symbolic execution on Rust MIR, allowing users to add annotations and leveraging the SMT solver Z3 [27] to prove program correctness. Implemented based on MIRAI, MirChecker [54] collects both the numerical and symbolic information from Rust MIR to detect runtime panics and memory-safety issues. SafeDrop [24] targets heap memory deallocation in Rust, detecting potential memory corruptions through a lightweight alias analysis. rCanary [26] introduces an encoder to abstract data having heap allocation and a leak-free memory model to automatically detect memory leaks in Rust packages. FFIChecker [55] detects memory bugs resulting from errors in integrating Rust code with C/C++ using FFI based on augmented taint analysis. Many of these tools rely on imprecise points-to/alias information and call graphs in analysis, hindering their bug detection capabilities.

Some dynamic analysis tools for Rust have been developed, which complements static analysis by testing and evaluating a program during its execution. Miri [69], an interpreter for Rust's MIR, plays a critical role in this space by executing code in an interpreted environment to catch undefined behaviours, including memory safety violations, data races, and out-of-bounds accesses in unsafe Rust. Jung at al. proposed Stacked Borrows [46], an aliasing model built on Miri that utilises a stack to dynamically track all valid references and pointers to each memory location. This model reports potential undefined behaviour and memory bugs when references are not used in a properly nested manner. XRust [57] and TRust [12] separates heap memory into two mutually exclusive regions: an unsafe region and

a safe region, where unsafe objects (i.e. memory objects potentially accessed by unsafe code) are placed in the unsafe region, and safe objects (accessed only by safe code) are stored in the safe region. XRust prevents any cross-region memory corruption when an instruction that should access an unsafe object accesses an safe object. ERASan [67] builds on Address Sanitizer (ASan) [79], using a specialised memory allocator and code instrumentation to detect memory safety violations in Rust programs. It optimises detection by instrumenting memory accesses only in code areas where Rust cannot guarantee safety.

Several empirical studies have investigated Rust's unique safety and performance characteristics. Astrauskas et al. [6] and Evans et al. [30] studied empirically how unsafe code is used in practice by analysing a large corpus of Rust projects. Xu et al. [105] conducted a survey of all Rust common vulnerability and exposures (CVEs) related to memory-safety issues prior to 2021, assessing Rust's effectiveness in preventing memory bugs and understanding common patterns of Rust memory bugs. In a similar vein, Qin et al. [74, 75] investigated memory and concurrency bugs across open-source Rust projects, libraries, and security databases, and developed static bug detectors based on simple heuristics informed by their findings. Cui et al. [25] analysed unsafe API documents in the standard library to infer safety properties, and then categorised unsafe APIs and existing CVEs based on these safety properties.

5.2 Pointer Analysis

5.2.1 Pointer Analysis Frameworks

As a complex problem in static program analysis, pointer analysis has been the subject of extensive research, leading to a wide array of approaches and tools, par-

ticularly for languages like Java and C/C++. For Java, Soot [96] and WALA [99] are widely used. Soot provides a comprehensive suite for Java bytecode analysis, including pointer analysis (known as Spark [49]) and other interprocedural analyses, and has been applied extensively in academic research. WALA, developed by IBM, also supports context-sensitive pointer analysis and is designed for large-scale applications, offering various tools for analysing Java bytecode and JavaScript. Another framework, Doop [19], performs pointer analysis on Java using a declarative, Datalog-based approach. Qilin [39] is a new pointer analysis framework designed for supporting fine-grained context-sensitivity for Java programs. For C/C++, SVF (Static Value-Flow Analysis) is a state-of-the-art pointer analysis framework built on LLVM IR that provides support for performing flow-sensitive pointer analysis on a sparse flow value graph. cclyzer++ [11, 15] is another LLVM-based framework based on a Datalog engine, enabling highly parallel and scalable analysis. In this thesis, We present Rupta, the first context-sensitive pointer analysis framework for Rust. Operating on Rust MIR, Rupta addresses key challenges in achieving precise pointer analysis and call graph construction.

5.2.2 Optimisations for Pointer Analysis

Recent research in pointer analysis has primarily focused on enhancing its effectiveness. In C/C++ pointer analysis, a common optimisation involves identifying pointers with guaranteed equivalent points-to sets, typically occurring in pointer flow graph cycles, and replacing them with a single representative pointer [33, 73, 77]. This optimisation reduces the problem's input size, thereby decreasing the running time and memory requirements of C/C++ pointer analysis. However, when applied to pointer analysis in other languages like Java, its performance improvements may be modest [40]. Hardekopf et al. [33] further leveraged location

equivalence to reduce the size of variables' points-to sets without sacrificing precision. They considered memory locations equivalent if they always belonged to the same points-to set. Tan et al. [93] introduced MAHJONG, which merges type-consistent heap abstractions to enhance the efficiency of pointer analysis while preserving precision for type-dependent clients like call graph construction and may-failing casting. However, this approach does compromise the precision of points-to information.

To enhance the efficiency of context-sensitive pointer analysis in Java, recent techniques have been focused on selective context sensitivity. The core concept involves choosing a subset of methods or variables that significantly contribute to analysis precision and exclusively applying context sensitivity to them. Various selection strategies have been proposed, utilising diverse schemes such as parameterized heuristics [35], machine-learning methods [42, 44], pattern matching for addressing imprecision in dataflow [51, 52], and CFL-reachability approaches [36,37,60,62]. These approaches effectively expedite context-sensitive pointer analysis while maintaining its precision or incurring only a negligible loss in precision. Furthermore, various approaches aim to enhance the precision of k-limited context-sensitive pointer analysis by optimising the selection of context elements. This optimisation is motivated by the observation that updating a method's context at every callsite can result in numerous context elements that offer little improvement in precision while consuming the limited available context slots. Tan et al. introduced BEAN [92], a tool designed to identify and eliminate redundant context elements. It achieves this by leveraging the object allocation graph they proposed. Jeon et al. introduced the concept of "context tunneling" [43], which utilises a machine-learning scheme to selectively update contexts and determine when to retain the

same context without modifications. These approaches contribute to refining the precision of k-limited context-sensitive pointer analysis.

In this thesis, we present stack filtering for Rust pointer analysis, an approach operating independently of the optimisation techniques commonly used in context-sensitive pointer analysis. Stack filtering relies on object liveness information to effectively filter out spurious points-to relations, a task that might otherwise require a longer context length in the analysis.

Chapter 6

Conclusions and Future Work

In this chapter, we conclude the thesis, summarising the key contributions, and then explore potential directions for future research.

6.1 Conclusions

Pointer analysis is an essential program analysis technique, critical for understanding memory usage and data dependencies in software systems. Traditionally, it has been extensively applied to languages like C/C++ and Java to improve compiler optimisations, security assessments, and bug detection. Rust, an increasingly popular system-level language, brings unique safety guarantees through its type system and borrow checker, enforcing safe memory access patterns. However, analysing pointer behaviour in Rust, especially within its unsafe code sections, remains crucial for optimising performance and detecting subtle bugs. Developing effective pointer analysis techniques tailored to Rust is therefore essential, as it enables accurate call graph construction, enhances compiler optimisations, and strengthens safety vulnerability detection.

This thesis contributes by introducing RUPTA, the first context-sensitive pointer analysis framework for Rust, along with a novel technique, *stack filtering*, which markedly enhances RUPTA's effectiveness by improving its precision and efficiency.

Chapter 3 presented RUPTA, which functions on Rust MIR and incorporates callsite-based context-sensitivity. RUPTA supports on-the-fly call graph construction and addresses key challenges unique to Rust's pointer analysis. Specifically, RUPTA accurately models various kinds of call semantics, including static and dynamic dispatch, calls on `Fn*` trait objects and function pointers, ensuring precise determination of call targets for each kind of call. Furthermore, RUPTA effectively handles pointer casts while maintaining precise type information for objects during analysis by adopting a novel approach which symbolises memory objects with multiple abstractions of different types. Additionally, RUPTA manages the complex memory access paradigms of Rust MIR, which operates at a higher level of abstraction than LLVM, and uses a projection-based representation for structure fields, preventing type confusion during analysis.

Chapter 4 presented stack filtering, a new approach for enhancing the effectiveness of Rust pointer analysis by utilising lifetime information. The key idea is to filter out spurious points-to relations involving stack objects that fall outside their valid lifetimes. Here the liveness of a stack objects can be determined based on the function reachability relationship. Stack filtering is lightweight, using an efficient pre-analysis composed of Rapid Type Analysis (RTA) for call graph construction and function reachability analysis for computing the reachability relationship. During pointer analysis, stack filtering is then applied to eliminate points-to relations where the pointed-to stack object is inactive. This approach effectively mitigates spurious points-to relations, which would otherwise require deeper context sensitivity, thus improving the precision of pointer analysis.

RUPTA, integrated with stack filtering, is available as an open-source project at https://rustanlys.github.io/rupta/. We anticipate that it will serve as a foundation for developing advanced program analysis tools that can enhance the safety and security of Rust programs.

Despite the advancements presented in this thesis, Rust pointer analysis still requires further optimisation and exploration. Below, we discuss several potential directions for future work, with the hope that these ideas will inspire further research in this challenging area.

6.2 Future Work

6.2.1 Different Context Sensitivities for Rust Pointer Analysis

In this thesis, we implemented and evaluated callsite-sensitivity in RUPTA, showing its effectiveness in analysing Rust programs, especially in handling library functions. Exploring alternative context-sensitivity approaches specifically tailored to Rust pointer analysis presents an intriguing opportunity. Several well-established context sensitivities, such as object sensitivity and type sensitivity, have proven highly effective for analysing object-oriented languages like Java. However, adapting these context sensitivities to Rust pointer analysis introduces unique challenges, given the fundamental differences between Rust and traditional object-oriented languages. For instance, Rust's trait system inherently discourages the use of complex class hierarchies, and its strict safety rules render many design patterns common in Java impractical. Exploring how to effectively tailor these context sensitivities to Rust's unique characteristics remains an important direction for future work.

In addition, designing a custom context-sensitivity approach specifically for Rust could offer substantial benefits, particularly by taking advantage of Rust's strict memory management rules and lifetime tracking. A Rust-tailored sensitivity could strike a more balanced trade-off between precision and performance, making this a promising direction for further research and enhancement within Rupta.

6.2.2 Flow-Sensitive Pointer Analysis for Rust

Rupta currently operates with context sensitivity and flow insensitivity. Expanding Rupta to support flow sensitivity can further enhance its precision, unlocking more opportunities for compiler optimisations and reducing false positives in bug detection tools. However, implementing flow-sensitivity would require careful optimisation to manage the added complexity and potential performance impact, especially for analysing large-scale programs. Sparse flow-sensitive analysis [34,89], which optimises the analysis by operating on a sparse value flow graph that captures def-use chains and value flows, has shown promising results in C/C++. However, this method is typically built around SSA-form IR, which Rust MIR does not follow. Adapting sparse analysis to Rust MIR would require additional transformations. Designing a flow-sensitive technique specifically suited for Rust presents a challenging and promising direction.

6.2.3 Demand-Driven Pointer Analysis for Rust

Rupta currently performs full-program analysis, processing the entire codebase to capture all pointer relations. Supporting demand-driven pointer analysis in Rupta could improve efficiency by focusing only on parts of the program that are relevant to specific queries or analysis needs, rather than analysing the entire program. This approach could be particularly beneficial for analysing unsafe Rust, where Rust's

usual memory safety guarantees are bypassed, introducing potential memory risks. By concentrating on high-risk unsafe Rust code, demand-driven pointer analysis can effectively reduce time and memory overhead, enabling more efficient real-time analysis within interactive development tools or IDEs. This would also provide more targeted insights for developers working with unsafe Rust, supporting safer and more efficient handling of memory in critical parts of Rust programs.

6.2.4 Leveraging Rust's Unique Features

Building on Rust's unique language features—such as ownership, lifetimes, and the borrowing mechanism—opens up highly promising future directions for Rust pointer analysis. In this thesis, we introduced stack filtering, a novel technique that leverages the lifetimes of stack objects to eliminate spurious points-to information. We approximate the lifetime of stack objects using the scope of their declaring functions, which, while generally effective, may be less precise in certain scenarios. Rust's borrow checker enforces strict lifetime constraints on borrowed data and references, playing a crucial role in establishing valid borrows, which provides opportunities to identify and eliminate more spurious points-to relations. Additionally, Rust's aliasing restrictions, such as the rule that a mutable reference can only exist if no other references to the same data exist, offer further insights to optimise points-to results. By incorporating these lifetime and pointer aliasing features, we expect Rupta to achieve high precision and efficiency.

6.2.5 Other Potential Directions

Several other promising directions could also enhance Rupta's capabilities and application. First, selective context sensitivity has proven beneficial in Java pointer analysis. Adapting this approach to Rust pointer analysis could potentially op-

timise RUPTA's performance further, particularly in contexts where deep context sensitivity is not uniformly necessary. Second, Rust programs often interface with other languages like C and C++ through Foreign Function Interface (FFI). Currently, RUPTA does not analyse these FFI interactions, which limits its ability to address cross-language pointer and memory issues. Future work will involve extending RUPTA's analysis capabilities to cover FFI boundaries, allowing it to more accurately reflect Rust programs' interactions with external libraries. Lastly, while we have demonstrated RUPTA's effectiveness in constructing call graphs, applying RUPTA to additional client analyses could more comprehensively validate its utility. We aim to explore RUPTA's effectiveness in identifying vulnerabilities in Rust programs, enhancing its relevance in real-world safety analysis. We also plan to leverage the points-to information to assist in compiler optimisations, such as inlining and alias-based transformations. Furthermore, there has been a growing body of work on automatically translating C code into Rust, though the translated output often exhibits extensive use of raw pointers. We aim to utilise the points-to information produced by RUPTA to eliminate unnecessary raw pointer usages and help migrate unsafe code into safe abstractions.

Pursuing these directions could significantly broaden RUPTA's impact, bringing it closer to a versatile analysis framework for Rust.

Bibliography

[1] The Rust programming language. https://doc.rust-lang.org/stable/book/, 2022.

[2] Rust for Linux. https://github.com/Rust-for-Linux/linux, 2023.

[3] L. O. Andersen. Program analysis and specialization for the C programming language. *PhD Thesis, University of Copenhagen*, 1994.

[4] B. Anderson, L. Bergstrom, M. Goregaokar, J. Matthews, K. McAllister, J. Moffitt, and S. Sapin. Engineering the Servo web browser engine using Rust. In *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 81–89, 2016.

[5] V. Astrauskas, A. Bílý, J. Fiala, Z. Grannan, C. Matheja, P. Müller, F. Poli, and A. J. Summers. The Prusti project: Formal verification for Rust. In *NASA Formal Methods: 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings*, page 88–108, Berlin, Heidelberg, 2022. Springer-Verlag.

[6] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. How do programmers use unsafe Rust? *Proc. ACM Program. Lang.*, 4(OOPSLA), Nov. 2020.

[7] V. Astrauskas, P. Müller, F. Poli, and A. J. Summers. Leveraging Rust types for modular specification and verification. *Proc. ACM Program. Lang.*, 3(OOPSLA), Oct. 2019.

[8] D. Avots, M. Dalton, V. B. Livshits, and M. S. Lam. Improving software security with a C pointer analysis. In *Proceedings of the 27th International Conference on Software Engineering*, ICSE '05, page 332–341, New York, NY, USA, 2005. Association for Computing Machinery.

[9] D. F. Bacon and P. F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA '96, page 324–341, New York, NY, USA, 1996. Association for Computing Machinery.

[10] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim. Rudra: Finding memory safety bugs in Rust at the ecosystem scale. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 84–99, New York, NY, USA, 2021. Association for Computing Machinery.

[11] G. Balatsouras and Y. Smaragdakis. Structure-sensitive points-to analysis for C and C++. In *Static Analysis: 23rd International Symposium, SAS 2016, Edinburgh, UK, September 8-10, 2016, Proceedings 23*, pages 84–104. Springer, 2016.

[12] I. Bang, M. Kayondo, H. Moon, and Y. Paek. TRust: A compilation framework for in-process isolation to protect safe Rust against untrusted code. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6947–6964, Anaheim, CA, Aug. 2023. USENIX Association.

[13] M. Baranowski, S. He, and Z. Rakamarić. Verifying Rust programs with SMACK. In *Automated Technology for Verification and Analysis: 16th International Symposium, ATVA 2018, Los Angeles, CA, USA, October 7-10, 2018, Proceedings 16*, pages 528–535. Springer, 2018.

[14] M. Barnett, B.-Y. E. Chang, R. DeLine, B. Jacobs, and K. R. M. Leino. Boogie: A modular reusable verifier for object-oriented programs. In F. S. de Boer, M. M. Bonsangue, S. Graf, and W.-P. de Roever, editors, *Formal Methods for Components and Objects*, pages 364–387, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[15] L. Barrett and S. Moore. cclyzer++: Scalable and precise pointer analysis for llvm. https://galois.com/blog/2022/08/cclyzer-scalable-and-precise-pointer-analysis-for-llvm/, 2022.

[16] M. Berndl, O. Lhoták, F. Qian, L. Hendren, and N. Umanee. Points-to analysis using BDDs. In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation*, PLDI '03, page 103–114, New York, NY, USA, 2003. Association for Computing Machinery.

[17] T. R. U. Book. Trait upcasting. https://doc.rust-lang.org/beta/unstable-book/language-features/trait-upcasting.html, 2024.

[18] K. Boos, N. Liyanage, R. Ijaz, and L. Zhong. Theseus: an experiment in operating system structure and state management. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1–19. USENIX Association, Nov. 2020.

[19] M. Bravenboer and Y. Smaragdakis. Strictly declarative specification of sophisticated points-to analyses. In *Proceedings of the 24th ACM SIGPLAN*

conference on Object oriented programming systems languages and applications, OOPSLA '09, page 243–262, New York, NY, USA, 2009. Association for Computing Machinery.

[20] C. Cadar, D. Dunbar, and D. Engler. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 209–224, USA, 2008. USENIX Association.

[21] D. Clarke, J. Noble, and T. Wrigstad, editors. *Aliasing in Object-Oriented Programming: types, analysis, and verification.* Springer-Verlag, Berlin, Heidelberg, 2013.

[22] E. Clarke, D. Kroening, and F. Lerda. A tool for checking ANSI-C programs. In *Tools and Algorithms for the Construction and Analysis of Systems: 10th International Conference, TACAS 2004, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2004, Barcelona, Spain, March 29-April 2, 2004. Proceedings 10*, pages 168–176. Springer, 2004.

[23] W. Crichton, M. Patrignani, M. Agrawala, and P. Hanrahan. Modular information flow through ownership. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 1–14, New York, NY, USA, 2022. Association for Computing Machinery.

[24] M. Cui, C. Chen, H. Xu, and Y. Zhou. Safedrop: Detecting memory deallocation bugs of Rust programs via static data-flow analysis. *ACM Trans. Softw. Eng. Methodol.*, 32(4), May 2023.

[25] M. Cui, S. Sun, H. Xu, and Y. Zhou. Is unsafe an achilles' heel? a comprehensive study of safety requirements in unsafe Rust programming. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery.

[26] M. Cui, H. Xu, H. Tian, and Y. Zhou. rcanary: Detecting memory leaks across semi-automated memory management boundary in rust. *IEEE Transactions on Software Engineering*, 2024.

[27] L. De Moura and N. Bjørner. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS'08/ETAPS'08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag.

[28] W. Dietl and P. Müller. Universes: Lightweight ownership for jml. *J. Object Technol.*, 4(8):5–32, 2005.

[29] K. Driesen and U. Hölzle. The direct cost of virtual function calls in C++. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA '96, page 306–323, New York, NY, USA, 1996. Association for Computing Machinery.

[30] A. N. Evans, B. Campbell, and M. L. Soffa. Is Rust used safely by software developers? In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, page 246–257, New York, NY, USA, 2020. Association for Computing Machinery.

[31] S. J. Fink, E. Yahav, N. Dor, G. Ramalingam, and E. Geay. Effective typestate verification in the presence of aliasing. *ACM Trans. Softw. Eng. Methodol.*, 17(2), may 2008.

[32] F. Hahn. Rust2viper: Building a static verifier for Rust. Master's thesis, ETH Zürich, 2016.

[33] B. Hardekopf and C. Lin. Exploiting pointer and location equivalence to optimize pointer analysis. In *International Static Analysis Symposium*, pages 265–280. Springer Berlin Heidelberg, 2007.

[34] B. Hardekopf and C. Lin. Flow-sensitive pointer analysis for millions of lines of code. In *Proceedings of the 9th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '11, page 289–298, USA, 2011. IEEE Computer Society.

[35] B. Hassanshahi, R. K. Ramesh, P. Krishnan, B. Scholz, and Y. Lu. An efficient tunable selective points-to analysis for large codebases. In *Proceedings of the 6th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis*, SOAP 2017, page 13–18, New York, NY, USA, 2017. Association for Computing Machinery.

[36] D. He, Y. Gui, W. Li, Y. Tao, C. Zou, Y. Sui, and J. Xue. A container-usage-pattern-based context debloating approach for object-sensitive pointer analysis. *Proc. ACM Program. Lang.*, 7(OOPSLA2), oct 2023.

[37] D. He, J. Lu, Y. Gao, and J. Xue. Accelerating Object-Sensitive Pointer Analysis by Exploiting Object Containment and Reachability. In A. Møller and M. Sridharan, editors, *35th European Conference on Object-Oriented*

Programming (ECOOP 2021), volume 194 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:31, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[38] D. He, J. Lu, and J. Xue. Context debloating for object-sensitive pointer analysis. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 79–91, 2021.

[39] D. He, J. Lu, and J. Xue. Qilin: A New Framework For Supporting Fine-Grained Context-Sensitivity in Java Pointer Analysis. In K. Ali and J. Vitek, editors, *36th European Conference on Object-Oriented Programming (ECOOP 2022)*, volume 222 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 30:1–30:29, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[40] M. Hirzel, D. V. Dincklage, A. Diwan, and M. Hind. Fast online pointer analysis. *ACM Trans. Program. Lang. Syst.*, 29(2):11–es, apr 2007.

[41] S. H. Jeff Vander Stoep. Rust in the Android platform. https://security.googleblog.com/2021/04/rust-in-android-platform.html, 2021.

[42] M. Jeon, S. Jeong, S. Cha, and H. Oh. A machine-learning algorithm with disjunctive model for data-driven program analysis. *ACM Trans. Program. Lang. Syst.*, 41(2), jun 2019.

[43] M. Jeon, S. Jeong, and H. Oh. Precise and scalable points-to analysis via data-driven context tunneling. *Proc. ACM Program. Lang.*, 2(OOPSLA), oct 2018.

[44] M. Jeon, M. Lee, and H. Oh. Learning graph-based heuristics for pointer analysis without handcrafting application-specific features. *Proc. ACM Program. Lang.*, 4(OOPSLA), nov 2020.

[45] M. Jeon and H. Oh. Return of CFA: call-site sensitivity can be superior to object sensitivity even for object-oriented programs. *Proc. ACM Program. Lang.*, 6(POPL), jan 2022.

[46] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. Stacked borrows: an aliasing model for Rust. *Proc. ACM Program. Lang.*, 4(POPL), dec 2019.

[47] G. Kastrinis and Y. Smaragdakis. Hybrid context-sensitivity for points-to analysis. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '13, page 423–434, New York, NY, USA, 2013. Association for Computing Machinery.

[48] O. Lhoták and K.-C. A. Chung. Points-to analysis with efficient strong updates. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, page 3–16, New York, NY, USA, 2011. Association for Computing Machinery.

[49] O. Lhoták and L. Hendren. Scaling Java points-to analysis using Spark. In *Compiler Construction: 12th International Conference, CC 2003 Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2003 Warsaw, Poland, April 7–11, 2003 Proceedings 12*, pages 153–169. Springer, 2003.

[50] L. Li, C. Cifuentes, and N. Keynes. Precise and scalable context-sensitive pointer analysis via value flow graph. In *Proceedings of the 2013 International*

Symposium on Memory Management, ISMM '13, page 85–96, New York, NY, USA, 2013. Association for Computing Machinery.

[51] Y. Li, T. Tan, A. Møller, and Y. Smaragdakis. Precision-guided context sensitivity for pointer analysis. *Proc. ACM Program. Lang.*, 2(OOPSLA), Oct. 2018.

[52] Y. Li, T. Tan, A. Møller, and Y. Smaragdakis. A principled approach to selective context sensitivity for pointer analysis. *ACM Trans. Program. Lang. Syst.*, 42(2), may 2020.

[53] Y. Li, T. Tan, Y. Zhang, and J. Xue. Program Tailoring: Slicing by Sequential Criteria. In S. Krishnamurthi and B. S. Lerner, editors, *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, volume 56 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 15:1–15:27, Dagstuhl, Germany, 2016. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[54] Z. Li, J. Wang, M. Sun, and J. C. Lui. MirChecker: Detecting bugs in Rust programs via static analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 2183–2196, New York, NY, USA, 2021. Association for Computing Machinery.

[55] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. Detecting cross-language memory management issues in Rust. In V. Atluri, R. Di Pietro, C. D. Jensen, and W. Meng, editors, *Computer Security – ESORICS 2022*, pages 680–700, Cham, 2022. Springer Nature Switzerland.

[56] M. Lindner, J. Aparicius, and P. Lindgren. No panic! verification of Rust programs by symbolic execution. In *2018 IEEE 16th International Conference on Industrial Informatics (INDIN)*, pages 108–114, 2018.

[57] P. Liu, G. Zhao, and J. Huang. Securing unsafe rust programs with XRust. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, page 234–245, New York, NY, USA, 2020. Association for Computing Machinery.

[58] B. Livshits, M. Sridharan, Y. Smaragdakis, O. Lhoták, J. N. Amaral, B.-Y. E. Chang, S. Z. Guyer, U. P. Khedker, A. Møller, and D. Vardoulakis. In defense of soundiness: a manifesto. *Commun. ACM*, 58(2):44–46, Jan. 2015.

[59] V. B. Livshits and M. S. Lam. Finding security vulnerabilities in Java applications with static analysis. In *Proceedings of the 14th Conference on USENIX Security Symposium - Volume 14*, SSYM'05, page 18, USA, 2005. USENIX Association.

[60] J. Lu, D. He, and J. Xue. Eagle: CFL-reachability-based precision-preserving acceleration of object-sensitive pointer analysis with partial context sensitivity. *ACM Trans. Softw. Eng. Methodol.*, 30(4), July 2021.

[61] J. Lu, D. He, and J. Xue. Selective context-sensitivity for k-CFA with CFL-reachability. In *Static Analysis: 28th International Symposium, SAS 2021, Chicago, IL, USA, October 17–19, 2021, Proceedings*, page 261–285, Berlin, Heidelberg, 2021. Springer-Verlag.

[62] J. Lu and J. Xue. Precision-preserving yet fast object-sensitive pointer analysis with partial context sensitivity. *Proc. ACM Program. Lang.*, 3(OOPSLA), Oct. 2019.

[63] W. Ma, S. Yang, T. Tan, X. Ma, C. Xu, and Y. Li. Context sensitivity without contexts: A cut-shortcut approach to fast and precise pointer analysis. *Proc. ACM Program. Lang.*, 7(PLDI), jun 2023.

[64] M. Might, Y. Smaragdakis, and D. Van Horn. Resolving and exploiting the k-CFA paradox: illuminating functional vs. object-oriented program analysis. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '10, page 305–315, New York, NY, USA, 2010. Association for Computing Machinery.

[65] A. Milanova, A. Rountev, and B. G. Ryder. Parameterized object sensitivity for points-to and side-effect analyses for Java. In *Proceedings of the 2002 ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '02, page 1–11, New York, NY, USA, 2002. Association for Computing Machinery.

[66] A. Milanova, A. Rountev, and B. G. Ryder. Parameterized object sensitivity for points-to analysis for Java. *ACM Trans. Softw. Eng. Methodol.*, 14(1):1–41, Jan. 2005.

[67] J. Min, D. Yu, S. Jeong, D. Song, and Y. Jeon. ERASan: Efficient Rust Address Sanitizer . In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4053–4068, Los Alamitos, CA, USA, May 2024. IEEE Computer Society.

[68] MIRAI. Mirai: Rust mid-level ir abstract interpreter. https://github.com/facebookexperimental/MIRAI, 2024.

[69] MIRI. Miri: An interpreter for Rust's mid-level intermediate representation. https://github.com/rustlang/miri, 2024.

[70] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *Proceedings of the 17th International Conference on Verification, Model Checking, and Abstract Interpretation - Volume 9583*, VMCAI 2016, page 41–62, Berlin, Heidelberg, 2016. Springer-Verlag.

[71] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev. RedLeaf: Isolation and communication in a safe operating system. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 21–39. USENIX Association, Nov. 2020.

[72] D. J. Pearce, P. H. Kelly, and C. Hankin. Efficient field-sensitive pointer analysis of C. *ACM Trans. Program. Lang. Syst.*, 30(1):4–es, Nov. 2007.

[73] F. M. Q. Pereira and D. Berlin. Wave propagation and deep propagation for pointer analysis. In *Proceedings of the 7th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '09, page 126–135, USA, 2009. IEEE Computer Society.

[74] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang. Understanding and detecting real-world safety issues in rust. *IEEE Transactions on Software Engineering*, 50(06):1306–1324, June 2024.

[75] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. Understanding memory and thread safety practices and issues in real-world Rust programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, page 763–779, New York, NY, USA, 2020. Association for Computing Machinery.

[76] Z. Rakamarić and M. Emmi. Smack: Decoupling source language details from verifier implementations. In A. Biere and R. Bloem, editors, *Computer Aided Verification*, pages 106–113, Cham, 2014. Springer International Publishing.

[77] A. Rountev and S. Chandra. Off-line variable substitution for scaling points-to analysis. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*, PLDI '00, page 47–56, New York, NY, USA, 2000. Association for Computing Machinery.

[78] O. Sallenave and R. Ducournau. Lightweight generics in embedded systems through static analysis. In *Proceedings of the 13th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, Tools and Theory for Embedded Systems*, LCTES '12, page 11–20, New York, NY, USA, 2012. Association for Computing Machinery.

[79] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov. AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 309–318, Boston, MA, June 2012. USENIX Association.

[80] M. Sharir, A. Pnueli, et al. *Two approaches to interprocedural data flow analysis*. New York University. Courant Institute of Mathematical Sciences ..., 1978.

[81] Q. Shi, X. Xiao, R. Wu, J. Zhou, G. Fan, and C. Zhang. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2018, page 693–706, New York, NY, USA, 2018. Association for Computing Machinery.

[82] O. G. Shivers. *Control-flow analysis of higher-order languages of taming lambda*. Carnegie Mellon University, USA, 1991. UMI Order No. GAX91-26964.

[83] Y. Smaragdakis, G. Balatsouras, et al. Pointer analysis. *Foundations and Trends® in Programming Languages*, 2(1):1–69, 2015.

[84] Y. Smaragdakis, M. Bravenboer, and O. Lhoták. Pick your contexts well: understanding object-sensitivity. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, page 17–30, New York, NY, USA, 2011. Association for Computing Machinery.

[85] M. Sridharan and R. Bodík. Refinement-based context-sensitive points-to analysis for java. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '06, page 387–400, New York, NY, USA, 2006. Association for Computing Machinery.

[86] M. Sridharan, D. Gopan, L. Shan, and R. Bodík. Demand-driven points-to analysis for Java. In *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA '05, page 59–76, New York, NY, USA, 2005. Association for Computing Machinery.

[87] B. Steensgaard. Points-to analysis in almost linear time. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '96, page 32–41, New York, NY, USA, 1996. Association for Computing Machinery.

[88] Y. Sui and J. Xue. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th International Conference on Compiler Construction*, CC '16, page 265–266, New York, NY, USA, 2016. Association for Computing Machinery.

[89] Y. Sui, D. Ye, and J. Xue. Static memory leak detection using full-sparse value-flow analysis. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis*, ISSTA 2012, page 254–264, New York, NY, USA, 2012. Association for Computing Machinery.

[90] Y. Sui, S. Ye, J. Xue, and J. Zhang. Making context-sensitive inclusion-based pointer analysis practical for compilers using parameterised summarisation. *Software: Practice and Experience*, 44(12):1485–1510, 2014.

[91] V. Sundaresan, L. Hendren, C. Razafimahefa, R. Vallée-Rai, P. Lam, E. Gagnon, and C. Godin. Practical virtual method call resolution for Java. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA '00, page 264–280, New York, NY, USA, 2000. Association for Computing Machinery.

[92] T. Tan, Y. Li, and J. Xue. Making k-object-sensitive pointer analysis more precise with still k-limiting. In *International Static Analysis Symposium*, pages 489–510. Springer, Springer Berlin Heidelberg, 2016.

[93] T. Tan, Y. Li, and J. Xue. Efficient and precise points-to analysis: modeling the heap by merging equivalent automata. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, page 278–291, New York, NY, USA, 2017. Association for Computing Machinery.

[94] F. Tip and J. Palsberg. Scalable propagation-based call graph construction algorithms. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA '00, page 281–293, New York, NY, USA, 2000. Association for Computing Machinery.

[95] J. Toman, S. Pernsteiner, and E. Torlak. Crust: A bounded verifier for Rust (N). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 75–80, 2015.

[96] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan. Soot: a Java bytecode optimization framework. In *CASCON First Decade High Impact Papers*, CASCON '10, page 214–224, USA, 2010. IBM Corp.

[97] A. VanHattum, D. Schwartz-Narbonne, N. Chong, and A. Sampson. Verifying dynamic trait objects in Rust. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, ICSE-SEIP '22, page 321–330, New York, NY, USA, 2022. Association for Computing Machinery.

[98] P. Wadler. Linear types can change the world! In *Programming Concepts and Methods*, 1990.

[99] WALA. WALA: T.J. Watson Libraries for Analysis. http://wala.sourceforge.net/, 2023.

[100] J. Whaley and M. S. Lam. Cloning-based context-sensitive pointer alias analysis using binary decision diagrams. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implemen-*

tation, PLDI '04, page 131–144, New York, NY, USA, 2004. Association for Computing Machinery.

[101] C. Wimmer, C. Stancu, D. Kozak, and T. Würthinger. Scaling type-based points-to analysis with saturation. *Proc. ACM Program. Lang.*, 8(PLDI), June 2024.

[102] F. Wolff, A. Bílý, C. Matheja, P. Müller, and A. J. Summers. Modular specification and verification of closures in Rust. *Proc. ACM Program. Lang.*, 5(OOPSLA), Oct. 2021.

[103] D. Wu, D. He, S. Chen, and J. Xue. Exposing Android event-based races by selective branch instrumentation. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, pages 265–276, 2020.

[104] X. Xiao and C. Zhang. Geometric encoding: forging the high performance context sensitive points-to analysis for Java. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis*, ISSTA '11, page 188–198, New York, NY, USA, 2011. Association for Computing Machinery.

[105] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. Memory-safety challenge considered solved? an in-depth study with all Rust CVEs. *ACM Trans. Softw. Eng. Methodol.*, 31(1), Sept. 2021.

[106] H. Yan, Y. Sui, S. Chen, and J. Xue. Spatio-temporal context reduction: a pointer-analysis-based static approach for detecting use-after-free vulnerabilities. In *Proceedings of the 40th International Conference on Software Engineering*, ICSE '18, page 327–337, New York, NY, USA, 2018. Association for Computing Machinery.

[107] Z. Zhang, B. Qin, Y. Chen, L. Song, and Y. Zhang. VRLifeTime – an IDE tool to avoid concurrency and memory bugs in Rust. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 2085–2087, New York, NY, USA, 2020. Association for Computing Machinery.